HISPÂNIA

De Atapuerca a La Moncloa

Guia de Programação de Jogos do Zero

Tudo o que foi feito no jogo, como foi feito e por quê — explicado para quem nunca escreveu uma linha de código. Cada pasta, cada arquivo, cada função.

HTML · CSS · JavaScript · Canvas 2D · Web Audio · localStorage · GitHub Pages

Os trechos de código deste guia foram extraídos automaticamente do código-fonte real do jogo.

Sumário

Como usar este guia

O caminho de leitura
As caixas coloridas
Os blocos de código

Capítulo 1 Primeiros passos: o que é programar

1.1 O que é um programa
1.2 O que é um jogo, do ponto de vista do programador
1.3 As ferramentas que você precisa
1.4 Seu primeiro programa

Capítulo 2 Como a web funciona: HTML e CSS

2.1 O caminho de um arquivo até a tela
2.2 HTML: etiquetas, atributos e aninhamento
2.3 O index.html do jogo, linha a linha
2.4 CSS: seletores, propriedades e caixas

Capítulo 3 JavaScript do zero

3.1 Instruções, comentários e o Console
3.2 Variáveis: dando nome aos valores
3.3 Tipos de valores
3.4 Comparações e lógica
3.5 Decisões: if, else e o operador ternário
3.6 Repetições: for e while
3.7 Funções: receitas reutilizáveis
3.8 Listas (arrays)
3.9 Objetos: agrupando informações
3.10 Classes: moldes para criar objetos
3.11 Objetos com funções: o padrão "módulo"
3.12 Eventos: reagindo ao jogador
3.13 JSON: transformando objetos em texto
3.14 Erros e try/catch
3.15 Escopo: onde cada nome existe
3.16 'use strict'
3.17 Resumo do capítulo

Capítulo 4 Por que cada tecnologia foi escolhida

4.1 O que é uma biblioteca, um framework e um motor de jogo
4.2 Os critérios usados
4.3 Desenho: Canvas 2D em vez de Phaser, PixiJS ou Unity
4.4 Linguagem: JavaScript puro em vez de TypeScript
4.5 Organização: scripts clássicos em vez de módulos ES e sem "bundler"
4.6 Interface: HTML e CSS em vez de React ou desenhar no canvas
4.7 Som: Web Audio API em vez de Howler.js ou Tone.js
4.8 Salvamento: localStorage em vez de SQLite, IndexedDB ou um servidor
4.9 Tradução: um objeto de textos em vez de i18next
4.10 Ferramentas de bastidor (fora do jogo)
4.11 Quando você DEVE usar uma biblioteca

Capítulo 5 A arquitetura do projeto

5.1 A árvore de pastas e arquivos
5.2 O princípio mais importante: separar dados de lógica

5.3 A ordem de carregamento e quem depende de quem
5.4 Os objetos globais principais
5.5 O fluxo do jogo, do clique ao laço

Capítulo 6 O laço do jogo, o Canvas e a matemática dos jogos

6.1 O laço: requestAnimationFrame
6.2 O Canvas: sua tela de pintura
6.3 Transformações: o segredo para desenhar personagens
6.4 A matemática essencial dos jogos
6.5 A câmera
6.6 A renderização: a ordem importa

Capítulo 7 Gerando mundos: mapas procedurais

7.1 Mapas de tiles
7.2 Aleatoriedade controlada: sementes
7.3 Ruído: florestas que parecem naturais
7.4 A geração passo a passo: World.generate
7.5 Colisão com o mapa
7.6 Desenhando o mapa rápido: o cache em blocos (chunks)

Capítulo 8 Controles, movimento e combate

8.1 Lendo o teclado
8.2 A função update: um quadro de lógica
8.3 Ataques: três tipos de arma
8.4 Projéteis e colisão círculo com círculo
8.5 Dano, invulnerabilidade e morte
8.6 Interagindo com o mundo

Capítulo 9 Inteligência artificial dos inimigos

9.1 A ficha de um inimigo
9.2 Estados: ocioso ou em alerta
9.3 Os quatro comportamentos
9.4 Os chefes: padrões de ataque
9.5 Acampamentos e reaparecimento (respawn)
9.6 Progressão e equilíbrio (balanceamento)

Capítulo 10 Missões, capítulos e dados

10.1 Os dados base: armas, classes e inimigos
10.2 A estrutura de um capítulo
10.3 A missão: uma lista de etapas
10.4 O motor de missões
10.5 Entrando e saindo de capítulos

Capítulo 11 A interface: HUD, diálogos e janelas

11.1 Manipulando o HTML pelo JavaScript
11.2 O HUD
11.3 Notificações (toasts)
11.4 O sistema de diálogo
11.5 Janelas (modais)
11.6 Segurança: escapeHtml
11.7 Roteando o teclado
11.8 A tela de título e os espaços de save

Capítulo 12 Salvando o progresso

12.1 localStorage
12.2 Os cinco espaços
12.3 Migrando saves antigos

12.4 Excluir, exportar e importar

Capítulo 13 Dois idiomas: internacionalização

13.1 Qual idioma usar
13.2 Textos da interface: a função T
13.3 O conteúdo: tradução por sobreposição

Capítulo 14 Som e música gerados por código

14.1 O que é som, para um computador
14.2 A Web Audio API: um estúdio modular
14.3 Os efeitos sonoros: o objeto Sfx
14.4 Teoria musical mínima
14.5 Os temas
14.6 A composição automática
14.7 O agendador: tocar no tempo certo
14.8 Os instrumentos
14.9 Como testar áudio sem ouvir

Capítulo 15 Testar, depurar e publicar

15.1 Depurando com o DevTools
15.2 Os testes automáticos usados no projeto
15.3 Um servidor local mínimo
15.4 Git: guardando o histórico
15.5 Publicando no GitHub Pages

Capítulo 16 Mãos à obra: estendendo o jogo

16.1 Tutorial 1: um novo inimigo
16.2 Tutorial 2: uma nova missão com quiz
16.3 Tutorial 3: uma nova arma
16.4 Tutorial 4: um novo tipo de chapéu
16.5 Tutorial 5: um novo tema musical
16.6 Tutorial 6: um capítulo inteiro
16.7 Tutorial 7: uma mecânica nova (desafio)
16.8 Para onde ir depois deste guia

Capítulo 17 Referência completa: cada função do jogo

17.1 js/util.js: utilidades
17.2 js/i18n.js: idiomas
17.3 js/world.js: o mapa
17.4 js/entities.js: desenho e entidades
17.5 js/game.js: o núcleo
17.6 js/ui.js: a interface
17.7 js/music.js: a música

Glossário

Palavras finais

Como usar este guia

Este guia tem dois objetivos ao mesmo tempo. O primeiro é **ensinar a programar do zero**, sem supor que você já saiba qualquer coisa. O segundo é **explicar, peça por peça, como o jogo Hispânia foi construído**: cada pasta, cada arquivo, cada função e cada decisão.

Os dois objetivos andam juntos de propósito. É muito mais fácil aprender programação com um projeto real e divertido do que com exemplos soltos como "calcule a média de três números". Aqui, cada conceito novo aparece primeiro de forma simples e, logo depois, você vê **onde ele foi usado no jogo**.

O caminho de leitura

O guia está dividido em quatro partes:

1. **Fundamentos (capítulos 1 a 4).** O que é programar, as ferramentas, como a web funciona, a linguagem JavaScript do zero e o porquê de cada tecnologia escolhida.
2. **O motor do jogo (capítulos 5 a 9).** A arquitetura, o laço do jogo, o desenho na tela, a geração dos mapas, os controles, o combate e a inteligência dos inimigos.
3. **Os sistemas (capítulos 10 a 14).** Missões e dados, interface, salvamento, idiomas, som e música.
4. **Prática (capítulos 15 a 17).** Testes, publicação na internet, tutoriais para você estender o jogo sozinho e uma referência completa de todas as funções.

Leia na ordem. Os capítulos se apoiam uns nos outros. Se você já sabe um pouco de programação, pode passar rápido pelos capítulos 2 e 3, mas não os pule: eles também mostram detalhes do jogo.

As caixas coloridas

Ao longo do texto você vai encontrar caixas como estas:

> 💡 **Dica**
>
> Atalhos, truques e boas práticas que facilitam a sua vida.

> ⚠️ **Atenção**
>
> Erros comuns que quase todo iniciante comete. Leia com calma.

> ❓ **Por quê?**
>
> Explicações sobre **por que** algo foi feito de um jeito e não de outro. Programar é, em grande parte, tomar decisões. Estas caixas mostram o raciocínio.

> ✍️ **Exercício**
>
> Pequenos desafios para você fazer **de verdade**, digitando o código. Não pule. Só se aprende a programar programando.

> ✅ **Resposta**
>
> A solução dos exercícios, para você conferir depois de tentar.

> 🐞 **História de um bug**
>
> Histórias reais de erros que apareceram durante a construção deste jogo e como foram resolvidos.

Os blocos de código

Há dois tipos de bloco de código. Os que têm uma **barra marrom no topo**, com o nome do arquivo e o número das linhas, foram **copiados automaticamente do código real do jogo** no momento em que este PDF foi gerado. Eles são exatamente o que roda no jogo:

js/util.js — linhas 39–39

```
const clamp = (v, a, b) => (v < a ? a : v > b ? b : v);
```

Os blocos **sem a barra** são exemplos escritos para ensinar, que você pode digitar e testar:

```
// Isto é um exemplo didático, não faz parte do jogo
const nome = 'Íber';
console.log('Olá, ' + nome);
```

> 💡 **Dica**
>
> Não tente decorar nada. Programadores profissionais consultam documentação o tempo todo. O importante é **entender a ideia**. A sintaxe exata você consulta quando precisar.

CAPÍTULO 1

Primeiros passos: o que é programar

1.1 O que é um programa

Um computador é uma máquina extremamente rápida e extremamente literal. Ele não "entende" nada: ele executa instruções, uma depois da outra, exatamente como estão escritas. Um **programa** é uma lista dessas instruções.

Imagine que você escreve uma receita de bolo para alguém que nunca entrou numa cozinha e que faz **tudo ao pé da letra**. Se você escrever "bata os ovos", essa pessoa vai perguntar: quantos ovos? Com o quê? Por quanto tempo? Programar é escrever receitas tão precisas que não sobra dúvida nenhuma.

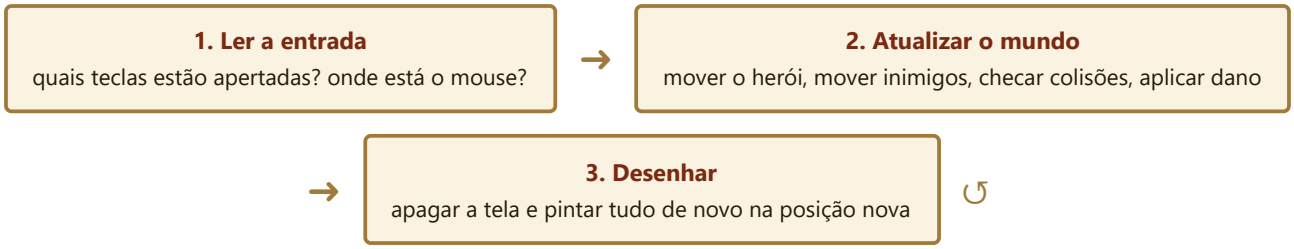
Essas receitas são escritas numa **linguagem de programação**. Existem centenas: Python, C, Java, C#, Lua, Rust... Este jogo usa três linguagens da web, cada uma com um papel:

Linguagem	Papel	Analogia
HTML	Diz **o que existe** na página: um título, um botão, uma área de desenho	O esqueleto de um corpo
CSS	Diz **como as coisas aparecem**: cores, tamanhos, posições, fontes	A roupa e a maquiagem
JavaScript	Diz **o que acontece**: a lógica, os cálculos, as reações aos cliques	O cérebro e os músculos

HTML e CSS não são exatamente linguagens de *programação* (elas descrevem, não calculam). A programação de verdade, o "cérebro" do jogo, está toda em **JavaScript**, e é nela que este guia vai se concentrar.

1.2 O que é um jogo, do ponto de vista do programador

Todo jogo de computador, do Pong de 1972 ao jogo mais moderno, funciona com a mesma ideia central, o **laço do jogo** (em inglês, *game loop*). Dezenas de vezes por segundo o programa repete:



Cada repetição se chama **quadro** (*frame*). Se o laço roda 60 vezes por segundo, dizemos que o jogo roda a **60 FPS** (*frames per second*, quadros por segundo). O olho humano vê essa sequência de imagens paradas como movimento, exatamente como num desenho animado.

Guarde essa ideia. Quase tudo neste guia é uma das três etapas: ler entrada, atualizar, desenhar.

1.3 As ferramentas que você precisa

Para programar este jogo você precisa de poucas coisas, e todas são gratuitas.

1. Um navegador moderno

Chrome, Edge ou Firefox. O navegador é o "motor" que executa o jogo. Ele já vem com tudo: um interpretador de JavaScript, uma área de desenho (Canvas), um sistema de som (Web Audio) e um pequeno banco de dados (localStorage). Por isso este jogo não precisou instalar nenhuma biblioteca, como você verá no capítulo 4.

2. Um editor de código

Você pode escrever código até no Bloco de Notas, mas um editor de verdade ajuda muito: ele colore o código, completa palavras e aponta erros. O mais usado no mundo é o **Visual Studio Code** (VS Code), gratuito, em `code.visualstudio.com`.

> 🛠️ **Mão na massa**
>
> Instale o VS Code. Depois use **Arquivo → Abrir Pasta...** e escolha a pasta `game`. À esquerda aparece a árvore de arquivos do projeto. Clique em `js/game.js` para ver o código principal.

3. As Ferramentas de Desenvolvedor do navegador (DevTools)

Este é o seu melhor amigo. Com o jogo aberto no navegador, aperte **F12**. Abre um painel com várias abas. As mais importantes:

- **Console**: mostra mensagens e erros, e deixa você digitar JavaScript e ver o resultado na hora.
- **Elements**: mostra o HTML da página "ao vivo". Você pode mudar cores e textos para testar.
- **Sources**: deixa você pausar o jogo numa linha específica (*breakpoint*) e inspecionar variáveis.
- **Application**: mostra o que o jogo salvou no navegador (os saves ficam em *Local Storage*).

> 🛠️ **Mão na massa**
>
> Abra o jogo, aperte F12, vá na aba **Console** e digite `G.S.gold = 9999` e depois Enter (com uma partida em andamento). Você acabou de mudar o dinheiro do herói "por dentro". `G` é o objeto principal do jogo; vamos estudá-lo no capítulo 5.

4. Node.js (opcional)

O Node.js é um programa que executa JavaScript **fora** do navegador. O jogo não precisa dele para funcionar. Ele foi usado só para duas tarefas de bastidor: rodar um pequeno servidor de testes (`.claude/serve.js`) e gerar este PDF (`guia/build.js`).

5. Git e GitHub (para publicar)

O Git guarda o histórico de versões do seu código; o GitHub hospeda esse histórico na internet e, com o **GitHub Pages**, publica o jogo de graça. O capítulo 15 ensina o passo a passo.

1.4 Seu primeiro programa

Antes de mergulhar no jogo, faça o clássico "Olá, mundo". Crie uma pasta qualquer, e dentro dela um arquivo chamado `ola.html` com este conteúdo:

```
<!doctype html>
<"f">class="k">html>
  <"f">class="k">head>
    <"f">class="k">meta "f">charset="utf-8">
    <"f">class="k">title>Meu primeiro programa</"f">class="k">title>
  </"f">class="k">head>
  <"f">class="k">body>
    <"f">class="k">h1>Olá, mundo!</"f">class="k">h1>
    <"f">class="k">button "f">id="botao">Clique em mim</"f">class="k">button>
    <"f">class="k">script>
      const botao = document.getElementById('botao');
      let cliques = 0;
      botao.onclick = () => {
        cliques = cliques + 1;
        botao.textContent = 'Você clicou ' + cliques + ' vezes';
      };
    </"f">class="k">script>
  </"f">class="k">body>
</"f">class="k">html>
```

Dê dois cliques no arquivo para abri-lo no navegador e clique no botão. Repare que as três linguagens apareceram juntas: o **HTML** criou o título e o botão; o **JavaScript** (dentro de `<script>`) reagiu ao clique. Não se preocupe em entender cada palavra agora: os próximos capítulos explicam tudo.

> ✍️ **Exercício**
>
> Mude o programa para que o texto do botão diga "Você clicou 1 vez" (no singular) quando `cliques` for 1, e "vezes" nos outros casos. Dica: você vai precisar de um `if`, que veremos no capítulo 3. Tente depois de ler a seção sobre decisões.

CAPÍTULO 2

Como a web funciona: HTML e CSS

2.1 O caminho de um arquivo até a tela

Quando você abre `index.html`, o navegador faz, em ordem:

1. **Lê o HTML** de cima para baixo e monta uma árvore de elementos na memória, chamada **DOM** (*Document Object Model*). Cada etiqueta (`<div>`, `<button>`, `<canvas>`) vira um "objeto" que o JavaScript pode ler e modificar.
2. **Lê o CSS** e calcula a aparência de cada elemento (cor, tamanho, posição).
3. **Executa cada** `<script>` no momento em que o encontra, na ordem em que aparecem.
4. **Desenha** a página e fica esperando eventos: cliques, teclas, mudanças de tamanho da janela...

Esse detalhe do item 3 é importante para o jogo: os scripts rodam **na ordem** em que aparecem. Um script só pode usar o que os scripts anteriores já criaram. Você verá isso na prática ao estudar o `index.html`.

2.2 HTML: etiquetas, atributos e aninhamento

HTML é feito de **etiquetas** (*tags*). A maioria vem em pares, uma de abertura e uma de fechamento, envolvendo um conteúdo:

```
<"f">class="k">p "f">class="aviso" "f">id="msg">Este é um parágrafo.</"f">class="k">p>
```

- `p` é o **nome** da etiqueta (parágrafo).
- `class="aviso"` e `id="msg"` são **atributos**: informações extras. O `id` é um nome único para achar o elemento pelo JavaScript; a `class` agrupa elementos que compartilham o mesmo estilo CSS.
- Etiquetas podem ficar **umas dentro das outras** (aninhamento), formando a árvore do DOM.

As etiquetas usadas no jogo são poucas:

Etiqueta	Para que serve	Onde aparece no jogo
`<canvas>`	Uma área retangular onde o JavaScript desenha pixels livremente	O mundo do jogo inteiro
`<div>`	Uma "caixa" genérica para agrupar e posicionar coisas	HUD, diálogos, janelas
`<span>`	Uma caixa pequena dentro de uma linha de texto	Números do HUD
`<button>`	Um botão clicável	Menus, loja, quiz
`<input>`	Um campo onde o jogador digita	Nome da linhagem
`<script>`	Carrega ou contém JavaScript	Todos os arquivos `.js`
`<link>`	Carrega um arquivo CSS	`css/style.css`

2.3 O index.html do jogo, linha a linha

O arquivo `index.html` é a porta de entrada. Ele é curto, porque quase tudo é criado pelo JavaScript:

index.html — linhas 1–56

```
<!doctype html>
<"f">class="k">html "f">lang="pt-BR">
<"f">class="k">head>
  <"f">class="k">meta "f">charset="utf-8">
  <"f">class="k">meta "f">name="viewport" "f">content="width=device-width, initial-scale=1">
  <"f">class="k">title>Hispânia — De Atapuerca a La Moncloa</"f">class="k">title>
  <"f">class="k">meta "f">name="description" "f">content="RPG 2D que conta a história da Espanha, da pré-
história até hoje.">
  <"f">class="k">link "f">rel="stylesheet" "f">href="css/style.css">
</"f">class="k">head>
<"f">class="k">body>
  <"f">class="k">canvas "f">id="game"></"f">class="k">canvas>

  <"f">class="k">div "f">id="hud" "f">class="hidden">
    <"f">class="k">div "f">class="panel stats">
      <"f">class="k">div "f">class="hname"><"f">class="k">b "f">id="h-name"></"f">class="k">b> ·
<"f">class="k">span "f">id="h-class"></"f">class="k">span></"f">class="k">div>
      <"f">class="k">div "f">class="bar hp"><"f">class="k">div "f">id="h-hp"></"f">class="k">div>
<"f">class="k">span "f">id="h-hpt"></"f">class="k">span></"f">class="k">div>
      <"f">class="k">div "f">class="bar xp"><"f">class="k">div "f">id="h-xp"></"f">class="k">div>
<"f">class="k">span "f">id="h-xpt"></"f">class="k">span></"f">class="k">div>
      <"f">class="k">div "f">class="hrow"><"f">class="k">span "f">id="h-gold"></"f">class="k">span>
<"f">class="k">span "f">id="h-pot"></"f">class="k">span></"f">class="k">div>
      <"f">class="k">div "f">class="hrow"><"f">class="k">span "f">id="h-ammo"></"f">class="k">span>
</"f">class="k">div>
    </"f">class="k">div>
    <"f">class="k">div "f">class="panel tracker" "f">id="tracker"></"f">class="k">div>
    <"f">class="k">div "f">class="chapter" "f">id="h-chapter"></"f">class="k">div>
    <"f">class="k">div "f">class="keys" "f">id="keysbar"></"f">class="k">div>
  </"f">class="k">div>

  <"f">class="k">div "f">id="toasts"></"f">class="k">div>

  <"f">class="k">div "f">id="dialog" "f">class="hidden">
    <"f">class="k">div "f">class="who"></"f">class="k">div>
    <"f">class="k">div "f">class="text"></"f">class="k">div>
    <"f">class="k">div "f">class="opts"></"f">class="k">div>
    <"f">class="k">div "f">class="hint"></"f">class="k">div>
  </"f">class="k">div>

  <"f">class="k">div "f">id="modal" "f">class="hidden"><"f">class="k">div "f">class="box">
</"f">class="k">div></"f">class="k">div>

  <"f">class="k">div "f">id="title" "f">class="hidden"></"f">class="k">div>

  <"f">class="k">script "f">src="js/util.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/i18n.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/base.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/chapters1.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/chapters2.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/chapters3.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/es_base.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/es1.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/data/es2.js"></"f">class="k">script>
```

```
  <"f">class="k">script "f">src="js/data/es3.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/music.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/world.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/entities.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/game.js"></"f">class="k">script>
  <"f">class="k">script "f">src="js/ui.js"></"f">class="k">script>
  <"f">class="k">script>applyLanguage(); UI.staticTexts(); UI.title();</"f">class="k">script>
</"f">class="k">body>
</"f">class="k">html>
```

O que cada parte faz:

- `<!doctype html>` avisa o navegador que este é um documento HTML moderno.
- `<html lang="pt-BR">` é a raiz da árvore. O `lang` ajuda leitores de tela e tradutores automáticos (o jogo troca para `es-ES` quando você escolhe espanhol).
- `<meta charset="utf-8">` diz que o texto usa a codificação UTF-8, que suporta acentos, "ç" e emojis. Sem isso, "Espanha" poderia aparecer certo, mas "Hispânia" viraria lixo.
- `<meta name="viewport" ...>` faz a página se ajustar à largura de celulares.
- `<link rel="stylesheet" href="css/style.css">` carrega o CSS.
- `<canvas id="game">` é a tela de pintura onde o mundo é desenhado.
- `<div id="hud">` é o HUD (*heads-up display*): os painéis de vida, XP, dinheiro e missões, que ficam **por cima** do canvas. A classe `hidden` esconde tudo até o jogo começar.
- `#toasts`, `#dialog`, `#modal`, `#title` são "contêineres vazios" que o JavaScript preenche: notificações, caixa de diálogo, janelas (loja, diário, mapa) e a tela de título.
- **A lista de** `<script>` carrega o JavaScript **em uma ordem pensada**: primeiro as ferramentas gerais (`util.js`, `i18n.js`), depois os dados (`base.js`, capítulos, traduções), depois os sistemas que usam esses dados (`music.js`, `world.js`, `entities.js`, `game.js`) e por último a interface (`ui.js`).
- **O último** `<script>` dá a partida: aplica o idioma, ajusta os textos fixos e mostra a tela de título.

> ❓ **Por quê?**
>
> **Por que o HUD é HTML e não desenhado no canvas?** Textos, botões, rolagem e quebra de linha automática já vêm prontos no HTML. Desenhar uma janela de loja com botões clicáveis direto no canvas exigiria programar tudo isso à mão: detectar onde o mouse clicou, quebrar o texto em linhas, desenhar a barra de rolagem... Usar HTML por cima do canvas é uma técnica comum em jogos de navegador e economiza centenas de linhas.

2.4 CSS: seletores, propriedades e caixas

Uma regra CSS tem um **seletor** (quem será estilizado) e uma lista de **propriedades** (o quê):

```
.toast.main {           /* seletor: elementos com as classes toast E main */
  border-color: #ffd84a; /* propriedade: valor; */
  color: #ffd84a;
}
```

Seletores mais usados:

Seletor	Significa	Exemplo no jogo
`button`	Todos os elementos `<button>`	Estilo dourado de todos os botões
`.panel`	Todos com `class="panel"`	Painéis do HUD
`#dialog`	O elemento com `id="dialog"`	A caixa de diálogo
`#dialog .who`	Elementos `.who` **dentro** de `#dialog`	O nome de quem fala
`button:hover`	Botões com o mouse em cima	Brilho ao passar o mouse
`.hidden`	Qualquer elemento com a classe `hidden`	Esconder telas

As cores aparecem em **hexadecimal**: `#ffd84a` significa vermelho `ff` (255), verde `d8` (216), azul `4a` (74), que dá um dourado. Cada par vai de `00` (nada) a `ff` (máximo).

O truque da classe `hidden`

Esta regra, sozinha, controla todas as telas do jogo:

css/style.css — linhas 4–4

```
.hidden { display: none !important; }
```

Para esconder qualquer coisa, o JavaScript adiciona a classe `hidden`; para mostrar, remove. Por exemplo, em `ui.js`: `$('dialog').classList.add('hidden')`. O `!important` garante que essa regra vença qualquer outra.

Posicionamento: como o HUD fica por cima do jogo

css/style.css — linhas 15–15

```
#hud { position: fixed; inset: 0; pointer-events: none; }
```

css/style.css — linhas 16–16

```
.panel { position: absolute; background: rgba(20, 14, 8, 0.78); border: 1px solid #6a5030; border-radius: 8px; padding: 8px 10px; font-size: 13px; }
```

- `position: fixed; inset: 0;` faz o HUD ocupar a janela inteira, preso à tela.
- `pointer-events: none;` é o detalhe esperto: o HUD **deixa os cliques passarem** para o canvas embaixo. Sem isso, o jogador não conseguiria atacar clicando no mundo, porque o HUD "engoliria" os cliques.
- Cada `.panel` usa `position: absolute` com `top` / `left` / `right` para se prender a um canto.

Flexbox: alinhando coisas em linha ou coluna

Muitos componentes do jogo usam `display: flex`, um modo de layout que alinha os filhos em linha (ou coluna) e distribui o espaço:

css/style.css — linhas 36–36

```
#toasts { position: fixed; top: 44px; left: 50%; transform: translateX(-50%); display: flex; flex-direction: column; align-items: center; gap: 5px; pointer-events: none; z-index: 5; }
```

Essa linha centraliza as notificações (*toasts*) no topo e as empilha em coluna, com 5 pixels entre elas.

Responsividade

No fim do CSS, uma **media query** aplica regras diferentes em telas estreitas:

```
@media (max-width: 700px) {
  .keys, .chapter { display: none; }   /* esconde a barra de teclas no celular */
}
```

> ✍️ **Exercício**
>
> Abra `css/style.css`, encontre a regra `.toast.level` e troque `background: rgba(90, 60, 0, 0.9)` por `background: rgba(0, 60, 120, 0.9)`. Recarregue o jogo (F5) e suba de nível. A notificação de nível agora é azul. Parabéns: você modificou o jogo.

> 💡 **Dica**
>
> No DevTools (F12), aba **Elements**, clique num elemento e edite o CSS dele no painel da direita. A mudança aparece na hora, sem salvar nada. É a forma mais rápida de experimentar visuais; quando gostar, copie o valor para o arquivo `.css`.

CAPÍTULO 3

JavaScript do zero

Este é o capítulo mais importante para quem nunca programou. Vamos construir o conhecimento em camadas: primeiro guardar valores, depois tomar decisões, repetir tarefas, agrupar instruções em funções e organizar dados em listas e objetos. A cada conceito, você verá onde ele aparece no Hispânia.

> 🛠️ **Mão na massa**
>
> Para testar os exemplos, abra qualquer página no navegador, aperte **F12**, vá na aba **Console** e digite (ou cole) o código. Aperte Enter para executar. O Console é o seu laboratório.

3.1 Instruções, comentários e o Console

Um programa JavaScript é uma sequência de **instruções**. Normalmente cada uma fica numa linha e termina com ponto e vírgula `;`:

```
console.log('Olá!');   // escreve "Olá!" no Console
console.log(2 + 3);    // escreve 5
```

Tudo o que vem depois de `//` numa linha é um **comentário**: o computador ignora, serve só para humanos. Comentários de várias linhas ficam entre `/*` e `*/`. O código do jogo tem comentários em português explicando as partes menos óbvias.

`console.log(...)` é a forma mais simples de "ver" o que o programa está fazendo. Programadores experientes usam isso o tempo todo para investigar problemas.

3.2 Variáveis: dando nome aos valores

Uma **variável** é uma caixa com etiqueta onde você guarda um valor para usar depois.

```
let vida = 100;        // cria a caixa "vida" com o valor 100
vida = vida - 25;      // tira 25: agora vale 75
const nome = 'Íber';   // uma constante: não pode ser trocada depois
```

- `let` cria uma variável que pode mudar de valor.
- `const` cria uma constante: uma vez atribuída, não pode receber outro valor. Use `const` sempre que puder. Isso evita bugs, porque ninguém troca o valor sem querer.
- Você vai ver `var` em códigos antigos. Ele funciona como `let`, mas com regras de escopo confusas. O jogo não usa `var`.

> ⚠️ **Atenção**
>
> O sinal `=` em programação **não** é "igual" da matemática. Ele significa **"guarde o valor da direita na caixa da esquerda"**. Por isso `vida = vida - 25` faz sentido: calcule `vida - 25` e guarde o resultado em `vida`.

No jogo, quase tudo é `const` ou `let`. Por exemplo, o tamanho de cada quadradinho do mapa nunca muda:

js/util.js linhas 4–4

```
const TILE = 32;
```

Regras para nomes

Nomes de variáveis podem ter letras, números, `_` e `$`, mas não começam com número e não têm espaço. O costume em JavaScript é o **camelCase**: a primeira palavra minúscula e as seguintes com inicial maiúscula (`maxHp`, `playerDmg`, `nearestInteractable`). Constantes "globais" importantes costumam ser escritas em MAIÚSCULAS (`TILE`, `SAVE_KEY`, `CHAPTERS`).

3.3 Tipos de valores

Todo valor tem um **tipo**. Os principais:

Tipo	Exemplo	Para que serve no jogo
number (número)	`100`, `0.5`, `-3`	Vida, posição, velocidade, dano
string (texto)	`'Lobo'`, `"Olá"`	Nomes, falas, cores (`'#ffd84a'`)
boolean (lógico)	`true`, `false`	`opened` (baú aberto?), `dead` (inimigo morto?)
undefined	`undefined`	"Ainda não tem valor"
null	`null`	"Não há nada aqui, de propósito"
object (objeto)	`{ x: 10, y: 20 }`	Jogador, inimigos, capítulos
array (lista)	`[1, 2, 3]`	Lista de inimigos, de missões, de falas
function (função)	`function f() {}`	Comportamentos

Você pode perguntar o tipo de algo com `typeof`: `typeof 42` dá `'number'`.

Números e operações

```
10 + 3    // 13   soma
10 - 3    // 7    subtração
10 * 3    // 30   multiplicação
10 / 4    // 2.5  divisão
10 % 3    // 1    resto da divisão (muito útil!)
2 ** 3    // 8    potência
```

O operador `%` (resto) aparece no jogo para fazer coisas "darem a volta". Por exemplo, para escolher a próxima fala de um NPC de forma circular: se ele tem 3 falas, `npc.line % 3` dá 0, 1, 2, 0, 1, 2...

Atalhos úteis: `x += 5` é o mesmo que `x = x + 5`; `x++` soma 1; `x -= dt` subtrai.

O objeto `Math` traz funções matemáticas prontas: `Math.round(2.6)` dá 3, `Math.floor(2.6)` dá 2 (arredonda para baixo), `Math.max(3, 8)` dá 8, `Math.random()` dá um número aleatório entre 0 e 1, `Math.sqrt(16)` dá 4, `Math.hypot(3, 4)` dá 5 (a hipotenusa). O jogo usa muito `Math.sin`, `Math.cos` e `Math.atan2` para ângulos, assunto do capítulo 6.

Textos (strings)

Textos ficam entre aspas simples `'...'`, duplas `"..."` ou crases `` `...` ``. A crase permite **inserir valores** no meio do texto com `${...}`, o que se chama *template string*:

```
const nome = 'Íber', nivel = 3;
'Olá, ' + nome + '!'              // juntar com +  → "Olá, Íber!"
`${nome} está no nível ${nivel}`  // template      → "Íber está no nível 3"
```

O jogo usa template strings para montar textos do HUD, como `` `💰 ${S.gold} ${G.ch.currency}` ``.

Strings têm métodos úteis: `.length` (tamanho), `.toUpperCase()`, `.replace(a, b)`, `.slice(0, 5)` (pedaço), `.trim()` (tira espaços das pontas), `.startsWith('x')`, `.split(',')` (corta numa lista).

3.4 Comparações e lógica

Comparações produzem `true` ou `false`:

```
5 > 3       // true
5 === 5     // true  (igual em valor E tipo)
5 !== 3     // true  (diferente)
'5' === 5   // false (texto não é número)
```

> ⚠️ **Atenção**
>
> Use sempre `===` e `!==`. Existe também `==`, que tenta "converter" os tipos antes de comparar (`'5' == 5` dá `true`), o que gera bugs difíceis de achar.

Para combinar condições:

- `a && b`: **E**, verdadeiro se os dois forem verdadeiros.
- `a || b`: **OU**, verdadeiro se pelo menos um for.
- `!a`: **NÃO**, inverte.

Um uso muito comum de `||` no jogo é dar um **valor padrão**: `step.icon || '✦'` significa "use o ícone da etapa; se ele não existir, use ✦".

Valores "verdadeiros" e "falsos"

Em condições, JavaScript trata como **falso**: `false`, `0`, `''` (texto vazio), `null`, `undefined` e `NaN`. Todo o resto conta como verdadeiro. Por isso o jogo pode escrever `if (G.boss)`, que quer dizer "se existe um chefe".

3.5 Decisões: if, else e o operador ternário

```
if (vida <= 0) {
  console.log('Você caiu...');
} else if (vida < 30) {
  console.log('Cuidado, vida baixa!');
} else {
  console.log('Tudo bem.');
}
```

O bloco entre `{ }` só roda se a condição entre `( )` for verdadeira. Veja um `if` real do jogo, na função que usa uma poção:

js/game.js — linhas 405–414

```
function usePotion() {
  const S = G.S;
  if (S.potions <= 0) { UI.toast(T('noPotions'), 'prog'); return; }
  if (S.hp >= maxHp()) return;
  S.potions--;
  S.hp = Math.min(maxHp(), S.hp + Math.round(maxHp() * 0.5));
  Sfx.play('pickup');
  floatText(G.player.x, G.player.y - 30, T('lifeFloat'), '#5f5', 1);
  for (let i = 0; i < 12; i++) particle(G.player.x, G.player.y, '#6f6', 80);
}
```

Leia devagar, linha a linha: se não há poções, avisa e **sai da função** (`return`); se a vida já está cheia, sai sem fazer nada; senão, gasta uma poção e recupera metade da vida máxima, sem passar do máximo (`Math.min`).

O **operador ternário** é um `if` compacto que produz um valor: `condição ? valorSeVerdadeiro : valorSeFalso`.

```
const texto = cliques === 1 ? 'vez' : 'vezes';
```

> ✅ **Resposta**
>
> O exercício do capítulo 1 (singular e plural) fica assim: `botao.textContent = 'Você clicou ' + cliques + (cliques === 1 ? ' vez' : ' vezes');`

switch: escolher entre muitos casos

Quando uma variável pode ter muitos valores e cada um exige uma ação, o `switch` fica mais legível que vários `if`:

```
switch (tecla) {
  case 'KeyQ': usarPocao(); break;
  case 'KeyR': recarregar(); break;
  default: console.log('tecla sem função');
}
```

O `break` encerra o `switch`. Sem ele, a execução "cai" no caso seguinte. O jogo usa `switch` para tratar teclas, tipos de IA dos inimigos e tipos de etapas de missão.

3.6 Repetições: for e while

Repetir é o que computadores fazem de melhor.

```
for (let i = 0; i < 5; i++) {
  console.log('volta número', i);   // 0, 1, 2, 3, 4
}
```

O `for` tem três partes: **início** (`let i = 0`), **condição para continuar** (`i < 5`) e **o que fazer ao fim de cada volta** (`i++`).

Para percorrer listas, a forma mais legível é o `for...of`, que entrega cada item:

```
const inimigos = ['Lobo', 'Javali', 'Auroque'];
for (const nome of inimigos) console.log(nome);
```

O `while` repete enquanto a condição for verdadeira. O jogo o usa para subir vários níveis de uma vez, se o jogador ganhar muita experiência:

js/game.js — linhas 392–403

```
function gainXp(n) {
  const S = G.S;
  S.xp += n;
  let up = false;
  while (S.xp >= xpNeed(S.level)) { S.xp -= xpNeed(S.level); S.level++; up = true; }
  if (up) {
    S.hp = maxHp();
    Sfx.play('level');
    UI.toast(T('levelUp', S.level), 'level');
    floatText(G.player.x, G.player.y - 40, T('levelFloat', S.level), '#ffd84a', 1.6);
  }
}
```

Repare no `while (S.xp >= xpNeed(S.level))`: enquanto a experiência acumulada for suficiente para o próximo nível, desconta e sobe de nível. Assim, ganhar 1000 XP de uma vez sobe vários níveis corretamente.

> ⚠️ **Atenção**
>
> Cuidado com **laços infinitos**: um `while` cuja condição nunca fica falsa trava a página. Se isso acontecer, feche a aba. No jogo, alguns laços têm um "guarda" (`guard++ < 4000`) como proteção extra.

`break` sai do laço na hora; `continue` pula para a próxima volta. Ambos aparecem várias vezes no código.

3.7 Funções: receitas reutilizáveis

Uma **função** é um bloco de instruções com nome, que você pode executar ("chamar") quantas vezes quiser, passando **parâmetros** e recebendo um **resultado** de volta (`return`).

```
function area(largura, altura) {
  return largura * altura;
}
```

```
area(3, 4);   // 12
area(10, 2);  // 20
```

Funções são o principal jeito de **organizar** um programa. Em vez de um código gigante, você o divide em pedaços com nomes claros: `usePotion`, `gainXp`, `hurtPlayer`, `killEnemy`. Lendo só os nomes, já dá para entender o que o programa faz.

Uma função curtíssima e muito usada no jogo:

js/util.js — linhas 40–40

```
const dist = (ax, ay, bx, by) => Math.hypot(bx - ax, by - ay);
```

Isso calcula a distância entre dois pontos (o teorema de Pitágoras, feito pelo `Math.hypot`). É chamada centenas de vezes por segundo: para saber se o inimigo está perto, se o jogador alcançou um lugar, se um projétil acertou alguém...

Funções de seta (arrow functions)

Existe uma forma mais curta de escrever funções, com `=>`:

```
const dobro = (x) => x * 2;          // retorna x * 2 automaticamente
const saudar = (nome) => {           // com chaves, precisa de return
  const texto = 'Olá, ' + nome;
  return texto;
};
```

O `dist` acima é uma arrow function. Elas são muito usadas para passar funções como parâmetro, como você verá em `filter`, `map` e nos eventos.

Parâmetros opcionais e valores padrão

Se você chamar uma função sem algum parâmetro, ele vale `undefined`. O jogo usa `||` para dar valores padrão:

js/game.js — linhas 417–420

```
function particle(x, y, color, spd, life, size) {
  const a = Math.random() * Math.PI * 2, s = Math.random() * (spd || 100);
  G.parts.push({ x, y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life: life || 0.5, max: life || 0.5, color,
size: size || 3 });
}
```

Aqui, `life || 0.5` significa "se não informaram a duração, use 0,5 segundo". Assim você pode chamar `particle(x, y, '#fff')` sem passar tudo.

Funções que recebem e devolvem funções

Em JavaScript, funções são valores como outros quaisquer: dá para guardá-las em variáveis, passá-las como parâmetro e devolvê-las como resultado. O gerador de números aleatórios do jogo é uma função que **fabrica** outra função:

js/util.js linhas 6–15

```js
function mulberry32(seed) {
  let a = seed >>> 0;
  return function () {
    a = (a + 0x6D2B79F5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

`mulberry32(1101)` devolve uma função nova. Cada vez que você chama essa função, ela dá o próximo número da sequência. A variável `a` fica "presa" dentro dela, lembrando onde parou. Isso se chama **closure** (fechamento), e o capítulo 7 explica por que este gerador é essencial para criar mapas.

3.8 Listas (arrays)

Um **array** é uma lista ordenada de valores, entre colchetes. Cada item tem uma posição (índice) que **começa em 0**:

```js
const falas = ['Olá!', 'Bom dia.', 'Cuidado com os lobos.'];
falas[0]          // 'Olá!'
falas.length      // 3
falas.push('Tchau!');  // adiciona no fim; agora tem 4
```

O jogo guarda quase tudo em arrays: `G.enemies` (inimigos vivos), `G.projs` (projéteis voando), `G.parts` (partículas), `G.objs` (NPCs, itens, baús), `CHAPTERS` (os 14 capítulos).

Os métodos de array que você mais vai usar

Estes métodos recebem uma **função** que é aplicada a cada item:

Método	O que faz	Exemplo no jogo
`forEach(f)`	Executa `f` para cada item	Percorrer botões e ligar cliques
`filter(f)`	Nova lista só com os itens em que `f` dá verdadeiro	`G.enemies.filter(e => !e.dead)` remove os mortos
`map(f)`	Nova lista com `f` aplicada a cada item	Montar o HTML de cada item da loja
`find(f)`	O **primeiro** item em que `f` dá verdadeiro	Achar um NPC pelo id
`some(f)`	`true` se **algum** item satisfaz `f`	
`sort(f)`	Ordena a lista	Ordenar personagens pela altura (y) para desenhar
`reduce(f, ini)`	"Reduz" a lista a um valor (ex.: soma)	Contar inimigos vivos de um acampamento
`includes(x)`	`true` se `x` está na lista	Teclas que não devem rolar a página

Um exemplo real: a cada quadro, o jogo remove da lista todos os projéteis que já acabaram:

```
G.projs = G.projs.filter(pr => pr.life > 0);
```

Leia em voz alta: "a nova lista de projéteis é a lista antiga filtrada, mantendo só os projéteis com vida maior que zero".

> ✍️ **Exercício**
>
> No Console, crie `const nums = [5, 12, 8, 1, 20];` e use `filter` para obter só os maiores que 7, e `map` para obter o dobro de cada um.

> ✅ **Resposta**
>
> `nums.filter(n => n > 7)` dá `[12, 8, 20]`. `nums.map(n => n * 2)` dá `[10, 24, 16, 2, 40]`.

3.9 Objetos: agrupando informações

Um **objeto** agrupa valores relacionados, cada um com um nome (chave). É como uma ficha:

```
const lobo = {
  name: 'Lobo',
  hp: 24,        // pontos de vida
  spd: 130,      // velocidade
  ai: 'melee'    // tipo de inteligência
};
lobo.name        // 'Lobo'      (acesso com ponto)
lobo['hp']       // 24          (acesso com colchetes)
lobo.hp = 10;    // muda um valor
lobo.raivoso = true;  // cria uma chave nova
```

Objetos podem conter outros objetos e arrays, formando estruturas tão complexas quanto você precisar. **Todos os dados do jogo são objetos.** Veja a ficha real de um inimigo no arquivo `base.js`:

js/data/base.js — linhas 43–43

```
lobo:      { name: 'Lobo',              hp: 24, spd: 130, dmg: 6,  ai: 'melee',  r: 13, xp: 8,  gold: 2,
shape: 'beast', color: '#7d7d80' },
```

E o estado de uma partida salva, criado quando você começa um jogo novo:

js/game.js — linhas 29–35

```
function newState(name) {
  return {
    v: 1, name: name || 'Íber', level: 1, xp: 0, gold: 0, potions: 3, sharp: 0, armor: 0,
    hp: 100, chapter: 0, maxChapter: 0, ch: {}, pos: null,
    stats: { kills: 0, deaths: 0, time: 0, quests: 0 }, sound: true
  };
}
```

Esse objeto, guardado em `G.S`, contém **tudo** o que precisa ser salvo: nome, nível, experiência, dinheiro, poções, melhorias, vida, capítulo atual, progresso das missões e estatísticas.

Atalhos modernos com objetos

```
const { x, y } = jogador;          // desestruturação: cria x e y a partir do objeto
const copia = Object.assign({}, a, b); // junta a e b num objeto novo (b sobrescreve a)
Object.keys(obj)                   // lista das chaves
Object.entries(obj)                // lista de pares [chave, valor]
```

O jogo usa `Object.assign` para criar a aparência dos NPCs a partir de modelos: `Object.assign({}, LOOK.noble, { body: '#2a4a7a' })` copia o modelo de "nobre" e troca só a cor do corpo.

3.10 Classes: moldes para criar objetos

Quando você precisa de muitos objetos parecidos (dezenas de inimigos, centenas de projéteis), escreve uma **classe**: um molde. A palavra `new` cria um objeto a partir do molde, e a função especial `constructor` prepara cada objeto novo. Dentro dela, `this` significa "o objeto que está sendo criado".

js/entities.js — linhas 247–253

```
class Proj {
  constructor(x, y, a, speed, dmg, owner, color, r, text) {
    this.x = x; this.y = y; this.vx = Math.cos(a) * speed; this.vy = Math.sin(a) * speed;
    this.dmg = dmg; this.owner = owner; this.color = color || '#fff'; this.r = r || 4;
    this.life = 1.4; this.text = text || null;
  }
}
```

Agora o jogo pode criar um projétil com uma linha: `new Proj(x, y, angulo, velocidade, dano, 'p', '#ffe080', 3)`. A classe calcula sozinha a velocidade em cada eixo (`vx`, `vy`) a partir do ângulo e já define que a bala vive 1,4 segundo.

Classes também podem ter **métodos**: funções que pertencem ao objeto. A classe `World` (o mapa) tem métodos como `generate()`, `collides()` e `draw()`. Dentro de um método, `this` é o próprio mapa: `this.W` é a largura dele.

3.11 Objetos com funções: o padrão "módulo"

Outro jeito de organizar código, muito usado no jogo, é um **objeto comum cheio de funções**. A interface inteira, por exemplo, é um objeto chamado `UI` com métodos como `UI.dialog()`, `UI.shop()` e `UI.map()`. O som é `Sfx`, a música é `Music`. Isso agrupa tudo o que é relacionado debaixo de um nome só, evitando que centenas de funções fiquem "soltas".

```
const Contador = {
  valor: 0,
  somar() { this.valor++; },
  mostrar() { console.log(this.valor); }
};
Contador.somar(); Contador.mostrar();  // 1
```

3.12 Eventos: reagindo ao jogador

Um jogo passa a maior parte do tempo **esperando** o jogador fazer algo. Você registra uma função para ser chamada quando um evento acontecer, com `addEventListener`:

```
window.addEventListener('keydown', (e) => {
  console.log('Apertou a tecla', e.code);
});
```

A função que você passa é chamada de **callback** ("chame de volta"). O navegador a executa no momento certo e entrega um objeto `e` com detalhes (qual tecla, onde o mouse está...). O jogo escuta `keydown`, `keyup`, `mousemove`, `mousedown`, `mouseup`, `resize` e `blur`. O capítulo 8 mostra como.

Callbacks também servem para dizer **"faça isto depois"**. A função de diálogo do jogo recebe uma função `cb` para executar quando o jogador terminar de ler: `UI.dialog(falas, () => advance(quest))` quer dizer "mostre estas falas e, quando acabar, avance a missão".

3.13 JSON: transformando objetos em texto

Para salvar uma partida, o jogo precisa transformar o objeto `G.S` em texto, porque o armazenamento do navegador só guarda textos. O formato usado é o **JSON**:

```
const texto = JSON.stringify({ nome: 'Íber', nivel: 3 }); // '{"nome":"Íber","nivel":3}'
const obj = JSON.parse(texto);                            // de volta a objeto
```

3.14 Erros e try/catch

Às vezes algo pode dar errado por motivos fora do seu controle: o navegador está em modo privado e não deixa salvar, ou o jogador importa um arquivo corrompido. O `try/catch` executa o código e, se ocorrer um erro, cai no `catch` em vez de travar o jogo:

js/game.js — linhas 82–84

```
function loadSave(n) {
  try { const s = localStorage.getItem(slotKey(n)); return s ? JSON.parse(s) : null; } catch (e) { return
null; }
}
```

Se o texto salvo estiver corrompido, `JSON.parse` lança um erro, o `catch` o captura e a função devolve `null` ("não há save"), e o jogo segue funcionando.

3.15 Escopo: onde cada nome existe

Uma variável criada dentro de `{ }` só existe ali dentro. Isso é o **escopo**:

```
function teste() {
  const segredo = 42;
}
console.log(segredo); // ERRO: segredo não existe aqui fora
```

Variáveis criadas fora de qualquer função ou bloco, no topo de um arquivo, são **globais**: todos os scripts da página as enxergam. É assim que o jogo conecta seus arquivos: `util.js` cria `TILE`, e depois `world.js` pode usar `TILE`. O lado ruim é que **dois arquivos não podem criar o mesmo nome global**.

> 🐞 **História de um bug**
>
> Ao adicionar o sistema de idiomas, criei uma função global chamada `T` (de "traduzir"). Mas o arquivo `world.js` já tinha uma constante global chamada `T` (os tipos de terreno). Resultado: o navegador recusou carregar o `world.js` com o erro "*Identifier 'T' has already been declared*" e o jogo inteiro parou. A solução foi renomear a constante do mapa para `TL`. **Lição:** nomes globais devem ser descritivos e únicos. O Console (F12) mostrou o erro exato, com arquivo e linha.

3.16 'use strict'

Todos os arquivos do jogo começam com `'use strict';`. Essa linha liga o **modo estrito**, que transforma alguns erros silenciosos em erros visíveis. Por exemplo, sem ele, um erro de digitação como `vdia = 10` (em vez de `vida`) cria uma variável nova sem avisar; com ele, o navegador acusa o erro na hora. É uma rede de segurança gratuita.

3.17 Resumo do capítulo

Você agora conhece as peças que formam qualquer programa: **valores e variáveis** para guardar dados, **condições** para decidir, **laços** para repetir, **funções** para organizar e reutilizar, **arrays e objetos** para estruturar informações, **classes** para criar muitos objetos parecidos e **eventos** para reagir ao jogador. O resto do guia mostra como essas mesmas peças, combinadas, formam um jogo completo.

> ✍️ **Exercício**
>
> Escreva uma função `nivelPorXp(xp)` que receba uma quantidade de experiência e diga em que nível o jogador estaria, sabendo que o nível 1 precisa de 50 XP para subir, o nível 2 de 123, o nível 3 de 209 (a fórmula do jogo é `Math.round(50 * Math.pow(nivel, 1.3))`). Use um `while`.

> ✅ **Resposta**

```
function nivelPorXp(xp) {
  let nivel = 1;
  while (xp >= Math.round(50 * Math.pow(nivel, 1.3))) {
    xp -= Math.round(50 * Math.pow(nivel, 1.3));
    nivel++;
  }
  return nivel;
}
nivelPorXp(200); // 3
```

Teste no Console: `nivelPorXp(200)` dá **3**, porque 200 − 50 = 150, e 150 − 123 = 27, que não basta para os 209 do nível 3.

CAPÍTULO 4

Por que cada tecnologia foi escolhida

Você pediu para eu explicar por que escolhi cada biblioteca. A resposta curta surpreende muita gente: **o jogo não usa nenhuma biblioteca externa**. Nenhuma. Tudo o que ele faz, desenho, som, música, salvamento e tradução, usa recursos que **já vêm dentro de qualquer navegador moderno**, as chamadas *APIs da Web* (interfaces de programação que o navegador oferece).

Isso foi uma decisão consciente, e este capítulo explica o raciocínio por trás dela, comparando cada escolha com as alternativas mais populares. Entender essas trocas é tão importante quanto saber programar: na vida real, você vai passar muito tempo decidindo **o que usar**.

4.1 O que é uma biblioteca, um framework e um motor de jogo

- Uma **biblioteca** é um pacote de código escrito por outras pessoas que você chama quando precisa. Exemplo: Howler.js toca sons; você chama `new Howl({ src: 'som.mp3' }).play()`.
- Um **framework** é uma estrutura maior que organiza o seu programa e chama o *seu* código nos momentos certos. Exemplo: Phaser, um framework de jogos, decide quando chamar sua função `update()`.
- Um **motor de jogo** (*game engine*) é um programa completo, com editor visual, física, animação e exportação. Exemplos: Unity, Godot, Unreal.

Todos são ótimos e economizam muito trabalho em projetos grandes. Mas cada um tem um custo: precisa ser instalado, aprendido, atualizado, e esconde de você *como as coisas funcionam por dentro*.

4.2 Os critérios usados

Antes de escolher qualquer ferramenta, listei o que o projeto precisava:

1. **Rodar no GitHub Pages**, que só hospeda arquivos estáticos (sem servidor).
2. **Abrir com dois cliques**, sem instalar nada, sem "compilar", sem terminal.
3. **Ser fácil de entender e modificar** por quem está aprendendo. Este guia é prova disso: dá para explicar cada linha.
4. **Ser leve**: carregar rápido, inclusive em internet lenta.
5. **Não depender de arquivos de arte ou som**, que eu não tinha. Tudo precisava ser **gerado por código**.

Com esses critérios, o navegador "puro" venceu em quase todas as categorias. Veja cada uma.

4.3 Desenho: Canvas 2D em vez de Phaser, PixiJS ou Unity

Opção	Vantagens	Por que não foi escolhida
Canvas 2D (escolhido)	Já vem no navegador; API simples (`fillRect` , `arc` , `drawImage`); controle total	Você mesmo escreve câmera, colisão e laço do jogo
Phaser	Framework de jogos 2D completo: física, cenas, carregamento de imagens, animações	~1 MB a mais; a maioria das funções seria inútil aqui; esconde os conceitos que este guia quer ensinar
PixiJS	Desenho muito rápido usando a placa de vídeo (WebGL)	Só faz sentido com milhares de sprites; aqui desenhamos poucas centenas de formas por quadro
Unity / Godot	Editor visual, física 3D/2D, exportação para celular e console	Exige instalar programas grandes; o resultado exportado para web pesa dezenas de MB; uma linguagem diferente (C#/GDScript)

O **Canvas 2D** é uma "tela de pintura" onde você desenha com comandos como "pinte um retângulo aqui" e "desenhe um círculo ali". Para um jogo 2D de visão de cima, com personagens desenhados por formas geométricas, ele é mais do que suficiente. O jogo roda a 60 quadros por segundo mesmo em computadores modestos, graças a um truque de cache de mapa que você verá no capítulo 7.

> ❓ **Por quê?**
>
> **Por que os personagens são desenhados com código e não com imagens (sprites)?** Eu não tinha artes prontas, e usar artes de terceiros traz questões de licença. Desenhar com formas permite criar **14 classes de herói e mais de 50 inimigos** variando só cores e acessórios (`hat: 'morion'` , `weapon: 'gun'`). Se um dia você quiser sprites desenhados à mão, o Canvas carrega imagens com `ctx.drawImage(imagem, x, y)` : basta trocar as funções de desenho.

4.4 Linguagem: JavaScript puro em vez de TypeScript

TypeScript é JavaScript com "tipos" declarados (`let vida: number = 100`). Ele pega muitos erros antes de você rodar o programa e é a escolha certa para equipes e projetos grandes. Mas exige um passo de **compilação** (transformar `.ts` em `.js`) e uma configuração. Para um iniciante, isso é uma camada a mais entre o código e o resultado. O JavaScript puro roda direto: você salva, aperta F5, vê o resultado.

4.5 Organização: scripts clássicos em vez de módulos ES e sem "bundler"

O JavaScript moderno tem **módulos** (`import` / `export`), que deixam cada arquivo declarar exatamente o que usa dos outros. É mais organizado. Mas há um detalhe prático: **módulos não funcionam ao abrir o arquivo com dois cliques** (endereço `file://`); o navegador exige um servidor. Como um dos critérios era "abrir com dois cliques", o jogo usa **scripts clássicos** carregados em ordem no `index.html` , que compartilham variáveis globais.

Ferramentas como **Vite, Webpack ou esbuild** ("bundlers") juntam e otimizam arquivos. Elas brilham em sites grandes, mas exigiriam Node.js, `npm install`, uma pasta `node_modules` com milhares de arquivos e um comando de *build* antes de cada teste. Aqui, nada disso é necessário.

> ⚠️ **Atenção**
>
> O preço dessa simplicidade é o que você viu no capítulo 3: nomes globais podem colidir (o bug do `T`). Em um projeto maior, eu trocaria para módulos ES com um servidor de desenvolvimento. É uma troca consciente, não um "jeito certo".

4.6 Interface: HTML e CSS em vez de React ou desenhar no canvas

Frameworks como **React, Vue ou Svelte** organizam interfaces complexas que mudam o tempo todo (como o Facebook). O Hispânia tem poucas janelas (diálogo, loja, diário, mapa, menu), que são montadas com texto HTML e mostradas ou escondidas. Um objeto `UI` com cerca de 30 métodos resolve tudo, sem dependências.

Já a alternativa de desenhar a interface **dentro do canvas** exigiria reimplementar à mão botões, quebra de linha, rolagem e campos de texto. O HTML faz isso de graça, com acessibilidade e boa aparência.

4.7 Som: Web Audio API em vez de Howler.js ou Tone.js

Opção	Para que serve	Decisão
Web Audio API (escolhida)	Sintetizador e mixer embutidos no navegador: osciladores, filtros, ganho, atraso, buffers	Gera todos os sons e músicas por código, sem arquivos
Howler.js	Tocar arquivos `.mp3` / `.ogg` de forma fácil e compatível	Não havia arquivos de áudio para tocar
Tone.js	Biblioteca musical poderosa sobre a Web Audio (sintetizadores, sequenciadores)	Ótima, mas ~300 KB; para os poucos instrumentos necessários, a Web Audio pura bastou

A **Web Audio API** funciona como um estúdio de som modular: você cria "caixinhas" (um oscilador que gera uma nota, um filtro que abafa o som, um controle de volume) e as conecta com fios virtuais até os alto-falantes. O capítulo 14 ensina como cada som e cada instrumento medieval foi construído assim, inclusive o alaúde dedilhado, feito com um algoritmo clássico chamado **Karplus-Strong**.

> ❓ **Por quê?**
>
> **Por que música gerada em vez de arquivos MP3?** Três motivos. (1) Não havia trilha pronta, e música de terceiros tem direitos autorais. (2) Arquivos de música pesam vários megabytes cada; 15 temas seriam dezenas de MB, enquanto o motor de música inteiro tem cerca de 300 linhas. (3) A música gerada **varia sozinha**: a cada duas repetições, a melodia é recomposta, então não cansa em 5 horas de jogo.

4.8 Salvamento: localStorage em vez de SQLite, IndexedDB ou um servidor

No início da nossa conversa, você perguntou sobre um "banco de dados acoplado", como o SQLite. Veja a comparação:

Opção	Como funciona	Decisão
localStorage (escolhido)	Guarda pares "chave → texto" no navegador; até ~5 MB; API de 3 funções	Um save do jogo tem poucos KB. É o mais simples possível
IndexedDB	Banco de dados de verdade dentro do navegador, para muitos dados	API complicada; desnecessário para 5 saves pequenos
SQLite no navegador (sql.js / WASM)	O SQLite real compilado para rodar no navegador	~1 MB de download e você ainda precisa salvar o arquivo do banco em algum lugar (normalmente... no IndexedDB)
Servidor (Supabase, Firebase, seu PC)	Dados ficam na nuvem, compartilhados entre jogadores	Só vale para ranking, multiplayer ou contas. Exige internet, contas e custos

Como o jogo é para um jogador só, o **localStorage** é suficiente. Para o jogador não perder o progresso ao trocar de computador, o jogo oferece **exportar e importar** o save como arquivo `.json`.

4.9 Tradução: um objeto de textos em vez de i18next

Bibliotecas como **i18next** cuidam de plurais complexos, dezenas de idiomas e carregamento sob demanda. Para dois idiomas, uma função `T('chave')` de 5 linhas e um dicionário resolvem. O conteúdo dos capítulos é traduzido por **sobreposição**: um arquivo espanhol com a mesma estrutura substitui os textos portugueses ao carregar (capítulo 13).

4.10 Ferramentas de bastidor (fora do jogo)

Algumas ferramentas foram usadas só durante o desenvolvimento e não fazem parte do jogo publicado:

- **Node.js**: executou o servidor de testes `.claude/serve.js` (que usa só módulos nativos do Node: `http`, `fs` e `path`), os testes automáticos e o gerador deste PDF. Nenhum pacote do `npm` foi instalado.
- **Microsoft Edge em modo headless** (sem janela): converteu este guia de HTML para PDF. Escolhi o Edge porque já vem no Windows. Alternativas como Puppeteer ou bibliotecas Python de PDF exigiriam instalação.
- **Git e GitHub Pages**: para versionar e publicar (capítulo 15).

4.11 Quando você DEVE usar uma biblioteca

Não leve deste capítulo a ideia de que bibliotecas são ruins. Elas são ótimas quando o problema é grande e difícil. Use uma quando:

- O problema é complexo e resolvido há anos por especialistas (física realista: **Matter.js**; 3D: **Three.js**; criptografia: **nunca** escreva a sua).

- Você precisa de compatibilidade com muitos navegadores e dispositivos antigos.
- Você trabalha em equipe e todos já conhecem a ferramenta.
- O tempo de entrega importa mais do que entender cada detalhe.

Para **aprender**, porém, construir as peças você mesmo é imbatível. Depois de terminar este guia, quando usar o Phaser, você vai entender o que ele faz por baixo dos panos, e isso vai fazer de você um programador muito melhor.

CAPÍTULO 5

A arquitetura do projeto

Antes de estudar cada sistema, vamos olhar o jogo de cima, como uma planta baixa. Saber **onde está cada coisa** e **quem conversa com quem** é o que permite mexer num projeto de 6 mil linhas sem se perder.

5.1 A árvore de pastas e arquivos

```
game/                        ← a pasta do projeto (vira o site no GitHub Pages)
├── index.html               ← porta de entrada: estrutura da página e ordem dos scripts
├── README.md                ← descrição curta do projeto
├── css/
│   └── style.css            ← aparência de toda a interface HTML (HUD, janelas, título)
├── js/                      ← todo o código JavaScript
│   ├── util.js              ← ferramentas gerais: números aleatórios, ruído, matemática, efeitos sonoros
│   ├── i18n.js              ← idiomas: textos da interface em PT/ES e a função T()
│   ├── music.js             ← motor de música medieval gerada em tempo real
│   ├── world.js             ← o mapa: geração procedural, colisão e desenho dos terrenos
│   ├── entities.js          ← desenho de personagens e as classes Player, Enemy e Proj
│   ├── game.js              ← o coração: estado, missões, combate, IA, laço e renderização
│   ├── ui.js                ← a interface: diálogos, loja, diário, mapa, menus, título
│   └── data/                ← DADOS (nenhuma lógica): o conteúdo do jogo
│       ├── base.js          ← armas, classes do herói e todos os inimigos
│       ├── chapters1.js     ← capítulos 1–5 (pré-história ao Califado)
│       ├── chapters2.js     ← capítulos 6–10 (El Cid a 1931)
│       ├── chapters3.js     ← capítulos 11–14 (Guerra Civil a hoje)
│       ├── es_base.js       ← tradução ES de armas, classes e inimigos
│       └── es1.js, es2.js, es3.js ← tradução ES dos capítulos
├── guia/                    ← este guia (não faz parte do jogo)
│   ├── build.js             ← gerador do PDF
│   ├── estilo.css, capa.html ← aparência do PDF
│   └── partes/*.md          ← o texto de cada capítulo do guia
└── .claude/                 ← ferramentas de teste (não fazem parte do jogo)
    ├── serve.js             ← servidor local mínimo em Node
    └── launch.json          ← configuração para o painel de pré-visualização
```

E o tamanho de cada arquivo, em linhas:

Arquivo	Linhas	Tipo de conteúdo
`js/game.js`	1168	Lógica principal
`js/ui.js`	534	Interface
`js/world.js`	515	Mapa
`js/music.js`	336	Música
`js/entities.js`	253	Desenho e entidades
`js/i18n.js`	166	Idiomas

Arquivo	Linhas	Tipo de conteúdo
`js/util.js`	113	Utilidades
`js/data/*.js`	~2700	Conteúdo (dados)
`css/style.css`	149	Estilo

Repare numa coisa: **quase metade do projeto são dados**, não lógica. Isso é intencional e é o assunto da próxima seção.

5.2 O princípio mais importante: separar dados de lógica

O jogo tem 14 capítulos, 66 missões e quase 300 etapas, mas **não existe uma linha de código específica para nenhuma missão**. Não há um `if (missao === 'caçar lobos')` em lugar nenhum. Em vez disso:

- Os arquivos em `js/data/` **descrevem** o conteúdo: "a etapa 2 é matar 5 lobos no Bosque dos Lobos".
- O `game.js` sabe **executar** qualquer descrição desse tipo: "se a etapa é do tipo `kill`, conte as mortes do inimigo `e` até chegar em `n`".

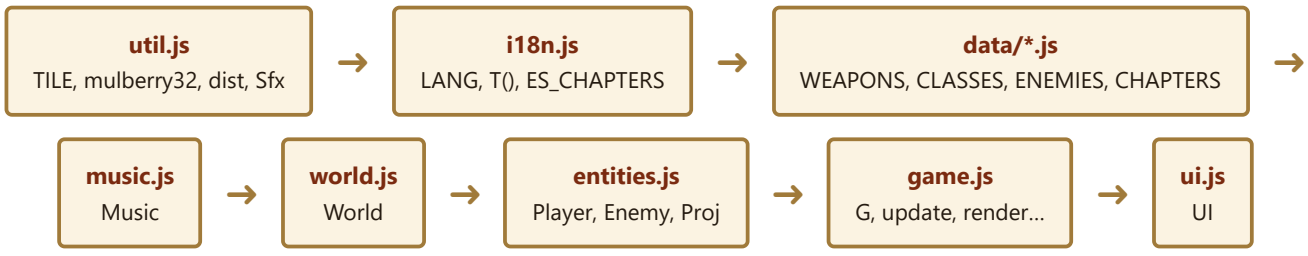
Isso se chama **design orientado a dados** (*data-driven design*). As vantagens são enormes:

- Para criar uma missão nova, você escreve dados, não código. Até quem não programa consegue.
- A tradução vira só "outro arquivo de dados com a mesma forma".
- Um único motor executa 296 etapas. Corrigir um bug no motor corrige todas.
- Os dados podem ser **validados automaticamente** (capítulo 15).

> ❓ **Por quê?**
>
> Imagine o contrário: 66 missões, cada uma com seu próprio código. Seriam milhares de linhas repetidas, e cada bug apareceria em 66 lugares diferentes. Toda vez que você se pegar copiando e colando código com pequenas diferenças, pergunte-se: "isso poderia ser dado em vez de código?"

5.3 A ordem de carregamento e quem depende de quem

Como os scripts compartilham variáveis globais, **a ordem no `index.html` é uma lista de dependências**. Cada arquivo só pode usar o que os anteriores criaram *no momento em que é carregado*:



Existe uma sutileza: funções podem **chamar** coisas definidas em arquivos posteriores, desde que a chamada só aconteça **depois** que tudo carregou. Por exemplo, `game.js` chama `UI.toast(...)` mesmo que `UI` só seja criado em `ui.js`. Funciona, porque `UI.toast` só é executado durante o jogo, quando todos os arquivos já

foram lidos. O que não pode é usar `UI` **no momento do carregamento** de `game.js`.

5.4 Os objetos globais principais

Nome	Arquivo	O que é
`G`	game.js	O **estado do jogo em execução**: mapa, jogador, inimigos, câmera, teclas...
`G.S`	game.js	O **estado salvável**: só o que vai para o save (nível, ouro, progresso)
`UI`	ui.js	Todas as funções de interface
`Sfx`	util.js	Efeitos sonoros
`Music`	music.js	Trilha sonora
`CHAPTERS`	data/chapters1.js	Lista dos 14 capítulos (dados)
`ENEMIES`, `WEAPONS`, `CLASSES`	data/base.js	Fichas de inimigos, armas e classes
`World`	world.js	A classe do mapa (`G.world` é uma instância dela)

Vale a pena olhar o objeto `G` inteiro, porque ele aparece em quase todas as funções:

js/game.js — linhas 11–16

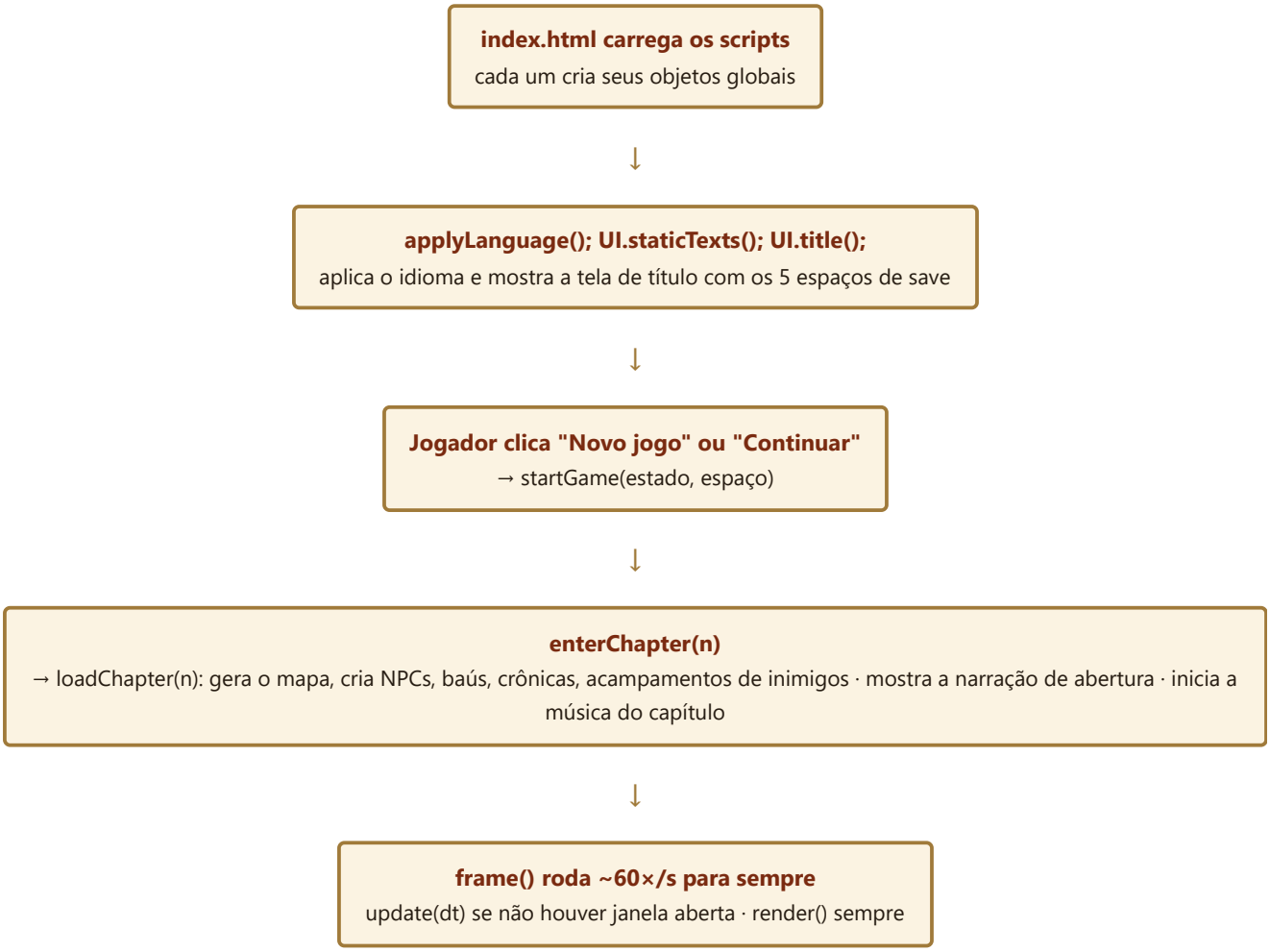
```
const G = {
  S: null, world: null, ch: null, chIdx: 0, mult: 1,
  player: null, enemies: [], projs: [], parts: [], texts: [], objs: [], camps: [], fx: [],
  boss: null, cam: { x: 0, y: 0 }, shake: 0, keys: {}, mouse: { x: 0, y: 0, down: false },
  time: 0, running: false, ui: null, hover: null, saveT: 0, hudT: 0, deadFlag: false
};
```

- `S`: o save (nível, XP, ouro...). Fica separado porque é a única parte que precisa ir para o disco.
- `world`, `ch`, `chIdx`, `mult`: o mapa atual, os dados do capítulo atual, o número dele e o **multiplicador de dificuldade** do capítulo.
- `player`, `enemies`, `projs`, `parts`, `texts`, `objs`, `camps`, `fx`: tudo o que existe no mundo agora.
- `cam`, `shake`: posição da câmera e intensidade do tremor de tela.
- `keys`, `mouse`: o estado da entrada (quais teclas estão pressionadas, onde está o mouse).
- `ui`: se há uma janela aberta (`'dialog'` ou `'modal'`), o jogo **pausa** a lógica.

> ❓ **Por quê?**
>
> **Por que separar `G` de `G.S`?** O mundo (inimigos, partículas, projéteis) é recriado do zero sempre que você entra num capítulo. Não faz sentido salvar 300 partículas. Separando o que é "temporário" (G) do que é "permanente" (G.S), o save fica pequeno e simples: `JSON.stringify(G.S)`.

5.5 O fluxo do jogo, do clique ao laço

Este é o caminho que o programa percorre desde que a página abre:



As duas últimas linhas de `game.js` **ligam o motor** assim que o arquivo carrega: `resize();` ajusta o canvas e `requestAnimationFrame(frame);` começa o laço. Ele gira "em vazio" (sem desenhar nada) até que `G.running` vire verdadeiro, o que acontece aqui:

js/game.js — linhas 1184–1194

```
function startGame(state, slot) {
  G.S = state;
  G.slot = slot;
  Sfx.enabled = state.sound !== false;
  CHAPTERS.sort((a, b) => a.num - b.num);
  G.running = true;
  const hp = state.hp;
  const idx = clamp(state.chapter || 0, 0, CHAPTERS.length - 1);
  enterChapter(idx, true);
  if (hp > 0) G.S.hp = Math.min(hp, maxHp());
}
```

`startGame` é chamada pela tela de título. Ela guarda o estado e o espaço de save escolhidos, ajusta o som e entra no capítulo salvo. A partir daí, quem manda é o laço `frame`, assunto do próximo capítulo.

CAPÍTULO 6

O laço do jogo, o Canvas e a matemática dos jogos

6.1 O laço: requestAnimationFrame

Lembra das três etapas de todo jogo (entrada, atualizar, desenhar)? No navegador, quem dá o "ritmo" é a função `requestAnimationFrame` (pedir quadro de animação). Você entrega uma função a ela, e o navegador a chama **logo antes de desenhar a próxima imagem na tela**, normalmente 60 vezes por segundo (ou 120/144 em monitores mais rápidos).

js/game.js — linhas 1173–1182

```
function frame(now) {
  const dt = Math.min(0.05, (now - last) / 1000);
  last = now;
  if (G.running && G.world) {
    if (!G.ui) update(dt);
    else G.time += dt;
    render();
  }
  requestAnimationFrame(frame);
}
```

Linha a linha:

1. O navegador passa `now`, o horário atual em milissegundos.
2. `dt` (*delta time*) é o **tempo que passou desde o último quadro**, em segundos. Se o jogo roda a 60 FPS, `dt` vale cerca de 0,0167.
3. `Math.min(0.05, ...)` limita `dt` a no máximo 0,05 s. Se o computador engasgar ou você trocar de aba e voltar, um `dt` gigante faria o herói "teleportar" através das paredes. Esse limite evita isso.
4. Se há um jogo rodando, chama `update(dt)` (só se nenhuma janela estiver aberta, assim os diálogos pausam o jogo) e depois `render()`.
5. Por fim, **pede o próximo quadro**. É isso que mantém o laço girando para sempre.

Por que multiplicar tudo por dt

Este é um dos conceitos mais importantes de programação de jogos. Compare:

```
heroi.x += 3;              // ERRADO: anda 3 pixels POR QUADRO
heroi.x += 175 * dt;       // CERTO: anda 175 pixels POR SEGUNDO
```

Na versão errada, num monitor de 144 Hz o herói anda mais que o dobro de rápido do que num monitor de 60 Hz. Multiplicando pela fração de segundo que passou, a velocidade fica igual em qualquer computador. **Toda velocidade, cronômetro e animação do jogo usa `dt`.** Por exemplo, os cronômetros de recarga funcionam assim: `p.cd -= dt`, que significa "desconte o tempo que passou; quando chegar a zero, pode atacar de novo".

> 🐞 **História de um bug**
>
> Durante os testes, o jogo parecia "congelado" dentro do painel do editor: os inimigos não reapareciam e o tempo não andava. O motivo: navegadores **pausam o** `requestAnimationFrame` **em abas ocultas**, para economizar bateria. Não era um bug do jogo, e sim do ambiente de teste. A solução foi chamar `update(1/30)` manualmente nos testes. Lição: entenda o ambiente onde o código roda.

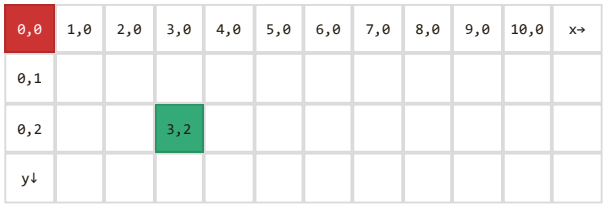
6.2 O Canvas: sua tela de pintura

O `<canvas>` é uma grade de pixels. Para desenhar nele, você pega um **contexto 2D**, um objeto cheio de comandos de desenho:

```
const canvas = document.getElementById('game');
const ctx = canvas.getContext('2d');
ctx.fillStyle = '#c33';          // escolhe a cor da tinta
ctx.fillRect(10, 20, 100, 50);   // retângulo em x=10, y=20, largura 100, altura 50
ctx.beginPath();
ctx.arc(200, 60, 30, 0, Math.PI * 2);  // círculo: centro (200,60), raio 30
ctx.fill();
```

O sistema de coordenadas

Atenção: no Canvas, o ponto (0, 0) fica no **canto superior esquerdo**, e o **y cresce para baixo** (ao contrário da matemática da escola). Por isso, no jogo, apertar W (para cima) **diminui** o y do herói.



Os comandos de desenho usados no jogo

Comando	O que faz
`fillStyle = cor` / `strokeStyle = cor`	Cor de preenchimento / de contorno
`fillRect(x, y, l, a)`	Retângulo preenchido
`beginPath()` … `fill()` / `stroke()`	Começa uma forma livre e a preenche ou contorna
`arc(x, y, r, a0, a1)`	Arco/círculo (ângulos em radianos)
`ellipse(x, y, rx, ry, rot, a0, a1)`	Elipse (sombras, corpos de animais)
`moveTo(x, y)` / `lineTo(x, y)`	Linhas e polígonos (triângulos dos pinheiros)
`quadraticCurveTo(...)`	Curvas (lâmina da falcata, chamas)
`fillText(texto, x, y)` / `font = ...`	Texto (números de dano, nomes)
`drawImage(img, x, y)`	Desenha uma imagem ou outro canvas
`globalAlpha = 0.5`	Transparência

Comando	O que faz
`save()` / `restore()`	Guarda e restaura o estado (cor, posição, rotação)
`translate(x, y)` / `rotate(a)` / `scale(s, s)`	Move, gira e redimensiona o "papel"

Ajustando o canvas à tela (e a telas de alta densidade)

js/game.js — linhas 18–25

```
function resize() {
  const dpr = Math.min(window.devicePixelRatio || 1, 2);
  VW = window.innerWidth; VH = window.innerHeight;
  canvas.width = Math.floor(VW * dpr); canvas.height = Math.floor(VH * dpr);
  canvas.style.width = VW + 'px'; canvas.style.height = VH + 'px';
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  ctx.imageSmoothingEnabled = false;
}
```

Telas de celular e notebooks modernos têm mais pixels físicos do que pixels "CSS" (o `devicePixelRatio`, ou DPR, pode ser 2 ou 3). Se o canvas tivesse só 800 pixels numa tela com DPR 2, tudo ficaria borrado. A função cria o canvas com `largura × DPR` pixels reais, mas o exibe com a largura normal, e usa `setTransform` para que o resto do código continue pensando em pixels normais. O DPR é limitado a 2 para não pesar em telas de 4K.

6.3 Transformações: o segredo para desenhar personagens

Desenhar uma espada girada na direção do mouse parece difícil: você teria que calcular onde fica cada canto da lâmina. As **transformações** resolvem isso de forma elegante. Em vez de calcular os pontos, você **move e gira o papel**, desenha a espada "reta", e depois desfaz:

```
ctx.save();                // guarda o estado atual do papel
ctx.translate(heroi.x, heroi.y);  // o (0,0) agora é o centro do herói
ctx.rotate(angulo);        // gira o papel na direção da mira
ctx.fillRect(0, -2, 30, 4); // desenha a lâmina "deitada", apontando para a direita
ctx.restore();             // desfaz o translate e o rotate
```

É exatamente assim que `drawHuman` desenha todos os humanos do jogo. Veja a parte da arma:

```
if (look.weapon && look.weapon !== 'none') {
  ctx.save();
  ctx.translate(dir * 5, -6);      // posição da mão
  let a = aim;
  if (o.swingArc) a += o.swingArc; // durante o golpe, gira um pouco mais
  ctx.rotate(a);
  drawWeapon(ctx, look.weapon, look.weaponColor, o.thrust || 0);
  ctx.restore();
}
```

E a animação de golpe? A espada não tem "quadros de animação" desenhados à mão. Durante os 0,2 s do ataque, `swingArc` vai de −arco/2 até +arco/2, então a espada **varre** o ângulo do golpe. Para a lança, em vez de girar, ela avança e volta (`thrust = Math.sin(k * Math.PI)`).

Como um personagem é montado

A função `drawHuman` desenha, em ordem (o que vem depois fica por cima): sombra no chão → cavalo (se for montado) → capa → pernas → corpo → detalhes (faixa, cruz, gravata) → cabeça e olho → chapéu → escudo → arma. Cada parte é uma forma simples. A variedade vem do objeto `look`:

js/data/base.js linhas 30–30

```
tercio:       { name: 'Soldado do Tercio',            weapon: 'arcabuz',    look: { body: '#c23b22', legs:
'#e0c060', skin: '#e0b090', hat: 'morion', hatColor: '#b8c0c8' } },
```

Trocando `hat: 'morion'` por `hat: 'tricorn'`, o soldado ganha outro chapéu. O jogo tem 17 tipos de chapéu (`drawHat`) e 15 tipos de arma (`drawWeapon`), o que dá centenas de combinações.

A animação de caminhada

As pernas "andam" usando a função **seno**, que oscila suavemente entre −1 e 1:

```
const bob = o.moving ? Math.sin(t * 14) : 0;
ctx.fillRect(-6, 2 + bob * 2, 5, 9 - bob * 2);  // perna esquerda sobe...
ctx.fillRect(1, 2 - bob * 2, 5, 9 + bob * 2);   // ...enquanto a direita desce
```

Seno e cosseno são as ferramentas favoritas dos programadores de jogos para tudo o que **oscila**: pernas, objetos flutuando (`Math.sin(G.time * 4)` nos itens brilhantes), chamas tremulando, o marcador de objetivo pulando.

6.4 A matemática essencial dos jogos

Você não precisa de muita matemática para fazer jogos 2D, mas precisa destes cinco conceitos.

1. Distância entre dois pontos

Pelo teorema de Pitágoras: `Math.hypot(bx - ax, by - ay)`. É a função `dist` do jogo.

2. Vetores e normalização

Um **vetor** é uma seta com direção e tamanho, representada por (dx, dy). Para andar na direção do jogador **com velocidade constante**, o inimigo precisa de um vetor de tamanho 1 (**normalizado**), que se obtém dividindo pelo comprimento:

```
const dx = p.x - e.x, dy = p.y - e.y;
const d = Math.hypot(dx, dy);
const ux = dx / d, uy = dy / d;      // vetor unitário apontando para o jogador
e.x += ux * velocidade * dt;
e.y += uy * velocidade * dt;
```

O mesmo vale para o herói: ao apertar W e D juntos, o movimento seria (1, −1), com tamanho 1,41, ou seja, **41% mais rápido na diagonal**. O jogo normaliza o vetor das teclas para evitar esse bug clássico:

```
const len = Math.hypot(mx, my) || 1;
mx /= len; my /= len;
```

3. Ângulos e radianos

Computadores medem ângulos em **radianos**: uma volta completa é `2π` (≈ 6,28), meia volta é `π`. Para converter: `radianos = graus × π / 180`.

4. Do ângulo para a direção (cos e sin)

Dado um ângulo `a`, a direção correspondente é `(Math.cos(a), Math.sin(a))`. É assim que a classe `Proj` calcula a velocidade da bala: `vx = cos(a) × speed`, `vy = sin(a) × speed`.

5. Da direção para o ângulo (atan2)

O caminho inverso: dado um vetor (dx, dy), qual é o ângulo? `Math.atan2(dy, dx)`. É o que faz o herói **mirar no mouse**:

```
p.aim = Math.atan2(mouseY_no_mundo - p.y, mouseX_no_mundo - p.x);
```

Bônus: diferença entre ângulos

Para saber se um inimigo está **dentro do arco do golpe**, precisamos da diferença entre dois ângulos, mas cuidado: entre 350° e 10° a diferença é 20°, não 340°. A função `angleDiff` resolve isso "dando a volta":

js/util.js — linhas 44–49

```
function angleDiff(a, b) {
  let d = b - a;
  while (d > Math.PI) d -= Math.PI * 2;
  while (d < -Math.PI) d += Math.PI * 2;
  return d;
}
```

6.5 A câmera

O mapa é muito maior que a tela (110 × 80 quadrados de 32 pixels = 3520 × 2560 pixels). A **câmera** é só um par de números (`G.cam.x`, `G.cam.y`) que diz qual parte do mundo aparece no canto superior esquerdo da tela. Para desenhar, o jogo "empurra" tudo na direção contrária:

```
ctx.translate(-camX, -camY);   // tudo desenhado depois aparece deslocado
```

Assim, o código de desenho pode usar **coordenadas do mundo** (onde o lobo realmente está) e a câmera cuida do resto. A câmera segue o herói com suavidade usando **interpolação linear** (`lerp`):

```
G.cam.x = lerp(G.cam.x, alvoX, Math.min(1, dt * 8));
```

`lerp(a, b, t)` devolve um valor entre `a` e `b`: com `t = 0,13`, a câmera anda 13% do caminho até o alvo a cada quadro. O resultado é um acompanhamento macio, que "atrasa" um pouquinho. Depois, `clamp` impede a câmera de mostrar o vazio além das bordas do mapa.

Tremor de tela (screen shake)

Quando o herói leva dano ou um chefe aparece, a tela treme. O truque é somar um deslocamento aleatório à câmera, que diminui com o tempo:

```
const sx = (Math.random() - 0.5) * G.shake;   // até ±shake/2 pixels
G.shake = Math.max(0, G.shake - dt * 30);     // vai sumindo
```

É um efeito de 3 linhas que dá muito "peso" aos impactos. Jogos profissionais usam muito esse tipo de efeito (*game feel*).

6.6 A renderização: a ordem importa

A função `render` desenha o quadro inteiro, sempre na mesma ordem, de trás para a frente, como um pintor:

1. O chão e os obstáculos (o mapa).
2. Os marcos de cada lugar (caverna, aqueduto, castelo).
3. Avisos de área de ataque dos chefes (círculos vermelhos).
4. Objetos no chão: moedas, poções, itens, crônicas, baús.
5. **Personagens ordenados pela altura (y)**: NPCs, inimigos e o herói.
6. Efeitos de ataque (o flash da câmera fotográfica).
7. Projéteis, partículas e números de dano flutuantes.
8. A seta do objetivo e o rótulo do objeto sob o mouse.
9. Por cima de tudo, sem câmera: barra do chefe, minimapa e mira.

O item 5 merece atenção:

```
list.sort((a, b) => a.y - b.y);
```

Ordenar pelo `y` faz com que quem está **mais embaixo na tela seja desenhado por último**, ficando "na frente". Assim, quando o herói passa atrás de um NPC, ele some atrás dele. É uma ilusão de profundidade simples e eficaz, chamada *y-sorting*.

Partículas e textos flutuantes

Partículas (sangue, poeira, faíscas, moedas) são pequenos objetos com posição, velocidade e tempo de vida:

js/game.js — linhas 417–420

```
function particle(x, y, color, spd, life, size) {
  const a = Math.random() * Math.PI * 2, s = Math.random() * (spd || 100);
  G.parts.push({ x, y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life: life || 0.5, max: life || 0.5, color,
size: size || 3 });
}
```

A cada quadro, elas se movem, desaceleram (`vx *= 0.92`) e perdem vida; quando a vida acaba, são removidas com `filter`. Ao desenhar, a transparência acompanha a vida restante, então elas **desaparecem suavemente**. Os números de dano flutuantes funcionam igual, subindo 30 pixels por segundo.

A seta do objetivo

Quando o objetivo está fora da tela, uma seta dourada aparece na borda, apontando para ele, com a distância em metros. Ela usa exatamente a matemática da seção anterior: `atan2` para o ângulo, `cos` / `sin` para posicionar a seta num círculo em volta do herói, e `clamp` para mantê-la dentro da tela.

js/game.js — linhas 1062–1086

```
function drawObjective(t, color) {
  const sx = t.x - G.cam.x, sy = t.y - G.cam.y;
  const onScreen = sx > 30 && sy > 60 && sx < VW - 30 && sy < VH - 30;
  ctx.save();
  ctx.translate(G.cam.x, G.cam.y);
  if (onScreen) {
    const b = Math.sin(G.time * 5) * 4;
    ctx.fillStyle = color;
    ctx.beginPath(); ctx.moveTo(sx, sy - 52 + b); ctx.lineTo(sx - 8, sy - 66 + b); ctx.lineTo(sx + 8, sy -
66 + b); ctx.fill();
  } else {
    const px = G.player.x - G.cam.x, py = G.player.y - G.cam.y;
    const a = Math.atan2(sy - py, sx - px);
    const r = Math.min(VW, VH) / 2 - 50;
    const ax = clamp(px + Math.cos(a) * r, 40, VW - 40), ay = clamp(py + Math.sin(a) * r, 70, VH - 40);
    ctx.translate(ax, ay); ctx.rotate(a);
    ctx.fillStyle = color; ctx.strokeStyle = '#000'; ctx.lineWidth = 2;
    ctx.beginPath(); ctx.moveTo(18, 0); ctx.lineTo(-8, -11); ctx.lineTo(-3, 0); ctx.lineTo(-8, 11);
ctx.closePath(); ctx.fill(); ctx.stroke();
    ctx.rotate(-a);
    const m = Math.round(dist(G.player.x, G.player.y, t.x, t.y) / 16);
    ctx.font = 'bold 12px sans-serif'; ctx.textAlign = 'center';
    ctx.strokeStyle = '#000'; ctx.lineWidth = 3; ctx.strokeText(m + ' m', 0, 26);
    ctx.fillStyle = '#fff'; ctx.fillText(m + ' m', 0, 26);
  }
  ctx.restore();
}
```

> ✍️ **Exercício**
>
> No Console, com uma partida aberta, digite `G.shake = 40`. Depois experimente mudar, em `game.js`, o fator `dt * 8` da câmera para `dt * 2` e recarregue. O que muda? (Resposta: a câmera fica "preguiçosa" e demora a alcançar o herói.)

CAPÍTULO 7

Gerando mundos: mapas procedurais

Cada capítulo tem um mapa de 110 × 80 quadrados, com florestas, lagos, rios, aldeias, estradas e pontes. Nenhum foi desenhado à mão: todos são **gerados por código**, a partir de uma "semente" e de uma lista de lugares importantes. Isso se chama **geração procedural**, a mesma técnica de jogos como Minecraft, Terraria e No Man's Sky.

7.1 Mapas de tiles

O mundo é uma grade de quadradinhos de 32 × 32 pixels chamados **tiles** (ladrilhos). Cada tile tem um tipo:

js/world.js linhas 4–4

```
const TL = { G: 0, P: 1, W: 2, O: 3, B: 4, F: 5, BR: 6, ROAD: 7, SIDE: 8 };
```

js/world.js linhas 5–5

```
const WALKABLE = [true, true, false, false, false, true, true, true, true];
```

js/world.js linhas 6–6

```
const TALL = [false, false, false, true, true, false, false, false, false];
```

Código	Tipo	Dá para andar?	Bloqueia tiros?
`G` (0)	Chão (grama, terra, areia)	sim	não
`P` (1)	Caminho	sim	não
`W` (2)	Água	não	não (tiros passam por cima)
`O` (3)	Obstáculo (árvore, rocha, coluna...)	não	sim
`B` (4)	Construção (casa, prédio)	não	sim
`F` (5)	Piso (praça de aldeia)	sim	não
`BR` (6)	Ponte sobre a água	sim	não
`ROAD` (7)	Asfalto (capítulos modernos)	sim	não
`SIDE` (8)	Calçada	sim	não

As duas listas `WALKABLE` e `TALL` respondem às duas perguntas da tabela. Usar o número do tipo como **índice** de um array (`WALKABLE[tipo]`) é uma forma rapidíssima de consultar propriedades.

Guardando a grade: um array de uma dimensão

Uma grade 2D poderia ser um "array de arrays" (`mapa[y][x]`), mas o jogo usa um único array comprido, com as linhas uma depois da outra:

```
this.t = new Uint8Array(this.W * this.H);   // 110 × 80 = 8800 tiles
idx(x, y) { return y * this.W + x; }         // posição (x, y) → índice no array
```

`Uint8Array` é um array especial que só guarda números inteiros de 0 a 255, ocupando 1 byte cada. É compacto e muito rápido. A fórmula `y * W + x` "pula" `y` linhas inteiras e anda `x` casas.

De pixels para tiles e de volta

```
const tx = Math.floor(px / TILE);   // pixel → tile (arredonda para baixo)
const px = tx * TILE + TILE / 2;    // tile → pixel do CENTRO do tile
```

O herói vive em **pixels** (precisa de movimento suave), o mapa vive em **tiles**. Essas duas linhas fazem a ponte e aparecem dezenas de vezes no código.

7.2 Aleatoriedade controlada: sementes

Se o mapa fosse gerado com `Math.random()`, ele seria diferente **cada vez** que o jogo abrisse. Isso quebraria tudo: o baú que você abriu ontem estaria em outro lugar hoje. A solução é um **gerador de números pseudoaleatórios com semente**: uma fórmula que produz uma sequência que *parece* aleatória, mas que é **sempre a mesma** para a mesma semente.

js/util.js — linhas 6–15

```
function mulberry32(seed) {
  let a = seed >>> 0;
  return function () {
    a = (a + 0x6D2B79F5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

Cada capítulo tem um campo `seed` (o capítulo 1 usa 1101). `mulberry32(1101)` sempre produz a mesma sequência de números, então o mapa do capítulo 1 é **idêntico em todos os computadores do mundo, para sempre**. Mesmo assim, não foi preciso desenhar nada. O conteúdo da função (multiplicações e operações de bits como `^` e `>>>`) é uma receita conhecida para "embaralhar" bem os bits; você não precisa entendê-la para usá-la.

Há também o `hash2(x, y)`, que dá um número "aleatório" fixo para cada coordenada. É usado para variar detalhes visuais de cada tile (qual tom de grama, se tem florzinha) sem precisar guardar nada.

> ❓ **Por quê?**
>
> **Por que não guardar o mapa pronto num arquivo?** 14 mapas de 8.800 tiles, com variações, seriam muitos dados e daria trabalho editá-los. Com sementes, cada mapa ocupa **um número** (a semente) mais a lista de lugares. Mudar a semente gera um mapa novo instantaneamente.

7.3 Ruído: florestas que parecem naturais

Se cada tile virasse árvore com 30% de chance, o resultado seria "chuvisco": árvores espalhadas uniformemente, sem bosques nem clareiras. A natureza tem **agrupamentos**. Para imitar isso, o jogo usa **ruído de valor** (*value noise*):

js/util.js — linhas 24–37

```
function makeNoise(rng, w, h, cell) {
  const gw = Math.ceil(w / cell) + 2, gh = Math.ceil(h / cell) + 2;
  const g = new Float32Array(gw * gh);
  for (let i = 0; i < g.length; i++) g[i] = rng();
  return function (x, y) {
    const fx = x / cell, fy = y / cell;
    const ix = Math.floor(fx), iy = Math.floor(fy);
    const tx = fx - ix, ty = fy - iy;
    const sx = tx * tx * (3 - 2 * tx), sy = ty * ty * (3 - 2 * ty);
    const a = g[iy * gw + ix], b = g[iy * gw + ix + 1];
    const c = g[(iy + 1) * gw + ix], d = g[(iy + 1) * gw + ix + 1];
    return (a + (b - a) * sx) * (1 - sy) + (c + (d - c) * sx) * sy;
  };
}
```

A ideia, em português:

1. Crie uma grade **grossa** (um ponto a cada 9 ou 15 tiles) com números aleatórios entre 0 e 1.
2. Para qualquer posição, ache os 4 pontos da grade grossa ao redor e **misture** seus valores de acordo com a distância (interpolação bilinear).
3. A curva `t * t * (3 - 2 * t)` (chamada *smoothstep*) suaviza as transições.

O resultado é uma "paisagem" de valores que sobem e descem suavemente. O jogo usa o ruído assim: onde o valor passa de 0,56, há chance de floresta; onde outro ruído passa de 0,74, há lago. Como os valores altos vêm em "manchas", surgem bosques e lagos com formas orgânicas.

7.4 A geração passo a passo: World.generate

A classe `World` recebe os dados de um capítulo e monta o mapa em etapas. Veja o método inteiro e depois a explicação:

js/world.js — linhas 26–128

```
generate() {
  const { W, H, b } = this;
  const rng = mulberry32(this.ch.seed);
  const nObs = makeNoise(rng, W, H, 9);
  const nWat = makeNoise(rng, W, H, 15);
```

```
    const obsCount = b.obs.length;

    if (b.urban) {
      for (let y = 0; y < H; y++) for (let x = 0; x < W; x++) {
        const rx = x % 16, ry = y % 14;
        let v;
        if (rx < 3 || ry < 3) v = TL.ROAD;
        else if (rx === 3 || ry === 3 || rx === 15 || ry === 13) v = TL.SIDE;
        else v = TL.B;
        this.t[this.idx(x, y)] = v;
      }
      // Alguns quarteirões viram praças e parques
      for (let by = 0; by < H; by += 14) for (let bx = 0; bx < W; bx += 16) {
        const r = rng();
        if (r < 0.3) {
          for (let y = by + 4; y < by + 13; y++) for (let x = bx + 4; x < bx + 15; x++) {
            if (x < W && y < H) {
              this.t[this.idx(x, y)] = rng() < 0.18 ? TL.O : TL.G;
              this.k[this.idx(x, y)] = Math.floor(rng() * obsCount);
            }
          }
        }
      }
    } else {
      for (let y = 0; y < H; y++) for (let x = 0; x < W; x++) {
        const i = this.idx(x, y);
        let v = TL.G;
        if (b.lakes && nWat(x, y) > 0.74) v = TL.W;
        else if (nObs(x, y) > (b.thr || 0.56) && rng() < (b.density || 0.55)) v = TL.O;
        else if (rng() < (b.scatter || 0.025)) v = TL.O;
        this.t[i] = v;
        this.k[i] = Math.floor(rng() * obsCount);
      }
      if (b.sea) {
        for (let x = 0; x < W; x++) {
          const depth = 6 + Math.floor(nWat(x, 3) * 6);
          for (let d = 0; d < depth; d++) {
            if (b.sea === 'south') this.set(x, H - 1 - d, TL.W);
            else if (b.sea === 'north') this.set(x, d, TL.W);
          }
        }
        for (let y = 0; y < H; y++) {
          const depth = 6 + Math.floor(nWat(3, y) * 6);
          for (let d = 0; d < depth; d++) {
            if (b.sea === 'east') this.set(W - 1 - d, y, TL.W);
            else if (b.sea === 'west') this.set(d, y, TL.W);
          }
        }
      }
      if (b.river) {
        let x = Math.floor(W * 0.55);
        for (let y = 0; y < H; y++) {
          x += Math.round((rng() - 0.5) * 2);
          x = clamp(x, 10, W - 10);
          for (let d = -1; d <= 1; d++) this.set(x + d, y, TL.W);
        }
      }
    }

    // Borda intransponível
    for (let y = 0; y < H; y++) for (let x = 0; x < W; x++) {
```

```
    if (x < 2 || y < 2 || x >= W - 2 || y >= H - 2) this.t[this.idx(x, y)] = b.urban ? TL.B : TL.O;
  }

  // Lugares
  for (const p of this.ch.places) {
    const r = p.r || 6;
    for (let y = -r; y <= r; y++) for (let x = -r; x <= r; x++) {
      if (x * x + y * y <= r * r) {
        const tx = p.x + x, ty = p.y + y;
        if (tx < 2 || ty < 2 || tx >= W - 2 || ty >= H - 2) continue;
        this.set(tx, ty, (p.type === 'village' || p.type === 'city') ? TL.F : (b.urban ? TL.SIDE : TL.G));
      }
    }
    if (p.type === 'village' && !b.urban) {
      const n = 5;
      for (let i = 0; i < n; i++) {
        const a = (i / n) * Math.PI * 2 + rng() * 0.5 + 0.3;
        const hx = Math.round(p.x + Math.cos(a) * (r - 1.5)) - 1;
        const hy = Math.round(p.y + Math.sin(a) * (r - 1.5)) - 1;
        for (let y = 0; y < 2; y++) for (let x = 0; x < 3; x++) this.set(hx + x, hy + y, TL.B);
      }
    }
    this.places[p.id] = Object.assign({}, p, { r, px: p.x * TILE + TILE / 2, py: p.y * TILE + TILE / 2 });
  }

  // Caminhos ligando os lugares em sequência + alguns atalhos
  const ps = this.ch.places;
  const links = [];
  for (let i = 0; i < ps.length - 1; i++) links.push([ps[i], ps[i + 1]]);
  for (let i = 0; i < ps.length - 2; i += 2) links.push([ps[i], ps[i + 2]]);
  for (const [a, c] of links) this.carvePath(a.x, a.y, c.x, c.y, rng);

  // Garante que o centro de cada lugar seja livre
  for (const p of ps) this.set(p.x, p.y, (p.type === 'village' || p.type === 'city') ? TL.F : (b.urban ?
TL.SIDE : TL.G));

  this.computeReach();
}
```

1. **Preparação**: cria o gerador com a semente do capítulo e dois ruídos (obstáculos e água).
2. **Terreno base**:
   - Nos capítulos **urbanos** (12 a 14), desenha uma grade de ruas a cada 16 tiles na horizontal e 14 na vertical, com calçadas e prédios no meio dos quarteirões. Cerca de 30% dos quarteirões viram praças e parques.
   - Nos outros, cada tile vira água (se o ruído de água for alto, e o capítulo tiver lagos), obstáculo (se o ruído de obstáculo for alto e um sorteio passar) ou chão.
3. **Mar e rio** (opcionais, conforme o bioma): uma faixa de água de largura irregular em um dos lados; um rio serpenteando de cima a baixo.
4. **Bordas**: duas fileiras de obstáculos em volta de todo o mapa, para ninguém sair do mundo.
5. **Lugares**: para cada lugar da lista do capítulo (aldeia, acampamento, ruína, arena), limpa um círculo de raio `r` e, nas aldeias, ergue 5 casas de 3 × 2 tiles em volta da praça.
6. **Caminhos**: liga os lugares em sequência (1→2, 2→3...) e alguns atalhos (1→3, 3→5...), cavando estradas.
7. **Alcance**: calcula quais tiles podem ser alcançados a pé a partir do ponto de partida.

Cavando estradas: carvePath

js/world.js linhas 130–148

```
carvePath(x0, y0, x1, y1, rng) {
  let x = x0, y = y0;
  let guard = 0;
  while ((x !== x1 || y !== y1) && guard++ < 4000) {
    const dx = Math.sign(x1 - x), dy = Math.sign(y1 - y);
    if (dx !== 0 && (dy === 0 || rng() < 0.5)) x += dx; else y += dy;
    if (rng() < 0.12) { const s = rng() < 0.5 ? 1 : -1; if (rng() < 0.5) x += s; else y += s; }
    x = clamp(x, 2, this.W - 3); y = clamp(y, 2, this.H - 3);
    for (let oy = 0; oy <= 1; oy++) for (let ox = 0; ox <= 1; ox++) {
      const tx = x + ox, ty = y + oy;
      if (tx < 2 || ty < 2 || tx >= this.W - 2 || ty >= this.H - 2) continue;
      const cur = this.get(tx, ty);
      if (cur === TL.W) this.set(tx, ty, TL.BR);
      else if (cur === TL.F || cur === TL.BR || cur === TL.ROAD) continue;
      else if (this.b.urban) this.set(tx, ty, TL.ROAD);
      else this.set(tx, ty, TL.P);
    }
  }
}
```

O algoritmo é um "andarilho teimoso": a cada passo, ele anda um tile na direção do destino (às vezes na horizontal, às vezes na vertical) e, em 12% das vezes, dá um passo para o lado, o que deixa a estrada **sinuosa** em vez de reta. Por onde passa, abre uma faixa de 2 tiles: se havia água, vira **ponte**; se havia mato ou árvore, vira **caminho**. Isso garante que todos os lugares importantes estejam ligados.

Quais lugares dá para alcançar? Busca em largura

Mesmo com estradas, um sorteio infeliz pode criar um "bolsão" de chão cercado de árvores, de onde não se sai. Se um item de missão nascesse ali, o jogador ficaria preso. O método `computeReach` resolve isso com um algoritmo clássico de computação, a **busca em largura** (*BFS*, ou *flood fill*, "inundação"):

js/world.js linhas 150–167

```
computeReach() {
  const { W, H } = this;
  this.reach = new Uint8Array(W * H);
  const s = this.ch.places[0];
  const q = [s.x + s.y * W];
  this.reach[q[0]] = 1;
  while (q.length) {
    const i = q.pop();
    const x = i % W, y = (i / W) | 0;
    const nb = [i - 1, i + 1, i - W, i + W];
    const ok = [x > 0, x < W - 1, y > 0, y < H - 1];
    for (let k = 0; k < 4; k++) {
      if (!ok[k]) continue;
      const j = nb[k];
      if (!this.reach[j] && WALKABLE[this.t[j]]) { this.reach[j] = 1; q.push(j); }
    }
  }
}
```

Funciona como derramar tinta no ponto de partida:

1. Marque o tile inicial e coloque-o numa fila de "a visitar".
2. Enquanto a fila não estiver vazia: tire um tile e olhe os 4 vizinhos (esquerda, direita, cima, baixo). Para cada vizinho que é andável e ainda não foi marcado, marque e coloque na fila.
3. No fim, todos os tiles marcados em `reach` são alcançáveis.

A partir daí, toda função que escolhe um ponto aleatório para colocar algo (item de missão, inimigo, baú, crônica) **só aceita tiles marcados**. É uma garantia matemática de que tudo pode ser alcançado.

js/world.js — linhas 170–180

```
randomSpot(px, py, rTiles, minR) {
  const cx = Math.floor(px / TILE), cy = Math.floor(py / TILE);
  for (let tries = 0; tries < 300; tries++) {
    const a = Math.random() * Math.PI * 2;
    const d = (minR || 0) + Math.random() * (rTiles - (minR || 0));
    const tx = Math.round(cx + Math.cos(a) * d), ty = Math.round(cy + Math.sin(a) * d);
    if (tx < 2 || ty < 2 || tx >= this.W - 2 || ty >= this.H - 2) continue;
    if (this.reach[this.idx(tx, ty)]) return { x: tx * TILE + TILE / 2, y: ty * TILE + TILE / 2 };
  }
  return { x: px, y: py };
}
```

> 📌 **Nota**
>
> O teste automático do capítulo 15 verifica, para os 14 mapas, que todos os lugares e todos os NPCs são alcançáveis. Foi assim que se garantiu que nenhuma missão fica impossível.

7.5 Colisão com o mapa

Para saber se o herói pode ir para uma posição, o jogo testa os **4 cantos** de um quadrado em volta dele:

js/world.js — linhas 202–205

```
collides(x, y, r) {
  return this.solidAt(x - r, y - r) || this.solidAt(x + r, y - r) ||
         this.solidAt(x - r, y + r) || this.solidAt(x + r, y + r);
}
```

E o movimento é feito **um eixo de cada vez**:

js/game.js — linhas 495–499

```
function moveEnt(ent, mx, my) {
  const w = G.world, r = ent.r * 0.8;
  if (mx) { const nx = ent.x + mx; if (!w.collides(nx, ent.y, r)) ent.x = nx; }
  if (my) { const ny = ent.y + my; if (!w.collides(ent.x, ny, r)) ent.y = ny; }
}
```

> ❓ **Por quê?**
>
> **Por que mover x e y separadamente?** Imagine o herói andando na diagonal contra uma parede vertical. Se o jogo testasse o movimento diagonal inteiro, ele bateria e **pararia completamente**. Testando separado, o movimento em x é bloqueado pela parede, mas o movimento em y continua: o herói **desliza** ao longo da parede. É um detalhe pequeno que faz o controle parecer muito mais agradável.

O raio usado é 80% do raio do personagem (`ent.r * 0.8`), o que deixa os personagens passarem um pouquinho "por dentro" das quinas, evitando que fiquem presos em cantos.

7.6 Desenhando o mapa rápido: o cache em blocos (chunks)

Desenhar milhares de tiles, cada um com árvores e detalhes, **a cada quadro** seria pesado. O truque: desenhar cada pedaço do mapa **uma única vez** num canvas invisível e, depois, só copiar essa "foto" para a tela.

js/world.js — linhas 208–223

```
chunk(cx, cy) {
  const key = cx + ',' + cy;
  let c = this.chunks.get(key);
  if (c) return c;
  const CS = 16;
  c = document.createElement('canvas');
  c.width = c.height = CS * TILE;
  const g = c.getContext('2d');
  for (let y = 0; y < CS; y++) for (let x = 0; x < CS; x++) {
    const tx = cx * CS + x, ty = cy * CS + y;
    if (tx >= this.W || ty >= this.H) continue;
    this.drawTile(g, tx, ty, x * TILE, y * TILE);
  }
  this.chunks.set(key, c);
  return c;
}
```

O mapa é dividido em **chunks** (blocos) de 16 × 16 tiles (512 × 512 pixels). Na primeira vez que um bloco aparece na tela, `chunk()` cria um canvas fora da tela, desenha os 256 tiles nele e o guarda num `Map` (um dicionário). Nas vezes seguintes, só devolve o que já estava guardado. O método `draw` então copia os 4 a 9 blocos visíveis com `drawImage`, que é extremamente rápido.

js/world.js — linhas 359–369

```
draw(ctx, camX, camY, vw, vh) {
  const CSP = 16 * TILE;
  const x0 = Math.max(0, Math.floor(camX / CSP)), y0 = Math.max(0, Math.floor(camY / CSP));
  const x1 = Math.floor((camX + vw) / CSP), y1 = Math.floor((camY + vh) / CSP);
  ctx.fillStyle = '#111';
  ctx.fillRect(0, 0, vw, vh);
  for (let cy = y0; cy <= y1; cy++) for (let cx = x0; cx <= x1; cx++) {
    if (cx * 16 >= this.W || cy * 16 >= this.H) continue;
    ctx.drawImage(this.chunk(cx, cy), Math.round(cx * CSP - camX), Math.round(cy * CSP - camY));
  }
}
```

Esse padrão (calcular uma vez e reaproveitar) se chama **cache** e é uma das otimizações mais importantes da programação.

Desenhando cada tile

`drawTile` decide a cor pelo tipo do tile e usa o `hash2` para variar detalhes de forma estável. `drawObstacle` desenha 15 tipos de obstáculo com formas simples:

```
case 'pine':   // pinheiro: três triângulos verdes empilhados sobre um tronco
  ...
  for (let i = 0; i < 3; i++) {
    g.beginPath();
    g.moveTo(cx, cy - 16 + i * 8);
    g.lineTo(cx - 13 + i, cy + 2 + i * 4);
    g.lineTo(cx + 13 - i, cy + 2 + i * 4);
    g.fill();
  }
```

Cada capítulo escolhe quais obstáculos usar no seu **bioma**:

js/data/chapters1.js — linhas 33–34

```
biome: { ground: '#4f7f3c', ground2: '#4a7838', path: '#8a7350', water: '#2f6a9a', floor: '#7a6a4a', roof:
'#6a4a2a', wall: '#6a4a2a',
  obs: ['tree', 'pine', 'rock', 'bush'], density: 0.6, lakes: true, flowers: '#e8d870' },
```

Mudando só esses números, o mesmo gerador cria a floresta verde da Cantábria, a meseta seca dos celtiberos, as montanhas nevadas das Astúrias e as ruas de Madri.

Marcos (landmarks)

Cada lugar pode ter um `icon` (caverna, aqueduto, castelo, mesquita, navio, parlamento...) desenhado por `drawLandmark` no centro do lugar. São desenhos maiores, feitos com as mesmas formas simples, que dão identidade a cada local.

O minimapa

O minimapa é feito pixel a pixel com `ImageData`, que dá acesso direto às cores de cada pixel de uma imagem: 1 pixel por tile, com a cor do tipo do tile. É criado uma vez (cache, de novo) e depois desenhado em tamanho maior no canto da tela e na janela do mapa (tecla M).

js/world.js — linhas 371–391

```
minimap() {
  if (this._mini) return this._mini;
  const c = document.createElement('canvas');
  c.width = this.W; c.height = this.H;
  const g = c.getContext('2d');
  const img = g.createImageData(this.W, this.H);
  const col = (hex) => { const n = parseInt(hex.slice(1), 16); return [n >> 16, (n >> 8) & 255, n & 255]; };
  const b = this.b;
  const pal = {
    [TL.G]: col(b.ground), [TL.P]: col(b.path), [TL.W]: col(b.water), [TL.O]: col(shade(b.ground, -45)),
    [TL.B]: col(b.urban ? shade(b.wall, -20) : b.roof), [TL.F]: col(b.floor), [TL.BR]: [138, 106, 68],
    [TL.ROAD]: [70, 70, 75], [TL.SIDE]: [150, 150, 145]
  };
```

```
  for (let i = 0; i < this.t.length; i++) {
    const p = pal[this.t[i]];
    img.data[i * 4] = p[0]; img.data[i * 4 + 1] = p[1]; img.data[i * 4 + 2] = p[2]; img.data[i * 4 + 3] =
255;
  }
  g.putImageData(img, 0, 0);
  this._mini = c;
  return c;
}
```

> ✍️ **Exercício**
>
> Em `js/data/chapters1.js`, mude a `seed` do capítulo 1 de `1101` para `42` e recarregue começando um jogo novo. O mapa inteiro muda, mas as aldeias e lugares continuam nos mesmos pontos e ligados por estradas. Depois volte para 1101. (Se você mudar a semente com um save em andamento, baús e crônicas mudam de lugar, porque as posições deles também vêm da semente.)

CAPÍTULO 8

Controles, movimento e combate

8.1 Lendo o teclado

O navegador avisa quando uma tecla é **apertada** (`keydown`) e quando é **solta** (`keyup`). Para movimentação, o jogo não reage ao evento em si: ele **anota** quais teclas estão pressionadas num objeto `G.keys` e, a cada quadro, consulta essa anotação.

js/game.js — linhas 1133–1156

```
window.addEventListener('keydown', (e) => {
  if (e.target && (e.target.tagName === 'INPUT' || e.target.tagName === 'TEXTAREA')) return;
  if (['Space', 'Tab', 'ArrowUp', 'ArrowDown', 'ArrowLeft', 'ArrowRight'].includes(e.code))
e.preventDefault();
  if (UI.onKey(e)) return;
  if (!G.running || G.ui) return;
  G.keys[e.code] = true;
  switch (e.code) {
    case 'Space':
      if (G.player.dashCd <= 0) {
        const p = G.player;
        let dx = 0, dy = 0;
        if (G.keys.KeyA || G.keys.ArrowLeft) dx -= 1; if (G.keys.KeyD || G.keys.ArrowRight) dx += 1;
        if (G.keys.KeyW || G.keys.ArrowUp) dy -= 1; if (G.keys.KeyS || G.keys.ArrowDown) dy += 1;
        if (!dx && !dy) { dx = Math.cos(p.aim); dy = Math.sin(p.aim); }
        const l = Math.hypot(dx, dy);
        p.dashX = dx / l; p.dashY = dy / l; p.dashT = 0.16; p.dashCd = 0.75;
        Sfx.play('dash');
      }
      break;
    case 'KeyE': interact(nearestInteractable()); break;
    case 'KeyQ': usePotion(); break;
    case 'KeyR': startReload(); break;
  }
});
```

js/game.js — linhas 1157–1157

```
window.addEventListener('keyup', (e) => { G.keys[e.code] = false; });
```

Alguns detalhes importantes desse código:

- `e.code` **em vez de** `e.key`**.** `e.key` é a *letra* produzida (`'w'`, `'W'`, `'ẃ'`...), que muda com o layout do teclado e o Caps Lock. `e.code` é a *posição física* da tecla (`'KeyW'`). Usando `e.code`, o WASD funciona em teclados ABNT, americanos, franceses (AZERTY), qualquer um.
- `e.preventDefault()` para Espaço, Tab e setas impede o comportamento padrão do navegador (rolar a página, trocar de foco).
- `UI.onKey(e)` **é consultado primeiro.** Se uma janela está aberta, ela "consome" a tecla (Espaço avança o diálogo em vez de esquivar).

- Ações instantâneas (esquiva, interagir, poção, recarregar) acontecem **no evento**, porque devem ocorrer uma vez por toque. Movimentação acontece **no update**, porque é contínua.
- O evento `blur` (a janela perdeu o foco) **zera todas as teclas**. Sem isso, se você trocar de janela com o W apertado, o herói andaria para sempre, porque o `keyup` nunca chegaria.

O mouse

js/game.js — linhas 1160–1167

```
canvas.addEventListener('mousedown', (e) => {
  Sfx.init();
  if (!G.running || G.ui) return;
  if (e.button !== 0) return;
  if (G.hover && G.hoverInRange) { interact(G.hover); return; }
  G.mouse.down = true;
  playerAttack();
});
```

Ao clicar, se o mouse estiver sobre algo interativo **e perto o bastante** (90 pixels), o clique interage (fala com o NPC, abre o baú). Senão, ataca. `G.mouse.down` fica verdadeiro enquanto o botão estiver pressionado, o que permite **atacar sem parar** segurando o botão.

> 📌 **Nota**
>
> A posição do mouse vem em coordenadas **da tela**. Para saber onde ele está **no mundo**, soma-se a câmera: `mundoX = G.mouse.x + G.cam.x`. Essa conversão aparece na mira e na detecção do que está sob o cursor.

8.2 A função update: um quadro de lógica

`update(dt)` é a função mais longa do jogo, mas é apenas uma sequência de pequenas tarefas, sempre na mesma ordem. Veja-a inteira e depois o roteiro:

js/game.js — linhas 545–672

```
function update(dt) {
  const p = G.player, S = G.S;
  S.stats.time += dt;
  G.time += dt;

  // movimento: W cima, S baixo, A esquerda, D direita
  let mx = 0, my = 0;
  if (G.keys.KeyA || G.keys.ArrowLeft) mx -= 1;
  if (G.keys.KeyD || G.keys.ArrowRight) mx += 1;
  if (G.keys.KeyW || G.keys.ArrowUp) my -= 1;
  if (G.keys.KeyS || G.keys.ArrowDown) my += 1;
  const len = Math.hypot(mx, my) || 1;
  mx /= len; my /= len;
  p.moving = mx !== 0 || my !== 0;
  const speed = 175;
  if (p.dashT > 0) {
    p.dashT -= dt;
    moveEnt(p, p.dashX * speed * 3.2 * dt, p.dashY * speed * 3.2 * dt);
    if (Math.random() < 0.6) particle(p.x, p.y + 8, '#ddd', 30, 0.3);
  } else {
    moveEnt(p, mx * speed * dt, my * speed * dt);
```

```
  }
  if (p.moving) p.walkT += dt;
  p.cd -= dt; p.atkT -= dt; p.invT -= dt; p.dashCd -= dt; p.hurtT -= dt;
  if (p.reloadT > 0) { p.reloadT -= dt; if (p.reloadT <= 0) { p.ammo = weapon().mag; p.reloadT = 0; } }
  p.aim = Math.atan2(G.mouse.y + G.cam.y - (p.y - 6), G.mouse.x + G.cam.x - p.x);

  if (G.mouse.down && !G.hoverInRange) playerAttack();

  // inimigos
  const near = [];
  for (const e of G.enemies) {
    if (e.dead) continue;
    if (Math.abs(e.x - p.x) > 1300 || Math.abs(e.y - p.y) > 1000) continue;
    near.push(e);
    updateEnemy(e, dt);
  }
  // separação
  for (let i = 0; i < near.length; i++) for (let j = i + 1; j < near.length; j++) {
    const a = near[i], b = near[j];
    const dx = b.x - a.x, dy = b.y - a.y, d = Math.hypot(dx, dy), m = a.r + b.r;
    if (d > 0 && d < m) {
      const push = (m - d) / 2, ux = dx / d, uy = dy / d;
      if (!a.d.stationary) moveEnt(a, -ux * push, -uy * push);
      if (!b.d.stationary) moveEnt(b, ux * push, uy * push);
    }
  }
  G.enemies = G.enemies.filter(e => !e.dead);

  // respawn de acampamentos
  for (const camp of G.camps) {
    camp.respawnT -= dt;
    if (camp.respawnT > 0) continue;
    const alive = G.enemies.reduce((n, e) => n + (e.campId === camp.id ? 1 : 0), 0);
    if (alive < camp.n) { spawnCampEnemy(camp, true); camp.respawnT = 5; }
    else camp.respawnT = 2;
  }

  // projéteis
  for (const pr of G.projs) {
    pr.x += pr.vx * dt; pr.y += pr.vy * dt; pr.life -= dt;
    if (G.world.tallAt(pr.x, pr.y)) { pr.life = 0; for (let i = 0; i < 3; i++) particle(pr.x, pr.y, '#bbb',
60, 0.2); continue; }
    if (pr.owner === 'p') {
      for (const e of G.enemies) {
        if (e.dead) continue;
        if (dist(pr.x, pr.y, e.x, e.y - 6) < e.r + pr.r) {
          const l = Math.hypot(pr.vx, pr.vy);
          damageEnemy(e, pr.dmg, pr.vx / l, pr.vy / l);
          pr.life = 0; break;
        }
      }
    } else if (dist(pr.x, pr.y, p.x, p.y - 6) < p.r + pr.r) {
      hurtPlayer(pr.dmg, pr.x - pr.vx, pr.y - pr.vy);
      pr.life = 0;
    }
  }
  G.projs = G.projs.filter(pr => pr.life > 0);

  // objetos
  for (const o of G.objs) {
    if (o.kind === 'coin') {
```

```
      o.t += dt;
      const d = dist(p.x, p.y, o.x, o.y);
      if (d < 130) { const a = Math.atan2(p.y - o.y, p.x - o.x); o.x += Math.cos(a) * 320 * dt; o.y +=
Math.sin(a) * 320 * dt; }
      if (d < 16) { o.gone = true; S.gold += o.value; Sfx.play('coin'); }
    } else if (o.kind === 'potion') {
      if (dist(p.x, p.y, o.x, o.y) < 22) { o.gone = true; S.potions++; Sfx.play('pickup'); floatText(o.x,
o.y - 20, T('potionFloat'), '#f66'); }
    } else if (o.kind === 'item') {
      if (dist(p.x, p.y, o.x, o.y) < 24) pickItem(o);
    }
  }
  G.objs = G.objs.filter(o => !o.gone);

  // efeitos
  for (const pt of G.parts) { pt.x += pt.vx * dt; pt.y += pt.vy * dt; pt.vx *= 0.92; pt.vy *= 0.92; pt.life
-= dt; }
  G.parts = G.parts.filter(pt => pt.life > 0);
  for (const t of G.texts) { t.y -= 30 * dt; t.life -= dt; }
  G.texts = G.texts.filter(t => t.life > 0);
  for (const f of G.fx) f.life -= dt;
  G.fx = G.fx.filter(f => f.life > 0);

  checkGotoAndBoss();

  // câmera
  const tx = p.x - VW / 2, ty = p.y - VH / 2;
  G.cam.x = lerp(G.cam.x, tx, Math.min(1, dt * 8));
  G.cam.y = lerp(G.cam.y, ty, Math.min(1, dt * 8));
  G.cam.x = clamp(G.cam.x, 0, Math.max(0, G.world.W * TILE - VW));
  G.cam.y = clamp(G.cam.y, 0, Math.max(0, G.world.H * TILE - VH));
  G.shake = Math.max(0, G.shake - dt * 30);

  // hover
  const wx = G.mouse.x + G.cam.x, wy = G.mouse.y + G.cam.y;
  G.hover = null; G.hoverInRange = false;
  let best = 36;
  for (const o of G.objs) {
    if (o.kind === 'coin' || o.kind === 'potion') continue;
    if (o.kind === 'chest' && o.opened) continue;
    const d = dist(wx, wy, o.x, o.y - 10);
    if (d < best) { best = d; G.hover = o; }
  }
  if (G.hover) G.hoverInRange = dist(p.x, p.y, G.hover.x, G.hover.y) < INTERACT_RANGE;

  G.saveT += dt;
  if (G.saveT > 20) { G.saveT = 0; save(); }
  G.hudT -= dt;
  if (G.hudT <= 0) { G.hudT = 0.1; UI.hud(); }
}
```

Roteiro do que acontece a cada quadro:

1. **Tempo**: soma `dt` ao tempo de jogo (as estatísticas da tela de pausa vêm daqui).
2. **Movimento do herói**: monta o vetor das teclas WASD, normaliza (para a diagonal não ser mais rápida) e move. Durante a esquiva, move 3,2 vezes mais rápido na direção guardada e solta "poeira".
3. **Cronômetros**: diminui recarga de ataque, invulnerabilidade, esquiva, recarga da arma.
4. **Mira**: calcula o ângulo até o mouse com `atan2`.

5. **Ataque contínuo**, se o botão está pressionado.
6. **Inimigos**: atualiza só os que estão a até ~1300 pixels (os distantes ficam "congelados", economizando processamento) e os empurra uns para longe dos outros para não se sobreporem. Remove os mortos.
7. **Acampamentos**: repõe inimigos que faltam (detalhes no próximo capítulo).
8. **Projéteis**: move cada um, verifica se bateu em parede ou acertou alguém.
9. **Objetos**: moedas são atraídas pelo herói como ímã; poções e itens de missão são coletados ao encostar.
10. **Efeitos**: partículas, números flutuantes e flashes envelhecem e somem.
11. **Missões**: verifica se o herói chegou a algum lugar de missão ou à arena de um chefe.
12. **Câmera**: segue o herói suavemente e o tremor diminui.
13. **Hover**: descobre qual objeto está sob o mouse (para o rótulo e o clique).
14. **Salvamento automático** a cada 20 segundos, e o HUD é atualizado 10 vezes por segundo.

> ❓ **Por quê?**
>
> **Por que atualizar o HUD só 10 vezes por segundo?** Mexer em elementos HTML é bem mais lento que desenhar no canvas. Atualizar textos 60 vezes por segundo seria desperdício: o olho não percebe diferença entre 10 e 60 atualizações numa barra de vida. O cronômetro `G.hudT` controla isso. Isso se chama **limitar a frequência** (*throttling*).

8.3 Ataques: três tipos de arma

Todas as 14 armas do jogo se encaixam em três "tipos" (`kind`), definidos em `base.js`:

js/data/base.js — linhas 5-20

```
const WEAPONS = {
  lanca:     { name: 'Lança de sílex',     kind: 'melee', dmg: 16, range: 82, arc: 0.6, cd: 0.5, draw:
'spear', color: '#a0784a' },
  falcata:   { name: 'Falcata ibérica',    kind: 'melee', dmg: 15, range: 58, arc: 1.9, cd: 0.4, draw:
'falcata', color: '#cfd6dc' },
  gladio:    { name: 'Gládio e escudo',    kind: 'melee', dmg: 15, range: 54, arc: 1.7, cd: 0.32, draw:
'sword', color: '#d9dee2', shield: '#a32020', def: 0.12 },
  spatha:    { name: 'Spatha visigoda',    kind: 'melee', dmg: 18, range: 64, arc: 1.9, cd: 0.45, draw:
'sword', color: '#c0c8d0' },
  espada:    { name: 'Espada e escudo',    kind: 'melee', dmg: 17, range: 58, arc: 1.8, cd: 0.38, draw:
'sword', color: '#d0d6dc', shield: '#3b6b2c', def: 0.15 },
  montante:  { name: 'Espada longa',       kind: 'melee', dmg: 21, range: 74, arc: 2.1, cd: 0.5, draw:
'longsword', color: '#e2e6ea' },
  espingarda:{ name: 'Espingarda de mecha',kind: 'gun', dmg: 42, cd: 0.3, mag: 1, reload: 1.3, speed: 760,
spread: 0.04, draw: 'gun', color: '#5a3b1e' },
  arcabuz:   { name: 'Arcabuz',            kind: 'gun', dmg: 40, cd: 0.3, mag: 1, reload: 1.05, speed: 820,
spread: 0.035, draw: 'gun', color: '#4a2f18' },
  mosquete:  { name: 'Mosquete e baioneta',kind: 'gun', dmg: 36, cd: 0.3, mag: 2, reload: 1.3, speed: 860,
spread: 0.03, draw: 'gun', color: '#3e2a16' },
  mauser:    { name: 'Fuzil Mauser',       kind: 'gun', dmg: 24, cd: 0.32, mag: 5, reload: 1.6, speed: 940,
spread: 0.02, draw: 'rifle', color: '#2e2e2e' },
  fuzil:     { name: 'Fuzil de ferrolho',  kind: 'gun', dmg: 24, cd: 0.3, mag: 5, reload: 1.5, speed: 940,
spread: 0.02, draw: 'rifle', color: '#333' },
  camera:    { name: 'Câmera fotográfica', kind: 'melee', dmg: 19, range: 120, arc: 1.1, cd: 0.55, draw:
'camera', color: '#222', fx: 'flash' },
  discurso:  { name: 'Discurso',           kind: 'voice', dmg: 11, cd: 0.22, speed: 520, spread: 0.08, draw:
'paper', color: '#fff', words: ['Diálogo!', 'Consenso!', 'Liberdade!', 'Votem!', 'Democracia!', 'Acordo!',
```

```
'Lei!'] },
  megafone:  { name: 'Megafone',          kind: 'voice', dmg: 12, cd: 0.18, speed: 560, spread: 0.1, draw:
'megaphone', color: '#e63', words: ['Futuro!', 'Juntos!', 'Ciência!', 'Europa!', 'Verdade!', 'Trabalho!',
'Respeito!'] }
};
```

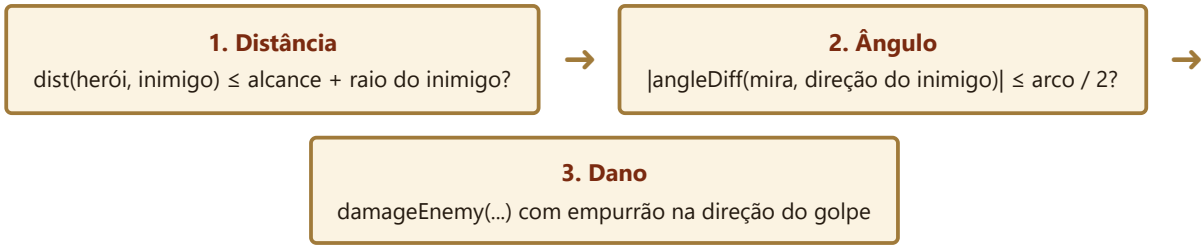
E uma única função executa qualquer uma delas:

js/game.js — linhas 501–536

```
function playerAttack() {
  const p = G.player, w = weapon();
  if (p.cd > 0 || p.reloadT > 0) return;
  if (w.kind === 'melee') {
    p.cd = w.cd; p.atkT = 0.2; p.atkMax = 0.2;
    Sfx.play(w.fx === 'flash' ? 'flash' : 'swing');
    if (w.fx === 'flash') G.fx.push({ type: 'flash', x: p.x, y: p.y - 6, a: p.aim, range: w.range, arc:
w.arc, life: 0.18, max: 0.18 });
    for (const e of G.enemies) {
      if (e.dead) continue;
      const d = dist(p.x, p.y, e.x, e.y);
      if (d > w.range + e.r) continue;
      const a = Math.atan2(e.y - p.y, e.x - p.x);
      if (Math.abs(angleDiff(p.aim, a)) <= w.arc / 2 + (d < 30 ? 1 : 0.15)) {
        damageEnemy(e, playerDmg(), Math.cos(a), Math.sin(a));
      }
    }
  } else if (w.kind === 'gun') {
    if (p.ammo <= 0) { startReload(); return; }
    p.cd = w.cd; p.ammo--; p.atkT = 0.12; p.atkMax = 0.12;
    const a = p.aim + randRange(-w.spread, w.spread);
    const mx = p.x + Math.cos(p.aim) * 30, my = p.y - 6 + Math.sin(p.aim) * 30;
    G.projs.push(new Proj(mx, my, a, w.speed, playerDmg(), 'p', '#ffe080', 3));
    for (let i = 0; i < 8; i++) particle(mx, my, i < 4 ? '#ffd040' : '#aaa', 80, 0.3);
    G.shake = Math.max(G.shake, 3);
    Sfx.play('shot');
    if (p.ammo <= 0) startReload();
  } else if (w.kind === 'voice') {
    p.cd = w.cd; p.atkT = 0.12; p.atkMax = 0.12;
    const a = p.aim + randRange(-w.spread, w.spread);
    const word = w.words[Math.floor(Math.random() * w.words.length)];
    const pr = new Proj(p.x + Math.cos(p.aim) * 20, p.y - 8 + Math.sin(p.aim) * 20, a, w.speed, playerDmg(),
'p', w.color, 9, word);
    pr.life = 0.9;
    G.projs.push(pr);
    Sfx.play('speech');
  }
}
```

Corpo a corpo (melee): o arco de ataque

Para lanças, espadas e até a câmera fotográfica, o ataque é instantâneo: o jogo procura inimigos que estejam **perto** (dentro do alcance) **e na frente** (dentro do arco). O arco é um ângulo: a lança tem 0,6 radiano (um golpe estreito, mas que vai longe), a espada longa tem 2,1 radianos (um corte largo).



Existe uma tolerância extra: inimigos **colados** no herói (menos de 30 pixels) são atingidos mesmo um pouco fora do arco. Sem isso, um lobo encostado de lado ficaria imune, o que parece injusto para o jogador.

Armas de fogo: munição e recarga

Espingarda, arcabuz, mosquete e fuzil criam um **projétil** (`new Proj`) que voa na direção da mira com um pequeno desvio aleatório (`spread`), simulando imprecisão. Cada arma tem um pente (`mag`): a espingarda dispara 1 vez e precisa recarregar (1,3 s); o fuzil Mauser dispara 5. A recarga começa sozinha quando a munição acaba, ou ao apertar R.

Palavras como arma

Nos capítulos da Transição e do século XXI, o herói é deputado e depois candidato. A "arma" dispara **palavras** ("Diálogo!", "Consenso!") como projéteis desenhados em balões. Mecanicamente, é uma arma de fogo sem munição e com cadência alta. Narrativamente, mostra que a forma de "lutar" mudou com a história. É um bom exemplo de como **a mesma mecânica pode contar coisas diferentes** só trocando o desenho e o texto.

8.4 Projéteis e colisão círculo com círculo

Todo projétil, a cada quadro: anda (`x += vx * dt`), perde vida, e verifica:

- Se o tile onde está é **alto** (árvore, prédio), ele morre e solta faíscas. A água não para tiros.
- Se é do jogador, testa cada inimigo; se é inimigo, testa o jogador.

O teste de acerto é o mais simples possível entre dois círculos: **acertou se a distância entre os centros for menor que a soma dos raios**.

```
if (dist(pr.x, pr.y, e.x, e.y - 6) < e.r + pr.r) { /* acertou */ }
```

8.5 Dano, invulnerabilidade e morte

js/game.js — linhas 426–440

```
function hurtPlayer(dmg, fromX, fromY) {
  const p = G.player;
  if (p.invT > 0 || p.dashT > 0 || G.ui) return;
  const d = Math.max(1, Math.round(dmg * (1 - playerDef()) * randRange(0.85, 1.15)));
  G.S.hp -= d;
  p.invT = 0.7; p.hurtT = 0.2;
  G.shake = Math.max(G.shake, 6);
  Sfx.play('hurt');
  floatText(p.x, p.y - 30, '-' + d, '#ff5050');
  if (fromX != null) {
    const a = Math.atan2(p.y - fromY, p.x - fromX);
```

```
    moveEnt(p, Math.cos(a) * 14, Math.sin(a) * 14);
  }
  if (G.S.hp <= 0) { G.S.hp = 0; playerDie(); }
}
```

Pontos de *game design* escondidos nessa função:

- **Defesa** reduz o dano em porcentagem (armadura comprada + escudo da arma), limitada a 60%.
- **Variação de ±15%** no dano deixa o combate menos previsível.
- **Invulnerabilidade de 0,7 s** depois de cada golpe: sem isso, quatro lobos juntos causariam dano todo quadro, e o herói morreria em meio segundo. O herói pisca enquanto está invulnerável, para o jogador entender o que está acontecendo.
- **Empurrão** (*knockback*) afasta o herói de quem bateu, dando uma chance de reagir.
- A **esquiva** (Espaço) também dá invulnerabilidade, o que recompensa reflexos.

Do lado dos inimigos, `damageEnemy` aplica o dano, mostra o número, solta partículas e empurra o inimigo (exceto chefes). Quando a vida chega a zero, `killEnemy` dá experiência e moedas, sorteia uma poção (3,5% de chance; chefes sempre deixam uma), avisa o sistema de missões (`onKill`) e, se era um chefe, avança a missão correspondente.

js/game.js — linhas 472–493

```
function killEnemy(e) {
  e.dead = true;
  G.S.stats.kills++;
  Sfx.play('die');
  for (let i = 0; i < (e.boss ? 60 : 14); i++) particle(e.x, e.y, e.d.color || (e.d.look && e.d.look.body)
|| '#888', e.boss ? 260 : 140, 0.7, e.boss ? 5 : 3);
  gainXp(Math.round(e.d.xp * G.mult));
  const gold = Math.round(e.d.gold * G.mult * randRange(0.7, 1.3));
  const coins = e.boss ? 10 : Math.min(3, 1 + Math.floor(gold / 6));
  for (let i = 0; i < coins; i++) {
    G.objs.push({ kind: 'coin', value: Math.max(1, Math.round(gold / coins)), x: e.x + randRange(-14, 14),
y: e.y + randRange(-14, 14), r: 6, t: 0 });
  }
  if (Math.random() < (e.boss ? 1 : 0.035)) G.objs.push({ kind: 'potion', x: e.x, y: e.y + 8, r: 8, name:
T('potion') });
  if (e.boss) {
    G.boss = null;
    G.shake = 20;
    G.enemies.forEach(m => { if (m.summoned) { m.dead = true; } });
    const quest = G.ch.quests.find(q => q.id === e.questId);
    UI.toast(T('defeated', e.d.name), 'done');
    if (quest) advance(quest);
  }
  onKill(e.type);
}
```

Quando o herói cai, `playerDie` mostra a tela de queda, tira 10% do dinheiro e devolve o herói ao início do capítulo, com vida cheia. Se havia um chefe, ele é removido e reaparecerá inteiro, para a luta recomeçar de forma justa.

8.6 Interagindo com o mundo

`G.objs` guarda tudo com que se pode interagir: NPCs, itens de missão, crônicas, baús, moedas e poções. Cada objeto tem um campo `kind` (tipo), e `interact` decide o que fazer:

js/game.js — linhas 825–856

```
function interact(o) {
  if (!o) return;
  if (dist(G.player.x, G.player.y, o.x, o.y) > INTERACT_RANGE) { UI.toast(T('closer'), 'prog'); return; }
  Sfx.play('click');
  switch (o.kind) {
    case 'npc': talkTo(o); break;
    case 'item': pickItem(o); break;
    case 'codex': {
      o.gone = true;
      chState(G.ch.id).codex[o.id] = true;
      gainXp(Math.round(25 * G.mult));
      Sfx.play('quest');
      UI.codexFound(o.def);
      save();
      break;
    }
    case 'chest': {
      if (o.opened) return;
      o.opened = true; o.name = T('chestEmpty');
      chState(G.ch.id).chests[o.idx] = true;
      const gold = Math.round(randRange(25, 45) * G.mult);
      G.S.gold += gold;
      let msg = T('chestLoot', gold, G.ch.currency);
      if (Math.random() < 0.6) { G.S.potions++; msg += T('chestPot'); }
      Sfx.play('pickup');
      UI.toast(msg, 'done');
      for (let i = 0; i < 16; i++) particle(o.x, o.y, '#ffd84a', 120, 0.6);
      save();
      break;
    }
  }
}
```

Esse padrão (uma lista de objetos variados com um campo de tipo e um `switch` que trata cada tipo) é muito comum em jogos simples. Em projetos maiores, cada tipo vira uma classe com seu próprio método `interact()`.

CAPÍTULO 9

Inteligência artificial dos inimigos

"Inteligência artificial" em jogos raramente é algo sofisticado. Na maioria das vezes, são **regras simples** que, combinadas, dão a impressão de comportamento inteligente. O Hispânia tem quatro tipos de comportamento para inimigos comuns e um sistema de "padrões de ataque" para os chefes.

9.1 A ficha de um inimigo

js/data/base.js — linhas 44–44

```
javali:    { name: 'Javali',            hp: 34, spd: 90,  dmg: 10, ai: 'charger',r: 14, xp: 10, gold: 2,
shape: 'beast', color: '#5a4030' },
```

- `hp`, `spd`, `dmg`: vida, velocidade e dano **base** (no capítulo 1).
- `ai`: o comportamento (`melee`, `ranged`, `charger` ou `swarm`).
- `r`: raio (tamanho para colisão e desenho).
- `xp`, `gold`: recompensas.
- `shape`: como desenhar (`beast` para animais, `human`, `blob` para criaturas simbólicas como as Sombras e o Vírus, `tower` para o Bastião de Málaga).

A classe `Enemy` transforma essa ficha num inimigo "vivo", aplicando o multiplicador de dificuldade do capítulo:

js/entities.js — linhas 230–245

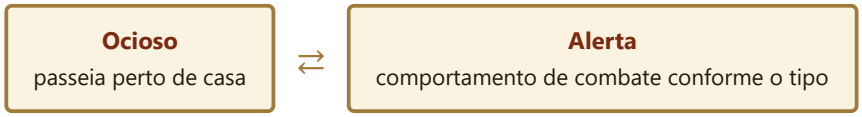
```
class Enemy {
  constructor(type, x, y, mult, campId) {
    const d = ENEMIES[type];
    this.type = type; this.d = d;
    this.x = x; this.y = y; this.homeX = x; this.homeY = y;
    this.r = d.r; this.boss = !!d.boss;
    this.maxHp = Math.round(d.hp * mult); this.hp = this.maxHp;
    this.dmg = d.dmg * mult; this.spd = d.spd;
    this.ai = d.ai || 'melee';
    this.campId = campId;
    this.t = Math.random() * 10; this.cd = 1 + Math.random(); this.state = 'idle';
    this.aim = 0; this.hurtT = 0; this.windT = 0; this.dashT = 0; this.dashX = 0; this.dashY = 0;
    this.wanderT = 0; this.wx = x; this.wy = y; this.aggro = false; this.moving = false;
    this.pattern = 0; this.slamT = 0; this.slamR = 0; this.dead = false;
  }
}
```

9.2 Estados: ocioso ou em alerta

Todo inimigo comum começa **ocioso**: passeia devagar em volta do seu ponto de origem, escolhendo um destino aleatório a cada 1,5 a 4 segundos. Ele entra em **alerta** (`aggro`) quando o herói chega a menos de 290 pixels, ou quando leva dano. E desiste se o herói se afastar muito (700 pixels) ou se ele mesmo se afastar demais de casa

(900 pixels). Isso impede que o jogador "arraste" um exército inteiro pelo mapa.

Essa ideia de ter **estados** e **regras de transição** entre eles se chama **máquina de estados** (*state machine*) e é a base da IA de quase todos os jogos.



9.3 Os quatro comportamentos

js/game.js — linhas 674–728

```
function updateEnemy(e, dt) {
  const p = G.player;
  const dx = p.x - e.x, dy = p.y - e.y, d = Math.hypot(dx, dy) || 1;
  e.t += dt; e.hurtT -= dt; e.cd -= dt;
  e.aim = Math.atan2(dy, dx);
  if (e.boss) return updateBoss(e, dt, d, dx, dy);
  if (!e.aggro && d < 290) e.aggro = true;
  const hd = dist(e.x, e.y, e.homeX, e.homeY);
  if (e.aggro && (hd > 900 || d > 700)) e.aggro = false;
  e.moving = false;
  const step = e.spd * dt;
  if (!e.aggro) {
    e.wanderT -= dt;
    if (e.wanderT <= 0) { e.wanderT = randRange(1.5, 4); e.wx = e.homeX + randRange(-110, 110); e.wy =
e.homeY + randRange(-110, 110); }
    const wd = dist(e.x, e.y, e.wx, e.wy);
    if (wd > 6) { moveEnt(e, (e.wx - e.x) / wd * step * 0.4, (e.wy - e.y) / wd * step * 0.4); e.moving =
true; e.aim = Math.atan2(e.wy - e.y, e.wx - e.x); }
    e.windT = 0; e.dashT = 0;
    return;
  }
  const ux = dx / d, uy = dy / d;
  const reach = e.r + p.r;
  switch (e.ai) {
    case 'melee':
      if (e.windT > 0) {
        e.windT -= dt;
        if (e.windT <= 0) { if (d < reach + 24) hurtPlayer(e.dmg, e.x, e.y); e.cd = 1.1; }
      } else if (d > reach + 6) { moveEnt(e, ux * step, uy * step); e.moving = true; }
      else if (e.cd <= 0) e.windT = 0.32;
      break;
    case 'swarm':
      moveEnt(e, ux * step, uy * step); e.moving = true;
      if (d < reach + 4 && e.cd <= 0) { hurtPlayer(e.dmg, e.x, e.y); e.cd = 0.8; }
      break;
    case 'ranged':
      if (d < 170) { moveEnt(e, -ux * step, -uy * step); e.moving = true; }
      else if (d > 290) { moveEnt(e, ux * step, uy * step); e.moving = true; }
      else { moveEnt(e, -uy * step * 0.4 * Math.sin(e.t), ux * step * 0.4 * Math.sin(e.t)); }
      if (e.cd <= 0 && d < 430) {
        e.cd = randRange(1.5, 2.2);
        G.projs.push(new Proj(e.x, e.y - 6, e.aim + randRange(-0.06, 0.06), 270, e.dmg, 'e', e.d.proj ||
'#ccc', 4));
      }
      break;
```

```
    case 'charger':
      if (e.dashT > 0) {
        e.dashT -= dt; e.moving = true;
        moveEnt(e, e.dashX * e.spd * 4.2 * dt, e.dashY * e.spd * 4.2 * dt);
        if (!e.hitDash && d < reach + 4) { e.hitDash = true; hurtPlayer(e.dmg * 1.3, e.x, e.y); }
      } else if (e.windT > 0) {
        e.windT -= dt;
        if (e.windT <= 0) { e.dashT = 0.5; e.dashX = ux; e.dashY = uy; e.hitDash = false; }
      } else if (d < 270 && e.cd <= 0) { e.windT = 0.55; e.cd = 2.4; }
      else { moveEnt(e, ux * step * 0.7, uy * step * 0.7); e.moving = true; if (d < reach + 4 && e.cd < 1.5)
{ hurtPlayer(e.dmg * 0.6, e.x, e.y); e.cd = 1.8; } }
      break;
  }
}
```

melee (corpo a corpo): lobos, soldados, bandidos

Persegue o herói. Ao chegar perto, **prepara o golpe** por 0,32 s (pisca e mostra um "!" vermelho) e só então ataca, e só acerta se o herói ainda estiver perto. Esse aviso (*telegraph*) é fundamental: dá ao jogador a chance de **ver e esquivar**. Ataques sem aviso parecem injustos.

swarm (enxame): bolhas imobiliárias, vírus, sombras da fome

Persegue sem parar e causa dano por contato, com intervalo de 0,8 s. São fracos individualmente, mas vêm em grupo e são rápidos.

ranged (à distância): arqueiros, fundeiros, fuzileiros, bots

Mantém distância: se o herói chega a menos de 170 pixels, recua; se passa de 290, se aproxima; no meio, anda de lado ("strafe"). A cada 1,5 a 2,2 segundos, dispara um projétil na direção do herói. Isso obriga o jogador a ir até eles, em vez de esperar.

charger (investida): javalis, auroques, cavaleiros, elefantes

Se aproxima devagar. Quando está a menos de 270 pixels, **para e treme** por 0,55 s (o aviso) e então **dispara em linha reta** a 4,2 vezes a velocidade normal, na direção onde o herói **estava**. Se o herói esquivar para o lado, o animal passa direto. É um comportamento clássico que transforma a esquiva numa habilidade importante.

9.4 Os chefes: padrões de ataque

Cada capítulo tem um chefe com uma lista de **padrões** que se repetem em ordem, como uma coreografia:

js/data/base.js — linhas 46–46

```
urso:      { name: 'Urso das Cavernas Ancião', hp: 900, spd: 85, dmg: 18, r: 34, xp: 250, gold: 60, shape:
'beast', color: '#4a3020', boss: true, patterns: ['charge', 'slam', 'summon', 'charge'], summon: 'lobo' },
```

js/game.js — linhas 730–809

```
function updateBoss(e, dt, d, dx, dy) {
  const p = G.player, d2 = e.d;
  const phase2 = e.hp < e.maxHp / 2;
  if (phase2 && !e.p2) { e.p2 = true; UI.toast(T('furious', d2.name), 'boss'); G.shake = 10; }
  const ux = dx / d, uy = dy / d;
  e.touchCd = (e.touchCd || 0) - dt;
```

```
  e.moving = false;
  if (e.dashT > 0) {
    e.dashT -= dt; e.moving = true;
    moveEnt(e, e.dashX * e.spd * 4 * dt, e.dashY * e.spd * 4 * dt);
    if (!e.hitDash && d < e.r + p.r + 4) { e.hitDash = true; hurtPlayer(e.dmg * 1.4, e.x, e.y); }
    return;
  }
  if (e.windT > 0) {
    e.windT -= dt;
    if (e.windT <= 0) { e.dashT = 0.65; e.dashX = ux; e.dashY = uy; e.hitDash = false; }
    return;
  }
  if (e.slamT > 0) {
    e.slamT -= dt;
    if (e.slamT <= 0) {
      G.shake = 10;
      for (let i = 0; i < 30; i++) particle(e.x + randRange(-e.slamR, e.slamR) * 0.7, e.y + randRange(-
e.slamR, e.slamR) * 0.7, '#a98', 120, 0.5, 4);
      if (d < e.slamR) hurtPlayer(e.dmg * 1.5, e.x, e.y);
    }
    return;
  }
  if (e.sprayN > 0) {
    e.sprayT -= dt;
    if (e.sprayT <= 0) {
      e.sprayN--; e.sprayT = 0.24;
      const n = phase2 ? 7 : 5;
      for (let i = 0; i < n; i++) {
        const a = e.aim + (i - (n - 1) / 2) * 0.16;
        G.projs.push(new Proj(e.x + Math.cos(a) * e.r, e.y - 6 + Math.sin(a) * e.r, a, 280, e.dmg * 0.8,
'e', d2.proj || '#ccc', 5));
      }
      Sfx.play('shot');
    }
    return;
  }
  if (!d2.stationary && d > e.r + p.r + 8) {
    const sp = e.spd * (phase2 ? 1.25 : 1) * dt;
    moveEnt(e, ux * sp, uy * sp); e.moving = true;
  }
  if (d < e.r + p.r + 6 && e.touchCd <= 0) { hurtPlayer(e.dmg, e.x, e.y); e.touchCd = 1; }
  if (e.cd <= 0) {
    const pat = d2.patterns[e.pattern++ % d2.patterns.length];
    e.cd = phase2 ? 1.5 : 2.2;
    switch (pat) {
      case 'charge':
        if (d2.stationary) break;
        e.windT = 0.6; break;
      case 'slam':
        e.slamT = 0.9; e.slamR = e.r + 95; break;
      case 'burst': {
        const n = phase2 ? 18 : 12;
        for (let i = 0; i < n; i++) {
          const a = (i / n) * Math.PI * 2 + e.t;
          G.projs.push(new Proj(e.x, e.y - 6, a, 230, e.dmg * 0.8, 'e', d2.proj || '#ccc', 6));
        }
        Sfx.play('shot');
        break;
      }
      case 'spray':
        e.sprayN = phase2 ? 4 : 3; e.sprayT = 0; break;
```

```
      case 'summon': {
        const minions = G.enemies.filter(m => m.summoned && !m.dead).length;
        const n = Math.min(phase2 ? 3 : 2, 6 - minions);
        for (let i = 0; i < n; i++) {
          const s = G.world.randomSpot(e.x, e.y, 4, 2);
          const m = new Enemy(d2.summon, s.x, s.y, G.mult, null);
          m.summoned = true; m.aggro = true; m.homeX = e.x; m.homeY = e.y;
          G.enemies.push(m);
          for (let k = 0; k < 10; k++) particle(s.x, s.y, '#fff', 90, 0.4);
        }
        break;
      }
    }
  }
}
```

Padrão	O que faz	Como o jogador escapa
`charge`	Investida como a dos javalis, mais forte	Esquivar para o lado
`slam`	Marca um círculo vermelho no chão; 0,9 s depois, causa dano em toda a área	Sair do círculo
`burst`	Dispara 12 projéteis em todas as direções (18 na fase 2)	Ficar nos espaços entre eles
`spray`	Rajadas de 5 projéteis em leque na direção do herói (7 na fase 2)	Movimentar-se de lado
`summon`	Chama 2 ajudantes (3 na fase 2), no máximo 6 ao mesmo tempo	Derrotá-los ou ignorá-los

Quando a vida do chefe cai abaixo da metade, ele entra na **fase 2** ("está furioso!"): fica 25% mais rápido, ataca com mais frequência e seus padrões ficam mais intensos. Essa mudança no meio da luta é um recurso clássico para manter a tensão.

O Bastião de Málaga é um caso especial: uma torre **imóvel** (`stationary: true`) que não investe, só atira e invoca soldados. A mesma função `updateBoss` trata isso pulando o padrão `charge`.

9.5 Acampamentos e reaparecimento (respawn)

Os inimigos não ficam espalhados ao acaso: eles vivem em **acampamentos** (*camps*), cada um ligado a um lugar do mapa. Os acampamentos são criados automaticamente a partir das missões: se uma etapa pede "derrote 8 legionários na Muralha de Cipião", o jogo cria um acampamento de legionários lá. Assim, **é impossível uma missão de caça ficar sem inimigos**.

js/game.js — linhas 181–190

```
function spawnCampEnemy(camp, awayFromPlayer) {
  const p = G.world.places[camp.at];
  if (!p) return;
  let s = G.world.randomSpot(p.px, p.py, p.r + 3, 1);
  if (awayFromPlayer) {
    // Reaparece no acampamento, mas longe o bastante do jogador para não surgir em cima dele
    for (let i = 0; i < 25 && dist(s.x, s.y, G.player.x, G.player.y) < 320; i++) s =
G.world.randomSpot(p.px, p.py, p.r + 8, 1);
  }
  G.enemies.push(new Enemy(camp.type, s.x, s.y, G.mult, camp.id));
}
```

E, dentro de `update`, a reposição:

```
for (const camp of G.camps) {
  camp.respawnT -= dt;
  if (camp.respawnT > 0) continue;
  const alive = G.enemies.reduce((n, e) => n + (e.campId === camp.id ? 1 : 0), 0);
  if (alive < camp.n) { spawnCampEnemy(camp, true); camp.respawnT = 5; }
  else camp.respawnT = 2;
}
```

A cada 2 segundos, cada acampamento conta seus inimigos vivos (`reduce` soma 1 para cada inimigo daquele acampamento). Se faltar alguém, cria um novo e espera 5 segundos antes do próximo.

> 🐞 **História de um bug**
>
> **"O jogo não está dando respawn dos bichos."** Este foi o primeiro problema que você reportou. Na versão original, a regra era: só repor inimigos se o herói estivesse a **mais de 650 pixels** do acampamento, para ninguém "nascer na cara" do jogador. Parecia sensato, mas na prática quem ficava caçando no acampamento **nunca** via inimigos novos, e as missões de caça travavam. A correção: repor **sempre**, mas escolher um ponto do acampamento a **pelo menos 320 pixels** do herói (`awayFromPlayer`), tentando até 25 sorteios. O teste confirmou: com o herói parado no Bosque dos Lobos, os lobos voltaram um a um, nascendo a cerca de 400 pixels e vindo atacar. **Lição:** regras que parecem boas no papel precisam ser testadas do jeito que o jogador realmente joga.

9.6 Progressão e equilíbrio (balanceamento)

Como o jogo tem 14 capítulos, os inimigos precisam ficar mais fortes, e o herói também. Tudo isso é controlado por algumas fórmulas curtas:

js/game.js — linhas 55–55

```
const xpNeed = (l) => Math.round(50 * Math.pow(l, 1.3));
```

js/game.js — linhas 56–56

```
function maxHp() { return Math.round((90 + G.S.level * 10) * (1 + 0.12 * G.chIdx)); }
```

js/game.js — linhas 58–58

```
function playerDmg() { return weapon().dmg * G.mult * (1 + 0.05 * (G.S.level - 1)) * (1 + 0.1 * G.S.sharp);
}
```

js/game.js — linhas 59–59

```
function playerDef() { return Math.min(0.6, G.S.armor * 0.06 + (weapon().def || 0)); }
```

E, ao carregar cada capítulo, o **multiplicador de dificuldade**: `G.mult = 1 + 0.32 * idx`. O capítulo 1 tem multiplicador 1; o 14, cerca de 5,2. Vida, dano, experiência e moedas dos inimigos são multiplicados por ele. O dano das armas também, de modo que cada capítulo tem uma dificuldade parecida em relação ao herói daquela

época. O que muda é o **nível** do herói (+5% de dano e +10 de vida por nível) e as **melhorias da loja** (+10% de dano e +6% de defesa por compra).

Nível	XP para o próximo	Vida máxima no cap. 1	Vida máxima no cap. 14
1	50	100	256
5	405	140	358
10	998	190	486
20	2456	290	742
30	4161	390	998

A curva `50 × nível^1,3` cresce **mais rápido que uma reta**: cada nível pede um pouco mais que o anterior. Isso faz o jogador subir de nível rapidamente no começo (recompensa constante) e mais devagar depois (a progressão não fica absurda).

> 💡 **Dica**
>
> Balancear um jogo é **experimentar**. Mude um número, jogue, sinta, ajuste. Ter todos os números importantes em fórmulas curtas e nos arquivos de dados (em vez de espalhados pelo código) é o que torna isso viável.

CAPÍTULO 10

Missões, capítulos e dados

Este capítulo mostra o "motor de histórias" do Hispânia: como um arquivo de dados vira 14 capítulos jogáveis, com diálogos, missões, quizzes, chefes e evolução do herói.

10.1 Os dados base: armas, classes e inimigos

O arquivo `js/data/base.js` tem três "tabelas":

- `WEAPONS`: as 14 armas, com tipo (`melee`, `gun`, `voice`), dano, alcance, arco, recarga, munição, cor e como desenhar.
- `CLASSES`: as 14 classes do herói. Cada uma aponta para uma arma e tem uma aparência (`look`).
- `ENEMIES`: cerca de 55 inimigos e chefes.

js/data/base.js — linhas 22–37

```
const CLASSES = {
  cacador:      { name: 'Caçador',                    weapon: 'lanca',      look: { body: '#7a5230',
legs: '#5a3a20', skin: '#d8a27a', hat: 'hood', hatColor: '#8d6a44' } },
  celtibero:    { name: 'Guerreiro Celtibero',        weapon: 'falcata',    look: { body: '#9c2f2f',
legs: '#5a4030', skin: '#d8a27a', hat: 'band', hatColor: '#e0c060' } },
  legionario:   { name: 'Legionário de Hispânia',     weapon: 'gladio',     look: { body: '#b02222',
legs: '#6a4a2a', skin: '#d8a27a', hat: 'galea', hatColor: '#c9a227' } },
  godo:         { name: 'Guerreiro Visigodo',         weapon: 'spatha',     look: { body: '#3a5f8a',
legs: '#4a3a2a', skin: '#e0b090', hat: 'nasal', hatColor: '#9aa3ab' } },
  asturiano:    { name: 'Infante Asturiano',          weapon: 'espada',     look: { body: '#4f6f34',
legs: '#4a3a2a', skin: '#e0b090', hat: 'nasal', hatColor: '#8a929a' } },
  cavaleiro:    { name: 'Cavaleiro de Castela',       weapon: 'montante',   look: { body: '#e8e2d0',
legs: '#707880', skin: '#e0b090', hat: 'great', hatColor: '#a8b0b8', cross: '#b02020' } },
  espingardeiro:{ name: 'Espingardeiro do Gran Capitán', weapon: 'espingarda', look: { body: '#b8860b',
legs: '#7a2020', skin: '#e0b090', hat: 'cap', hatColor: '#7a2020' } },
  tercio:       { name: 'Soldado do Tercio',          weapon: 'arcabuz',    look: { body: '#c23b22',
legs: '#e0c060', skin: '#e0b090', hat: 'morion', hatColor: '#b8c0c8' } },
  guerrilheiro: { name: 'Guerrilheiro de 1808',       weapon: 'mosquete',   look: { body: '#6b4b2b',
legs: '#3a3a3a', skin: '#e0b090', hat: 'wide', hatColor: '#2a2a2a', sash: '#b02020' } },
  soldado:      { name: 'Soldado de Infantaria',      weapon: 'mauser',     look: { body: '#2b3d6b',
legs: '#b02020', skin: '#e0b090', hat: 'shako', hatColor: '#2b2b3b' } },
  guardiao:     { name: 'Guardião do Tesouro Artístico', weapon: 'fuzil',   look: { body: '#5a5a3a',
legs: '#3a3a2a', skin: '#e0b090', hat: 'beret', hatColor: '#222' } },
  jornalista:   { name: 'Jornalista',                 weapon: 'camera',     look: { body: '#7a6a5a',
legs: '#3a3a3a', skin: '#e0b090', hat: 'fedora', hatColor: '#4a3a2a' } },
  deputado:     { name: 'Deputado Constituinte',      weapon: 'discurso',   look: { body: '#2a2a2a',
legs: '#1a1a1a', skin: '#e0b090', hat: 'none', tie: '#a02020' } },
  candidato:    { name: 'Candidato a Presidente',     weapon: 'megafone',   look: { body: '#1e3a6e',
legs: '#1a1a2a', skin: '#e0b090', hat: 'none', tie: '#e0b020' } }
};
```

A classe de cada capítulo é definida no próprio capítulo (`cls: 'cacador'`). Por isso o herói **muda de arma automaticamente** ao trocar de capítulo, e ao voltar a um capítulo antigo pelo menu ele volta a usar a lança. Nível, ouro e melhorias, porém, acompanham a linhagem inteira.

10.2 A estrutura de um capítulo

Cada capítulo é um objeto grande. Este é o esqueleto, com o significado de cada campo:

```
CHAPTERS.push({
  id: 'c1', num: 1,                    // identificador único e ordem
  title: 'As Origens', years: '...',   // nome e época
  cls: 'cacador',                      // classe/arma do herói neste capítulo
  seed: 1101,                          // semente do mapa
  currency: 'conchas',                 // nome da moeda (só visual)
  biome: { ... },                      // cores e obstáculos do mapa
  intro: ['página 1', 'página 2'],     // narração de abertura
  outro: ['...'],                      // narração de encerramento
  places: [ { id, name, x, y, r, type, icon } ],   // lugares do mapa
  npcs:   [ { id, name, at, dx, dy, look, lines, shop } ],  // personagens
  camps:  [ { at, type, n } ],         // acampamentos extras (opcional)
  codex:  [ { id, at, title, text } ], // crônicas históricas colecionáveis
  quests: [ main, s1, s2, ... ]        // missão principal + secundárias
});
```

Lugares

js/data/chapters1.js — linhas 45–53

```
places: [
  { id: 'aldeia', name: 'Aldeia do Clã', x: 16, y: 40, r: 7, type: 'village', icon: 'fire' },
  { id: 'bosque', name: 'Bosque dos Lobos', x: 38, y: 18, r: 6, type: 'camp' },
  { id: 'atapuerca', name: 'Serra de Atapuerca', x: 60, y: 30, r: 6, type: 'ruin', icon: 'cave' },
  { id: 'vale', name: 'Vale dos Auroques', x: 50, y: 62, r: 7, type: 'camp' },
  { id: 'altamira', name: 'Gruta de Altamira', x: 80, y: 48, r: 6, type: 'ruin', icon: 'cave' },
  { id: 'dolmen', name: 'Dólmen dos Antigos', x: 86, y: 16, r: 5, type: 'ruin', icon: 'dolmen' },
  { id: 'toca', name: 'Toca do Urso', x: 96, y: 66, r: 8, type: 'arena', icon: 'cave' }
],
```

Cada lugar tem posição **em tiles** (`x`, `y`), raio da área limpa (`r`), um `type` e um `icon` opcional (o marco desenhado no centro). O tipo muda o mapa e o minimapa:

type	Efeito no mapa	Cor no minimapa
`village` / `city`	Praça com piso e casas em volta	Branco
`camp`	Clareira onde inimigos ficam	Laranja
`ruin`	Clareira com um marco	Branco
`arena`	Clareira grande onde aparece o chefe	Vermelho

O **primeiro lugar** da lista é sempre o ponto de partida e de renascimento do herói.

NPCs

Um NPC (personagem não jogável) fica num lugar (`at`) com um deslocamento em tiles (`dx`, `dy`). Tem falas soltas (`lines`), que se alternam a cada conversa, e pode ser um **comerciante** (`shop: true`).

js/data/chapters1.js — linhas 55–55

```
{ id: 'ama', name: 'Anciã Ama', at: 'aldeia', dx: -2, dy: -1, look: LOOK.elder, lines: ['O fogo é a nossa vida. Nunca deixe que ele se apague.', 'Os mais velhos dizem que nossos ancestrais viviam nas serras desde antes das montanhas terem nome.'] },
```

Crônicas (codex)

As 92 crônicas são os textos históricos espalhados pelo mapa. Cada uma diz perto de qual lugar está (`at`); a posição exata é sorteada com uma semente própria, então é sempre a mesma.

10.3 A missão: uma lista de etapas

Toda missão é uma lista de **etapas** (`steps`). Existem 6 tipos de etapa:

`t` (tipo)	Significado	Campos	Termina quando
`talk`	Falar com alguém	`npc`, `say`	O jogador conversa com o NPC
`kill`	Derrotar inimigos	`e` (tipo), `n`, `at`	`n` inimigos do tipo `e` morrem
`collect`	Coletar itens	`item`, `icon`, `n`, `at`	`n` itens coletados
`goto`	Ir a um lugar	`at`, `say`	O herói entra na área do lugar
`boss`	Derrotar um chefe	`e`, `at`, `intro`	O chefe morre
`quiz`	Responder perguntas	`npc`, `qs`	Todas as respostas certas

Qualquer etapa pode ter um `desc` (o texto do objetivo no HUD) e um `done` (falas mostradas ao concluir). Veja a missão principal do capítulo 1 inteira, só com dados:

js/data/chapters1.js — linhas 72–96

```
{ id: 'main', title: 'O Inverno do Clã', reward: { xp: 120, gold: 30, potions: 2 }, steps: [
  { t: 'talk', npc: 'ama', desc: 'Fale com a Anciã Ama na aldeia', say: [
    ['ama', '{nome}, o inverno chegará cedo este ano. As crianças precisam de peles, e os lobos rondam as nossas trilhas.'],
    ['ama', 'Vá ao Bosque dos Lobos, ao norte. Siga a seta dourada e use sua lança. Volte inteiro.'],
    ['N', 'Dica: clique para atacar. Segure o botão para golpear sem parar. Espaço faz uma esquiva rápida.']] },
  { t: 'kill', e: 'lobo', n: 5, at: 'bosque', desc: 'Cace 5 lobos no Bosque dos Lobos', done: [['H', 'As peles estão garantidas. Mas a ponta da minha lança está lascada...']] },
  { t: 'goto', at: 'atapuerca', desc: 'Vá à Serra de Atapuerca procurar sílex', say: [['N', 'Na serra, há cavernas cheias de ossos antigos, mais velhos do que qualquer lenda do clã.']] },
  { t: 'talk', npc: 'urtzi', desc: 'Fale com o Xamã Urtzi', say: [
    ['urtzi', 'Você pisa sobre o lar dos primeiros. Há mais de um milhão de anos, gente como nós já caçava aqui.'],
    ['urtzi', 'O melhor sílex está espalhado pela encosta. Traga-me cinco pedras e farei pontas dignas de um caçador.']] },
  { t: 'collect', item: 'Sílex', icon: '🪨', n: 5, at: 'atapuerca', desc: 'Recolha 5 pedras de sílex na serra' },
  { t: 'talk', npc: 'urtzi', desc: 'Leve o sílex ao Xamã Urtzi', say: [
    ['urtzi', 'Boas pedras. Tome também isto: um amuleto de osso entalhado com a figura de um bisonte.'],
    ['urtzi', 'Ele guardará a memória de quem o usa. Passe-o aos seus filhos, e eles se lembrarão de você.'],
    ['N', 'Você recebeu o Amuleto de Atapuerca. Ele acompanhará a sua linhagem por toda a história.']] },
  { t: 'kill', e: 'auroque', n: 3, at: 'vale', desc: 'Cace 3 auroques no Vale dos Auroques', done: [['H',
```

```
'Carne para todo o inverno! Ekain, o pintor, vai querer saber disso.']] },
  { t: 'talk', npc: 'ekain', desc: 'Conte a caçada a Ekain na Gruta de Altamira', say: [
    ['ekain', 'Três auroques! Vou pintá-los no teto, ao lado dos bisontes, para que nossos netos saibam como
caçávamos.'],
    ['ekain', 'Mas cuidado: o Urso das Cavernas Ancião acordou. Ele destruiu o acampamento dos caçadores
perto da toca dele, a sudeste.']] },
  { t: 'boss', e: 'urso', at: 'toca', desc: 'Derrote o Urso das Cavernas na Toca do Urso', intro: [['N', 'Um
rugido faz tremer as pedras. O Urso das Cavernas Ancião defende o seu território!']],
    done: [['H', 'Acabou... O clã estará seguro neste inverno.']] },
  { t: 'talk', npc: 'ama', desc: 'Volte à Anciã Ama', say: [
    ['ama', 'Você trouxe peles, carne e paz ao clã. Os espíritos sorriram para você.'],
    ['ama', 'Guarde bem o amuleto. Um dia, os seus descendentes verão coisas que nem conseguimos
imaginar.']] }
] },
```

Note como **a história é contada pelos dados**. As falas têm o formato `[quem, texto]`, onde `quem` é o id de um NPC, `'N'` (narrador) ou `'H'` (o herói). O marcador `{nome}` é trocado pelo nome da linhagem escolhido pelo jogador.

Missões secundárias

As secundárias têm a mesma forma, mais dois detalhes:

- A **primeira etapa é sempre `talk` com quem dá a missão**. Enquanto o jogador não fala com essa pessoa, a missão fica "disponível", mas não aparece no rastreador.
- O campo `req` diz em que etapa da história principal ela é liberada. Por exemplo, `req: 8` significa que a missão só aparece depois que a missão principal chegou à etapa 8. Isso evita spoilers e garante a ordem cronológica.

E um `reward` (recompensa) com XP, ouro e poções, multiplicados pela dificuldade do capítulo.

10.4 O motor de missões

Agora o código que dá vida a esses dados. Todo o progresso fica em `G.S.ch[idCapitulo].q[idMissao]`, um objeto pequeno com três campos: `step` (etapa atual), `prog` (progresso na etapa, como "3 de 5 lobos") e `done` (concluída).

js/game.js — linhas 36–40

```
function chState(id) {
  const S = G.S;
  if (!S.ch[id]) S.ch[id] = { q: {}, codex: {}, chests: {}, done: false, intro: false };
  return S.ch[id];
}
```

js/game.js — linhas 41–45

```
function qState(qid) {
  const cs = chState(G.ch.id);
  if (!cs.q[qid]) cs.q[qid] = { step: 0, prog: 0, done: false };
  return cs.q[qid];
}
```

Repare na técnica: se o registro ainda não existe, ele é **criado na hora** com valores iniciais. Assim, o resto do código nunca precisa verificar "já existe?".

js/game.js linhas 47–52

```
function questActive(quest) {
  const q = qState(quest.id);
  if (q.done) return false;
  if (quest.id !== 'main' && quest.req && mainStep() < quest.req) return false;
  return true;
}
```

Avançar, começar e concluir

As três funções centrais formam um ciclo:

js/game.js linhas 201–213

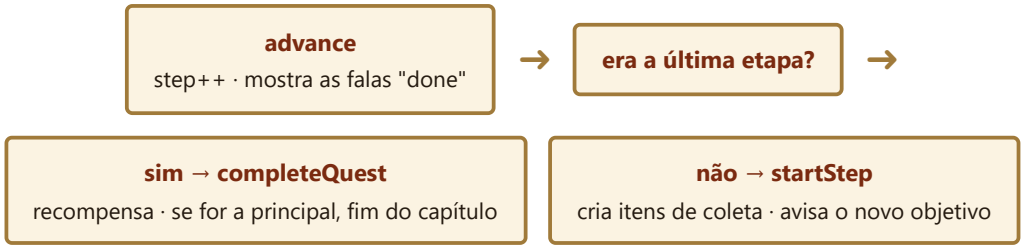
```
function advance(quest) {
  const q = qState(quest.id);
  const step = quest.steps[q.step];
  q.step++; q.prog = 0;
  Sfx.play('pickup');
  const after = () => {
    if (q.step >= quest.steps.length) completeQuest(quest);
    else startStep(quest);
    save();
    UI.hud(true);
  };
  if (step && step.done) UI.dialog(step.done, after); else after();
}
```

js/game.js linhas 215–220

```
function startStep(quest) {
  const s = curStep(quest);
  if (!s) return;
  if (s.t === 'collect') spawnItems(quest, s, s.n);
  UI.toast((quest.id === 'main' ? '★ ' : '◆ ') + s.desc, quest.id === 'main' ? 'main' : 'side');
}
```

js/game.js linhas 222–235

```
function completeQuest(quest) {
  const q = qState(quest.id);
  q.done = true;
  G.S.stats.quests++;
  const r = quest.reward || {};
  const xp = Math.round((r.xp || 60) * G.mult);
  const gold = Math.round((r.gold || 20) * G.mult);
  G.S.gold += gold;
  if (r.potions) G.S.potions += r.potions;
  gainXp(xp);
  Sfx.play('quest');
  UI.toast(T('questDone', quest.title, xp, gold, G.ch.currency) + (r.potions ? T('questDonePot', r.potions)
: ''), 'done');
  if (quest.id === 'main') chapterComplete();
}
```



Quem chama advance?

Cada tipo de etapa é detectado num lugar diferente do código:

- `talk` e `quiz`: em `talkTo`, quando o jogador clica num NPC.
- `kill`: em `onKill`, chamado sempre que um inimigo morre.
- `collect`: em `pickItem`, quando o herói encosta num item.
- `goto` e `boss`: em `checkGotoAndBoss`, chamado em todo quadro.

js/game.js — linhas 312–324

```
function onKill(type) {
  for (const quest of G.ch.quests) {
    if (!questActive(quest)) continue;
    const q = qState(quest.id);
    if (quest.id !== 'main' && q.step === 0) continue;
    const s = curStep(quest);
    if (s && s.t === 'kill' && s.e === type) {
      q.prog++;
      if (q.prog >= s.n) advance(quest);
      else UI.toast(`${ENEMIES[type].name}: ${q.prog}/${s.n}`, 'prog');
    }
  }
}
```

js/game.js — linhas 326–352

```
function checkGotoAndBoss() {
  for (const quest of G.ch.quests) {
    if (!questActive(quest)) continue;
    const q = qState(quest.id);
    if (quest.id !== 'main' && q.step === 0) continue;
    const s = curStep(quest);
    if (!s) continue;
    const p = G.world.places[s.at];
    if (!p) continue;
    const d = dist(G.player.x, G.player.y, p.px, p.py);
    if (s.t === 'goto' && d < p.r * TILE) {
      if (s.say) { const sv = s.say; UI.dialog(sv, () => advance(quest)); }
      else advance(quest);
      return;
    }
    if (s.t === 'boss' && !G.boss && d < p.r * TILE + 260) {
      const b = new Enemy(s.e, p.px, p.py - TILE, G.mult, null);
      b.aggro = true;
      G.enemies.push(b);
      G.boss = b; b.questId = quest.id;
      Sfx.play('boss');
      G.shake = 12;
      UI.toast(T('appears', b.d.name), 'boss');
```

```
      if (s.intro) UI.dialog(s.intro);
    }
  }
}
```

js/game.js — linhas 354–376

```
function talkTo(npc) {
  npc.aim = Math.atan2(G.player.y - npc.y, G.player.x - npc.x);
  for (const quest of G.ch.quests) {
    if (!questActive(quest)) continue;
    const s = curStep(quest);
    if (!s || s.npc !== npc.id) continue;
    if (s.t === 'talk') {
      if (quest.id !== 'main' && qState(quest.id).step === 0) UI.toast(T('newQuest', quest.title), 'side');
      UI.dialog(s.say || [[npc.id, '...']], () => advance(quest));
      return;
    }
    if (s.t === 'quiz') { runQuiz(quest, s, npc); return; }
  }
  if (canAdvanceHere() && guideNpc() === npc) {
    UI.dialog([[npc.id, T('guideLine')]], offerNextChapter);
    return;
  }
  const lines = npc.def.lines || ['...'];
  const line = lines[npc.line % lines.length];
  npc.line++;
  if (npc.def.shop) UI.dialog([[npc.id, line]], () => UI.shop(npc));
  else UI.dialog([[npc.id, line]]);
}
```

Observe a **ordem de prioridade** em `talkTo`: primeiro verifica se alguma missão quer conversar com esse NPC agora (a principal vem primeiro, porque é a primeira da lista). Se não houver, e o NPC for comerciante, abre a loja. Senão, diz uma das falas soltas.

O quiz

js/game.js — linhas 378–389

```
function runQuiz(quest, s, npc) {
  let i = 0;
  const ask = () => {
    if (i >= s.qs.length) { UI.dialog(s.win || [[npc.id, T('quizWin')]], () => advance(quest)); return; }
    const qq = s.qs[i];
    UI.quiz(npc.name, qq.q, qq.o, (choice) => {
      if (choice === qq.a) { Sfx.play('pickup'); i++; ask(); }
      else { Sfx.play('hurt'); UI.dialog([[npc.id, T('quizFail')]]); }
    });
  };
  UI.dialog(s.say || [[npc.id, T('quizAsk')]], ask);
}
```

`runQuiz` usa uma função interna `ask` que **chama a si mesma** depois de cada resposta certa, até acabarem as perguntas. Uma função que chama a si mesma é **recursiva**. Ao errar, o NPC sugere ler as crônicas, e o jogador pode tentar de novo, começando do zero.

A seta do objetivo

js/game.js — linhas 292–310

```
function questTarget(quest) {
  const s = curStep(quest);
  if (!s) return null;
  if (s.t === 'talk' || s.t === 'quiz') {
    const o = G.objs.find(o => o.kind === 'npc' && o.id === s.npc);
    return o ? { x: o.x, y: o.y, label: o.name } : null;
  }
  if (s.t === 'collect') {
    // aponta para o item restante mais próximo
    let best = null, bd = 1e9;
    for (const o of G.objs) if (o.kind === 'item' && o.qid === quest.id) {
      const d = dist(G.player.x, G.player.y, o.x, o.y);
      if (d < bd) { bd = d; best = o; }
    }
    if (best) return { x: best.x, y: best.y, label: s.item };
  }
  const p = G.world.places[s.at];
  return p ? { x: p.px, y: p.py, label: p.name } : null;
}
```

Para cada tipo de etapa, `questTarget` sabe onde é o objetivo: o NPC (falar), o item mais próximo (coletar) ou o centro do lugar (os demais). É esse ponto que a seta dourada e o minimapa mostram.

O "!" sobre os NPCs

js/game.js — linhas 1011–1019

```
function npcMarker(npc) {
  for (const quest of G.ch.quests) {
    if (!questActive(quest)) continue;
    const s = curStep(quest);
    if (s && (s.t === 'talk' || s.t === 'quiz') && s.npc === npc.id) return quest.id === 'main' ? '#ffd84a'
: '#9ad0ff';
  }
  if (canAdvanceHere() && guideNpc() === npc) return '#ffd84a';
  return null;
}
```

Um "!" **dourado** indica que a história principal quer falar com aquela pessoa; um "!" **azul**, uma missão secundária.

10.5 Entrando e saindo de capítulos

js/game.js — linhas 112–179

```
function loadChapter(idx, keepPos) {
  G.chIdx = idx;
  G.S.chapter = idx;
  G.S.maxChapter = Math.max(G.S.maxChapter, idx);
  G.ch = CHAPTERS[idx];
  G.mult = 1 + 0.32 * idx;
  G.world = new World(G.ch);
  G.enemies = []; G.projs = []; G.parts = []; G.texts = []; G.objs = []; G.camps = []; G.fx = [];
  G.boss = null;
```

```
  const start = G.world.places[G.ch.places[0].id];
  G.player = new Player(start.px, start.py + TILE * 2);
  if (keepPos && G.S.pos && G.S.pos.ch === idx && !G.world.collides(G.S.pos.x, G.S.pos.y, 10)) {
    G.player.x = G.S.pos.x; G.player.y = G.S.pos.y;
  }
  const w = weapon();
  G.player.ammo = w.mag || 0;
  G.S.hp = Math.min(G.S.hp || maxHp(), maxHp());
  if (G.S.hp <= 0) G.S.hp = maxHp();

  const cs = chState(G.ch.id);
  // NPCs
  for (const n of G.ch.npcs) {
    const p = G.world.places[n.at];
    G.objs.push({ kind: 'npc', id: n.id, def: n, name: n.name, x: p.px + (n.dx || 0) * TILE, y: p.py + (n.dy
|| 0) * TILE, r: 12, line: 0, aim: Math.PI / 2 });
  }
  // Crônicas (codex) em posições determinísticas
  const rng = mulberry32(G.ch.seed + 777);
  for (const c of G.ch.codex) {
    if (cs.codex[c.id]) continue;
    const p = G.world.places[c.at];
    let spot = null;
    for (let i = 0; i < 80 && !spot; i++) {
      const a = rng() * Math.PI * 2, d = p.r + 2 + rng() * 6;
      const tx = Math.round(p.x + Math.cos(a) * d), ty = Math.round(p.y + Math.sin(a) * d);
      if (tx > 2 && ty > 2 && tx < G.world.W - 3 && ty < G.world.H - 3 && G.world.reach[G.world.idx(tx,
ty)]) spot = { x: tx * TILE + 16, y: ty * TILE + 16 };
    }
    spot = spot || { x: p.px + 40, y: p.py };
    G.objs.push({ kind: 'codex', id: c.id, def: c, name: T('chronicle'), x: spot.x, y: spot.y, r: 12 });
  }
  // Baús
  const crng = mulberry32(G.ch.seed + 4242);
  for (let i = 0; i < 6; i++) {
    const spot = G.world.randomReachableFar(crng, G.ch.places, 9);
    if (!spot) continue;
    if (cs.chests[i]) { G.objs.push({ kind: 'chest', idx: i, opened: true, name: T('chestEmpty'), x: spot.x,
y: spot.y, r: 12 }); continue; }
    G.objs.push({ kind: 'chest', idx: i, opened: false, name: T('chest'), x: spot.x, y: spot.y, r: 12 });
  }
  // Acampamentos: os dos dados + um para cada etapa de "kill"
  const campKeys = new Set();
  const addCamp = (at, type, n) => {
    const key = at + ':' + type;
    if (campKeys.has(key)) return;
    campKeys.add(key);
    G.camps.push({ id: key, at, type, n, respawnT: 0 });
  };
  for (const c of (G.ch.camps || [])) addCamp(c.at, c.type, c.n);
  for (const q of G.ch.quests) for (const s of q.steps) if (s.t === 'kill') addCamp(s.at, s.e,
Math.min(Math.max(4, Math.ceil(s.n * 0.6)), 7));
  for (const camp of G.camps) for (let i = 0; i < camp.n; i++) spawnCampEnemy(camp);
  // Itens de missões de coleta ativas
  for (const q of G.ch.quests) {
    if (!questActive(q)) continue;
    const s = curStep(q);
    if (s && s.t === 'collect' && (q.id === 'main' || qState(q.id).step > 0)) spawnItems(q, s, s.n -
qState(q.id).prog);
  }
  G.cam.x = G.player.x - VW / 2; G.cam.y = G.player.y - VH / 2;
```

```
  UI.hud(true);
  save();
}
```

`loadChapter` é a "fábrica" de um capítulo. Em ordem: define o multiplicador, **gera o mapa** (`new World`), posiciona o herói (no início ou onde ele estava no save), cria NPCs, crônicas ainda não lidas e baús (lembrando quais já foram abertos), cria os acampamentos (os definidos nos dados mais um para cada etapa de caça), povoa os acampamentos, recria os itens de coleta que faltam e salva.

js/game.js — linhas 276–290

```
function enterChapter(idx, keepPos) {
  G.S.hp = 9999;
  loadChapter(idx, keepPos);
  G.S.hp = maxHp();
  Music.start(G.ch.id);
  const cs = chState(G.ch.id);
  if (!cs.intro) {
    cs.intro = true;
    UI.narration(T('chapterN', idx + 1, G.ch.title), G.ch.intro, () => {
      const s = curStep(G.ch.quests[0]);
      if (s) UI.toast('★ ' + s.desc, 'main');
      save();
    }, G.ch.years);
  }
}
```

`enterChapter` envolve `loadChapter`, enche a vida, troca a música e mostra a narração de abertura **só na primeira vez**.

js/game.js — linhas 237–247

```
function chapterComplete() {
  const cs = chState(G.ch.id);
  cs.done = true;
  const next = G.chIdx + 1;
  if (next < CHAPTERS.length) G.S.maxChapter = Math.max(G.S.maxChapter, next);
  save();
  UI.narration(G.ch.title, G.ch.outro, () => {
    if (next < CHAPTERS.length) offerNextChapter();
    else UI.ending();
  });
}
```

Ao concluir a missão principal, o jogo mostra a narração de encerramento e pergunta ao jogador se ele quer **partir para a próxima era** ou **ficar e explorar** a época atual (terminar missões secundárias, achar crônicas e baús). No último capítulo, mostra o final.

js/game.js — linhas 260–268

```
function offerNextChapter() {
  const next = CHAPTERS[G.chIdx + 1];
  const guide = guideNpc();
  const guideName = guide ? guide.name : '?';
  UI.quiz(T('crossroads'), T('crossroadsQ', next.title, next.years), [T('goNext'), T('stayHere',
guideName)], (i) => {
    if (i === 0) goToNextChapter();
```

```
    else { UI.toast(T('stayToast', guideName), 'main'); save(); UI.hud(true); }
  });
}
```

Quem fica pode partir quando quiser falando com o **guia** do capítulo, que é o personagem da primeira conversa da história (a Anciã Ama no capítulo 1). O guia ganha um "!" dourado e a seta do objetivo passa a apontar para ele:

js/game.js — linhas 251–254

```
function guideNpc() {
  const s = G.ch.quests[0].steps.find(st => st.t === 'talk');
  return s ? G.objs.find(o => o.kind === 'npc' && o.id === s.npc) : null;
}
```

Ao partir, `goToNextChapter` mostra a tela de **evolução da linhagem** (o herói antigo ao lado do novo, com a arma nova) e carrega o próximo capítulo.

🛠️ Mão na massa

Crie a sua primeira missão. No capítulo 1 (`js/data/chapters1.js`), adicione esta missão no fim da lista `quests` , logo antes do `]` final, separada por vírgula:

```
{ id: 's5', title: 'Mais Lobos', reward: { xp: 50, gold: 20 }, steps: [
  { t: 'talk', npc: 'beltz', desc: 'Fale com Beltz', say: [['beltz', 'Os lobos voltaram! Cace três.']]
},
  { t: 'kill', e: 'lobo', n: 3, at: 'bosque', desc: 'Cace 3 lobos' },
  { t: 'talk', npc: 'beltz', desc: 'Volte a Beltz', say: [['beltz', 'Obrigado, caçador!']] }
] }
```

Recarregue o jogo, fale com Beltz (ele terá um "!" azul) e cumpra a missão. Como Beltz também oferece a missão dos javalis, que vem antes na lista, ele vai propor primeiro aquela; conclua-a e a sua aparece em seguida. Você não escreveu nenhuma lógica: só dados.

Para a missão aparecer também em espanhol, acrescente a tradução em `js/data/es1.js` , dentro de `ES_CHAPTERS.c1.quests` , com o mesmo id `s5` e o mesmo número de etapas. Sem a tradução, o jogo em espanhol mostra o texto em português (não quebra).

CAPÍTULO 11

A interface: HUD, diálogos e janelas

Toda a interface do jogo (HUD, caixas de diálogo, loja, diário, crônicas, mapa, menus, tela de título) mora num único objeto, `UI`, no arquivo `ui.js`. Ela é feita em **HTML por cima do canvas**, como explicado no capítulo 2.

11.1 Manipulando o HTML pelo JavaScript

Tudo o que a `UI` faz se resume a quatro operações do DOM:

```
const el = document.getElementById('dialog'); // 1. achar um elemento
el.textContent = 'Olá!';                      // 2. trocar o texto
el.innerHTML = '<b>Olá!</b>';                 // 3. trocar o conteúdo HTML
el.classList.add('hidden');                   // 4. ligar/desligar classes CSS
el.onclick = () => { ... };                   //    e reagir a cliques
```

O arquivo começa com um atalho, porque `document.getElementById` é usado dezenas de vezes:

js/ui.js — linhas 4–4

```
const $ = (id) => document.getElementById(id);
```

11.2 O HUD

js/ui.js — linhas 11–44

```
hud(force) {
  if (!G.S || !G.ch) return;
  const S = G.S, cls = CLASSES[G.ch.cls], w = weapon();
  $('h-name').textContent = S.name;
  $('h-class').textContent = cls.name + ' · ' + T('lvl') + ' ' + S.level;
  const mh = maxHp();
  $('h-hp').style.width = (100 * S.hp / mh) + '%';
  $('h-hpt').textContent = `${Math.ceil(S.hp)} / ${mh}`;
  const need = xpNeed(S.level);
  $('h-xp').style.width = (100 * S.xp / need) + '%';
  $('h-xpt').textContent = `XP ${S.xp} / ${need}`;
  $('h-gold').textContent = ` 💰 ${S.gold} ${G.ch.currency}`;
  $('h-pot').textContent = ` 🧪 ${S.potions} [Q]`;
  let ammo = w.name;
  if (w.kind === 'gun') ammo += G.player.reloadT > 0 ? ' · ' + T('reloading') : ` ·
${G.player.ammo}/${w.mag} [R]`;
  $('h-ammo').textContent = '⚔ ' + ammo;
  $('h-chapter').textContent = T('chapterHud', G.chIdx + 1, CHAPTERS.length, G.ch.title, G.ch.years);
  if (force || !this._trT || G.time - this._trT > 0.5) {
    this._trT = G.time;
    let html = '';
    for (const quest of G.ch.quests) {
      const q = qState(quest.id);
      if (q.done) continue;
      if (quest.id !== 'main' && q.step === 0) continue;
```

```
      const s = curStep(quest);
      if (!s) continue;
      let prog = '';
      if (s.t === 'kill' || s.t === 'collect') prog = ` <b>${q.prog}/${s.n}</b>`;
      html += `<div class="tq ${quest.id === 'main' ? 'main' : ''}"><div class="tt">${quest.id === 'main' ?
'★' : '◆'} ${escapeHtml(quest.title)}</div><div>${escapeHtml(s.desc)}${prog}</div></div>`;
    }
    if (qState('main').done) html = `<div class="tq main"><div class="tt">${T('chapterDone')}</div>
<div>${canAdvanceHere() && guideNpc() ? T('chapterDoneTxt', escapeHtml(guideNpc().name)) :
T('chapterDoneLast')}</div></div>` + html;
    $('tracker').innerHTML = html;
  }
},
```

O HUD atualiza larguras de barras (`style.width = '75%'`) e textos. O rastreador de missões, à direita, é remontado no máximo 2 vezes por segundo, porque montar HTML é mais caro do que trocar um texto.

11.3 Notificações (toasts)

js/ui.js — linhas 46–55

```
toast(msg, kind) {
  const el = document.createElement('div');
  el.className = 'toast ' + (kind || '');
  el.textContent = msg;
  $('toasts').appendChild(el);
  const box = $('toasts');
  while (box.children.length > 5) box.removeChild(box.firstChild);
  setTimeout(() => el.classList.add('out'), kind === 'prog' ? 1400 : 3200);
  setTimeout(() => el.remove(), kind === 'prog' ? 1900 : 3800);
},
```

Cria um `<div>`, coloca no topo da tela e, com dois `setTimeout`, primeiro adiciona a classe `out` (que o CSS anima com transparência) e depois remove o elemento. `setTimeout(funcao, ms)` agenda uma função para daqui a tantos milissegundos. Se houver mais de 5 notificações, as mais antigas saem.

11.4 O sistema de diálogo

O diálogo é uma **fila de falas** com um callback no fim:

js/ui.js — linhas 65–72

```
dialog(lines, cb) {
  if (!lines || !lines.length) { if (cb) cb(); return; }
  G.ui = 'dialog';
  G.mouse.down = false;
  this.dlg = { lines: lines.map(l => typeof l === 'string' ? ['N', l] : l), i: 0, cb, t: performance.now()
};
  this.showLine();
  $('dialog').classList.remove('hidden');
},
```

js/ui.js linhas 84–97

```
nextLine() {
  const d = this.dlg;
  if (!d || d.quiz) return;
  if (performance.now() - d.t < 180) return;
  d.t = performance.now();
  Sfx.play('click');
  d.i++;
  if (d.i >= d.lines.length) {
    $('dialog').classList.add('hidden');
    this.dlg = null;
    G.ui = null;
    if (d.cb) d.cb();
  } else this.showLine();
},
```

- `dialog` guarda as falas, marca `G.ui = 'dialog'` (o que **pausa o jogo**, porque o laço só chama `update` quando `G.ui` está vazio) e mostra a primeira.
- `nextLine` avança. Quando acabam as falas, esconde a caixa, libera o jogo e **chama o callback**.
- A pausa de 180 ms (`performance.now() - d.t < 180`) evita que um clique duplo pule duas falas sem querer.

Esse padrão de "mostre isto e, quando acabar, faça aquilo" permite encadear sequências inteiras: falas → avançar missão → falas de conclusão → próxima etapa.

11.5 Janelas (modais)

Loja, diário, crônicas, mapa, menu de pausa, narração, evolução, queda e final usam a mesma peça: `modal(html, classe)`, que coloca o HTML numa caixa centralizada sobre um fundo escurecido, e `closeModal()`, que a fecha.

js/ui.js linhas 128–137

```
modal(html, cls, onClose) {
  G.ui = 'modal';
  G.mouse.down = false;
  const m = $('modal');
  m.className = cls || '';
  m.querySelector('.box').innerHTML = html;
  m.classList.remove('hidden');
  this.modalClose = onClose || null;
  return m.querySelector('.box');
},
```

Cada tela é uma função que **monta um texto HTML** com template strings e depois liga os botões. A loja é um bom exemplo, que inclui a lógica de compra:

js/ui.js linhas 198–227

```
shop(npc) {
  const m = G.mult, S = G.S, cur = G.ch.currency;
  const items = [
    { id: 'pot', name: T('shopPot'), desc: T('shopPotD'), price: Math.round(12 * m) },
    { id: 'pot5', name: T('shopPot5'), desc: T('shopPot5D'), price: Math.round(48 * m) },
    { id: 'sharp', name: T('shopSharp', S.sharp + 1), desc: T('shopSharpD'), price: Math.round(45 * m * (1 +
S.sharp * 0.6)) },
    { id: 'armor', name: T('shopArmor', S.armor + 1), desc: T('shopArmorD'), price: Math.round(45 * m * (1 +
```

```
S.armor * 0.6)), max: S.armor >= 8 }
  ];
  const box = this.modal(`
    <h2>${escapeHtml(npc.name)}</h2>
    <p class="dim">${T('youHave')} <b>${S.gold} ${escapeHtml(cur)}</b> · ${S.potions} ${T('potions')} ·
${T('weapon')} +${S.sharp} · ${T('armor')} +${S.armor}</p>
    <div class="shop">${items.map(it => `
      <div class="shop-it"><div><b>${escapeHtml(it.name)}</b><div class="dim">${escapeHtml(it.desc)}</div>
</div>
      <button data-id="${it.id}" ${it.max || S.gold < it.price ? 'disabled' : ''}>${it.max ? T('max') :
it.price + ' ' + escapeHtml(cur)}</button></div>`).join('')}
    </div>
    <div class="center"><button id="sh-x">${T('leave')}</button></div>`, 'shop-modal');
  box.querySelectorAll('button[data-id]').forEach(b => b.onclick = () => {
    const it = items.find(i => i.id === b.dataset.id);
    if (S.gold < it.price || it.max) return;
    S.gold -= it.price;
    if (it.id === 'pot') S.potions++;
    if (it.id === 'pot5') S.potions += 5;
    if (it.id === 'sharp') S.sharp++;
    if (it.id === 'armor') S.armor++;
    Sfx.play('coin');
    save();
    this.shop(npc);
  });
  box.querySelector('#sh-x').onclick = () => this.closeModal();
},
```

Repare que, depois de cada compra, a loja chama `this.shop(npc)` de novo: ela se **redesenha inteira** com os valores atualizados (dinheiro, preços, botões desabilitados). Redesenhar tudo é mais simples e menos sujeito a erros do que atualizar cada pedaço.

11.6 Segurança: escapeHtml

js/util.js — linhas 56–58

```
function escapeHtml(s) {
  return String(s).replace(/[&<>"]/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;' }[c]));
}
```

Sempre que um texto que veio "de fora" (o nome que o jogador digitou, por exemplo) é colocado dentro de `innerHTML`, ele passa por `escapeHtml`. Se alguém digitasse `<img src=x onerror=alert(1)>` como nome da linhagem, sem essa proteção o navegador **executaria** aquilo como HTML. Com ela, os símbolos `<` e `>` viram texto inofensivo. Esse tipo de ataque se chama **XSS** (*cross-site scripting*) e é uma das falhas de segurança mais comuns na web.

> ⚠️ **Atenção**
>
> Regra de ouro: **nunca** coloque texto vindo do usuário (ou de um arquivo importado) em `innerHTML` sem escapar. Quando possível, use `textContent`, que já trata tudo como texto puro.

11.7 Roteando o teclado

js/ui.js — linhas 500–525

```
onKey(e) {
  if (G.ui === 'dialog') {
    if (this.dlg && this.dlg.quiz) {
      const n = parseInt(e.key, 10);
      if (n >= 1 && n <= this._quizN) this.answer(n - 1);
    } else if (['Space', 'Enter', 'KeyE', 'NumpadEnter'].includes(e.code)) this.nextLine();
    return true;
  }
  if (G.ui === 'modal') {
    if (this._narrNext && ['Space', 'Enter', 'NumpadEnter'].includes(e.code)) { e.preventDefault();
this._narrNext(); return true; }
    const cls = $('modal').className;
    if (e.code === 'Escape' || (e.code === 'KeyJ' && cls === 'journal') || (e.code === 'KeyC' && cls ===
'journal') || (e.code === 'KeyM' && cls === 'mapmodal')) {
      if (cls === 'narration') return true;
      this.closeModal();
    }
    return true;
  }
  if (!G.running) return false;
  switch (e.code) {
    case 'Escape': this.pause(); return true;
    case 'KeyJ': case 'Tab': this.journal(); return true;
    case 'KeyC': this.codex(); return true;
    case 'KeyM': this.map(); return true;
  }
  return false;
},
```

O teclado precisa saber "quem está no comando": se há um diálogo, Espaço avança a fala; se há um quiz, as teclas 1 a 4 respondem; se há uma janela, Esc a fecha; se não há nada, J, C, M e Esc abrem as janelas. `onKey` devolve `true` quando consumiu a tecla, e o jogo então ignora o resto.

11.8 A tela de título e os espaços de save

js/ui.js — linhas 408–490

```
title() {
  const el = $('title');
  const prevName = $('t-name') ? $('t-name').value : 'Íber';
  let rows = '';
  for (let n = 1; n <= SLOT_COUNT; n++) {
    const s = loadSave(n);
    if (s) {
      const ch = CHAPTERS[clamp(s.chapter || 0, 0, CHAPTERS.length - 1)];
      const t = Math.floor((s.stats && s.stats.time) || 0);
      const when = s.savedAt ? new Date(s.savedAt).toLocaleString(LANG === 'es' ? 'es-ES' : 'pt-BR', {
dateStyle: 'short', timeStyle: 'short' }) : '';
      rows += `
        <div class="slot">
          <div class="slot-info">
            <div class="slot-name"><span class="slot-n">${n}</span> ${escapeHtml(s.name)} <span
class="dim">· ${T('lvl')} ${s.level}</span></div>
            <div class="dim">${escapeHtml(T('chapterN', (s.chapter || 0) + 1, ch.title))} · ${Math.floor(t /
```

```
3600)}h ${Math.floor(t / 60) % 60}min${when ? ' · ' + T('savedAt', when) : ''}</div>
          </div>
          <button data-play="${n}">${T('slotPlay')}</button>
          <button class="danger" data-del="${n}">${T('deleteSave')}</button>
        </div>`;
    } else {
      rows += `
        <div class="slot empty">
          <div class="slot-info"><div class="slot-name"><span class="slot-n">${n}</span> <span
class="dim">${T('slotEmpty')}</span></div></div>
          <button data-new="${n}">${T('newGame')}</button>
          <label class="filebtn">${T('importSave')}<input type="file" data-imp="${n}"
accept=".json,application/json"></label>
        </div>`;
    }
  }
  el.innerHTML = `
    <div class="t-box">
      <div class="t-logo">${T('logo')}</div>
      <div class="t-sub">${T('subtitle')}</div>
      <p class="t-desc">${T('titleDesc')}</p>
      ${this.langButtons()}
      <div class="t-new">
        <label for="t-name" class="dim">${T('lineagePh')}</label>
        <input id="t-name" maxlength="16" placeholder="${T('lineagePh')}" value="${escapeHtml(prevName)}">
      </div>
      <div class="slots">${rows}</div>
      <div class="t-row">
        <button id="t-help">${T('help')}</button>
        <button id="t-mus" class="mute">${Music.enabled ? '🔊' : '🔇'}</button>
      </div>
      <p class="t-foot">${T('titleFoot')}</p>
    </div>`;
  el.classList.remove('hidden');
  // a música só pode começar depois de um clique (regra dos navegadores)
  el.onpointerdown = () => { Sfx.init(); Music.start('title'); };
  const begin = (state, n) => {
    Sfx.init();
    el.classList.add('hidden');
    el.onpointerdown = null;
    $('hud').classList.remove('hidden');
    startGame(state, n);
  };
  this.bindLang(el);
  el.querySelectorAll('[data-play]').forEach(b => b.onclick = () => {
    const n = +b.dataset.play;
    begin(loadSave(n), n);
  });
  el.querySelectorAll('[data-new]').forEach(b => b.onclick = () => {
    const n = +b.dataset.new;
    const name = ($('t-name').value || 'Íber').trim().slice(0, 16) || 'Íber';
    begin(newState(name), n);
  });
  el.querySelectorAll('[data-del]').forEach(b => b.onclick = () => {
    const n = +b.dataset.del, s = loadSave(n);
    if (!s || !confirm(T('confirmDelete', s.name, (s.chapter || 0) + 1, s.level, n))) return;
    deleteSave(n);
    this.title();
    this.titleToast(T('deleted', n));
  });
  el.querySelectorAll('[data-imp]').forEach(inp => inp.onchange = (e) => {
```

```
    const f = e.target.files[0], n = +inp.dataset.imp;
    if (!f) return;
    importSave(f, n, (ok) => { if (ok) this.title(); else alert(T('badSave')); });
  });
  $('t-mus').onclick = (e) => { e.stopPropagation(); Music.setEnabled(!Music.enabled); if (Music.enabled)
Music.start('title'); this.title(); };
  $('t-help').onclick = () => {
    el.classList.add('hidden');
    this.help(() => { G.ui = null; el.classList.remove('hidden'); });
  };
},
```

A tela de título lista os 5 espaços de save. Para cada espaço, lê o save com `loadSave(n)` e monta uma linha: preenchido (nome, nível, capítulo, tempo de jogo, data) com **Continuar** e **Excluir save**; ou vazio, com **Novo jogo** e **Importar save**.

Os botões carregam o número do espaço num atributo `data-*` (`data-play="3"`, `data-del="3"`), lido depois com `b.dataset.play`. É uma forma elegante de ligar muitos botões parecidos com um único trecho de código:

```
el.querySelectorAll('[data-del]').forEach(b => b.onclick = () => {
  const n = +b.dataset.del;     // o "+" converte o texto "3" no número 3
  ...
});
```

> 📌 **Nota**
>
> Os navegadores só permitem tocar som depois que o usuário interage com a página. Por isso a música do título só começa no **primeiro clique** (`el.onpointerdown`).

CAPÍTULO 12

Salvando o progresso

12.1 localStorage

O `localStorage` é um pequeno armazenamento que o navegador oferece a cada site. Ele guarda pares de **chave** → **texto** e tem três funções principais:

```
localStorage.setItem('chave', 'valor');   // guardar
localStorage.getItem('chave');            // ler (null se não existir)
localStorage.removeItem('chave');         // apagar
```

Os dados sobrevivem a fechar o navegador e desligar o computador. Mas atenção: ficam **naquele navegador, naquele computador**. Em modo anônimo, eles somem ao fechar a janela. Limpar os dados do site também os apaga. Por isso o jogo oferece exportar o save como arquivo.

12.2 Os cinco espaços

js/game.js linhas 64–64

```
const SLOT_COUNT = 5;
```

js/game.js linhas 65–65

```
const slotKey = (n) => SAVE_KEY + '_slot' + n;
```

Cada espaço é uma chave diferente: `hispania_save_v1_slot1` até `hispania_save_v1_slot5`. A variável `G.slot` lembra em qual espaço a partida atual está, e `save()` sempre grava ali:

js/game.js linhas 76–81

```
function save() {
  if (!G.S || !G.slot) return;
  if (G.player) { G.S.pos = { ch: G.chIdx, x: G.player.x, y: G.player.y }; }
  G.S.savedAt = Date.now();
  try { localStorage.setItem(slotKey(G.slot), JSON.stringify(G.S)); } catch (e) { /* armazenamento
indisponível */ }
}
```

`save()` é chamada em vários momentos: a cada 20 segundos, ao concluir etapas e missões, ao abrir baús, ao comprar na loja e ao trocar de capítulo. Assim, perder progresso é quase impossível.

12.3 Migrando saves antigos

A versão anterior do jogo usava uma única chave (`hispania_save_v1`). Para quem já tinha jogado não perder o progresso ao atualizar, há uma pequena **migração**:

js/game.js linhas 68–74

```
(function migrateOldSave() {
  try {
    const old = localStorage.getItem(SAVE_KEY);
    if (old && !localStorage.getItem(slotKey(1))) localStorage.setItem(slotKey(1), old);
    if (old) localStorage.removeItem(SAVE_KEY);
  } catch (e) { /* armazenamento indisponível */ }
})();
```

O formato `(function () { ... })();` cria uma função e a executa **imediatamente**, uma única vez, quando o arquivo carrega. Isso se chama **IIFE** (*Immediately Invoked Function Expression*). Se existir um save antigo e o espaço 1 estiver livre, ele é copiado para o espaço 1 e a chave antiga é apagada.

> 💡 **Dica**
>
> Sempre que mudar o formato de dados salvos de um programa, pense em quem já tem dados no formato antigo. Uma migração de 5 linhas evita que os jogadores percam horas de progresso.

12.4 Excluir, exportar e importar

js/game.js linhas 85–87

```
function deleteSave(n) {
  try { localStorage.removeItem(slotKey(n)); } catch (e) { /* armazenamento indisponível */ }
}
```

js/game.js linhas 88–96

```
function exportSave() {
  save();
  const blob = new Blob([JSON.stringify(G.S, null, 1)], { type: 'application/json' });
  const a = document.createElement('a');
  a.href = URL.createObjectURL(blob);
  a.download = `hispania-${(G.S.name || 'save').replace(/[^\wÀ-ú-]+/g, '_')}.json`;
  a.click();
  setTimeout(() => URL.revokeObjectURL(a.href), 1000);
}
```

js/game.js linhas 98–109

```
function importSave(file, n, cb) {
  const r = new FileReader();
  r.onload = () => {
    try {
      const s = JSON.parse(r.result);
      if (!s || typeof s.level !== 'number' || !s.ch) throw new Error('inválido');
      localStorage.setItem(slotKey(n), JSON.stringify(s));
      cb(true);
    } catch (e) { cb(false); }
  };
  r.readAsText(file);
}
```

- **Exportar**: transforma o save em texto JSON, embrulha num `Blob` (um "arquivo na memória"), cria um endereço temporário para ele com `URL.createObjectURL` e simula um clique num link de download.
- **Importar**: o `FileReader` lê o arquivo escolhido pelo jogador. O conteúdo passa por `JSON.parse` e por uma **validação** (tem `level` numérico? tem `ch`?) antes de ser aceito. Tudo dentro de `try/catch`, porque o jogador pode escolher qualquer arquivo, até uma foto.

CAPÍTULO 13

Dois idiomas: internacionalização

Fazer um programa funcionar em vários idiomas se chama **internacionalização**, abreviada como **i18n** (há 18 letras entre o "i" e o "n" de *internationalization*). O jogo resolve isso em duas camadas: os **textos da interface** e o **conteúdo** (falas, missões, crônicas).

13.1 Qual idioma usar

js/i18n.js — linhas 7–10

```
const LANG = (() => {
  try { const l = localStorage.getItem(LANG_KEY); if (l === 'pt' || l === 'es') return l; } catch (e) { /*
sem armazenamento */ }
  return (navigator.language || '').toLowerCase().startsWith('es') ? 'es' : 'pt';
})();
```

Na primeira visita, o jogo olha o idioma do navegador (`navigator.language`). Se começar com "es", usa espanhol. Depois que o jogador escolhe, a escolha fica salva em `localStorage`. Trocar de idioma salva e **recarrega a página**: é mais simples e seguro do que trocar todos os textos "ao vivo".

13.2 Textos da interface: a função T

Todos os textos fixos (botões, avisos, menus) estão num dicionário `STR` com uma versão `pt` e uma `es` para cada chave:

```
const STR = {
  pt: { closer: 'Chegue mais perto.', levelUp: 'Nível {0}! Vida e dano aumentaram.', ... },
  es: { closer: 'Acércate más.',      levelUp: '¡Nivel {0}! Aumentan la vida y el daño.', ... }
};
```

js/i18n.js — linhas 141–145

```
function T(key, ...args) {
  const s = (STR[LANG] && STR[LANG][key] != null) ? STR[LANG][key] : STR.pt[key];
  if (typeof s !== 'string') return s;
  return s.replace(/\{(\d+)\}/g, (_, i) => args[i] != null ? args[i] : '');
}
```

`T('levelUp', 5)` procura a chave no idioma atual (ou, se faltar, em português) e troca `{0}` pelo primeiro argumento. O `...args` (*rest parameter*) junta todos os argumentos extras numa lista.

No código, em vez de escrever `UI.toast('Chegue mais perto.')`, escreve-se `UI.toast(T('closer'))`. Essa troca foi feita em todo o jogo quando o espanhol foi adicionado.

13.3 O conteúdo: tradução por sobreposição

Traduzir 14 capítulos duplicando os arquivos inteiros seria arriscado: toda correção de lógica (um número de inimigos, por exemplo) teria de ser feita em dois lugares. A solução foi **sobrepor só os textos**. Os arquivos `es1.js`, `es2.js` e `es3.js` têm a mesma estrutura dos capítulos, mas **apenas com os campos de texto**, organizados pelos mesmos ids:

```
ES_CHAPTERS.c1 = {
  title: 'Los orígenes',
  places: { aldeia: 'Aldea del clan', bosque: 'Bosque de los Lobos', ... },
  npcs: { ama: { name: 'Anciana Ama', lines: ['El fuego es nuestra vida...'] } },
  quests: { main: { title: 'El invierno del clan', steps: [ { desc: '...', say: [...] }, ... ] } }
};
```

Ao iniciar, se o idioma for espanhol, `applyLanguage` copia esses textos por cima dos dados portugueses:

js/i18n.js — linhas 153–172

```
function applyLanguage() {
  if (LANG !== 'es') return;
  for (const [k, v] of Object.entries(ES_BASE.weapons)) if (WEAPONS[k]) Object.assign(WEAPONS[k], v);
  for (const [k, v] of Object.entries(ES_BASE.classes)) if (CLASSES[k]) CLASSES[k].name = v;
  for (const [k, v] of Object.entries(ES_BASE.enemies)) if (ENEMIES[k]) ENEMIES[k].name = v;
  for (const ch of CHAPTERS) {
    const o = ES_CHAPTERS[ch.id];
    if (!o) continue;
    for (const f of ['title', 'years', 'currency', 'intro', 'outro']) if (o[f] != null) ch[f] = o[f];
    if (o.places) for (const p of ch.places) if (o.places[p.id]) p.name = o.places[p.id];
    if (o.npcs) for (const n of ch.npcs) { const t = o.npcs[n.id]; if (t) { if (t.name) n.name = t.name; if
(t.lines) n.lines = t.lines; } }
    if (o.codex) for (const c of ch.codex) { const t = o.codex[c.id]; if (t) Object.assign(c, t); }
    if (o.quests) for (const q of ch.quests) {
      const t = o.quests[q.id];
      if (!t) continue;
      if (t.title) q.title = t.title;
      if (t.steps) t.steps.forEach((ts, i) => { if (ts && q.steps[i]) Object.assign(q.steps[i], ts); });
    }
  }
}
```

Para as etapas das missões, a tradução é aplicada **pela posição** (`steps[i]`). Por isso a ordem e a quantidade de etapas precisam ser idênticas nos dois idiomas, e foi criado um teste automático que confere isso (capítulo 15).

> ❓ **Por quê?**
>
> Números, tipos de inimigo, posições e regras continuam vindo **só** do arquivo português. A tradução nunca pode, sem querer, mudar a jogabilidade. Isso é separação de responsabilidades: um arquivo diz **o que acontece**, o outro diz **como se fala**.

CAPÍTULO 14

Som e música gerados por código

Todos os sons do Hispânia, do golpe de espada ao alaúde da trilha sonora, são **sintetizados na hora**, sem nenhum arquivo de áudio. Este capítulo ensina o mínimo de física e de teoria musical necessário e mostra como cada som foi construído.

14.1 O que é som, para um computador

Som é uma vibração do ar. Um alto-falante reproduz som movendo uma membrana para frente e para trás muitas vezes por segundo. Para o computador, um som é uma **lista de números** entre −1 e 1 dizendo a posição da membrana a cada instante: são 44.100 ou 48.000 números por segundo (a **taxa de amostragem**).

Duas propriedades importam:

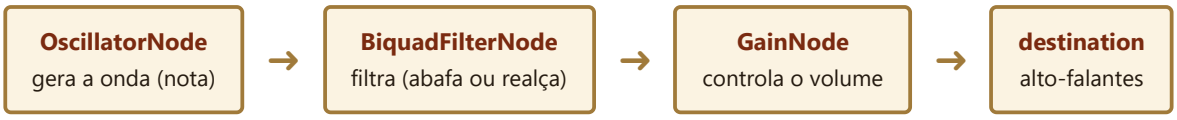
- **Frequência** (quantas vibrações por segundo, em hertz, Hz): define se o som é **grave ou agudo**. O "lá" de afinação é 440 Hz.
- **Amplitude** (o tamanho da vibração): define se o som é **alto ou baixo**.

E a **forma da onda** define o **timbre**, aquilo que faz uma flauta soar diferente de um violino tocando a mesma nota:

Forma	Som	Uso no jogo
`sine` (senoide)	Puro, suave, como um assobio	Flauta, sinos, efeitos de cura
`triangle` (triangular)	Suave, um pouco mais "cheio"	Flauta doce, golpes
`square` (quadrada)	Áspero, típico de videogame antigo	Moedas, nível, tiros
`sawtooth` (dente de serra)	Rico, brilhante, "rasgado"	Charamela, bordão de sanfona
Ruído	Chiado aleatório	Tiros, pandeiro, soalhas

14.2 A Web Audio API: um estúdio modular

A Web Audio API funciona como um estúdio de som com cabos. Você cria **nós** (*nodes*), cada um com uma função, e os conecta em cadeia até a saída (`destination`, os alto-falantes):



Uma característica importantíssima: os nós aceitam **agendamento com precisão de amostra**. Você pode dizer "comece esta nota exatamente daqui a 0,25 segundo" e "abaixe o volume até zero entre 0,25 e 0,4". É isso que permite fazer música com ritmo perfeito.

14.3 Os efeitos sonoros: o objeto Sfx

js/util.js linhas 68–80

```
tone(freq, dur, type, vol, slide) {
  if (!this.enabled || !this.ctx) return;
  const t = this.ctx.currentTime;
  const o = this.ctx.createOscillator();
  const g = this.ctx.createGain();
  o.type = type || 'square';
  o.frequency.setValueAtTime(freq, t);
  if (slide) o.frequency.exponentialRampToValueAtTime(Math.max(20, freq * slide), t + dur);
  g.gain.setValueAtTime(vol || 0.08, t);
  g.gain.exponentialRampToValueAtTime(0.0001, t + dur);
  o.connect(g); g.connect(this.ctx.destination);
  o.start(t); o.stop(t + dur);
},
```

Esta função toca uma nota simples. Linha a linha:

1. Cria um oscilador (`createOscillator`) e um controle de volume (`createGain`).
2. Define a forma de onda e a frequência. Se houver `slide`, a frequência **desliza** exponencialmente até `freq × slide`: é o "piu" descendente de um golpe (`slide` = 0,5) ou ascendente de uma esquiva (2).
3. Define o volume inicial e o faz cair até quase zero no fim. Essa curva de volume se chama **envelope**. Sem ela, o som começaria e pararia "cortado", com um estalo.
4. Conecta oscilador → volume → alto-falantes, inicia agora e para no fim.

O ruído é feito preenchendo um buffer com números aleatórios:

js/util.js linhas 81–93

```
noise(dur, vol) {
  if (!this.enabled || !this.ctx) return;
  const t = this.ctx.currentTime;
  const len = Math.floor(this.ctx.sampleRate * dur);
  const buf = this.ctx.createBuffer(1, len, this.ctx.sampleRate);
  const d = buf.getChannelData(0);
  for (let i = 0; i < len; i++) d[i] = (Math.random() * 2 - 1) * (1 - i / len);
  const s = this.ctx.createBufferSource();
  const g = this.ctx.createGain();
  g.gain.value = vol || 0.1;
  s.buffer = buf; s.connect(g); g.connect(this.ctx.destination);
  s.start(t);
},
```

E `Sfx.play(nome)` combina tons e ruídos para cada evento:

js/util.js linhas 94–112

```
play(name) {
  switch (name) {
    case 'swing': this.tone(300, 0.08, 'triangle', 0.05, 0.5); break;
    case 'shot': this.noise(0.18, 0.18); this.tone(120, 0.15, 'square', 0.06, 0.4); break;
    case 'speech': this.tone(520, 0.07, 'sine', 0.06, 1.5); break;
    case 'flash': this.tone(1200, 0.1, 'sine', 0.05, 0.3); break;
    case 'hit': this.tone(180, 0.07, 'square', 0.06, 0.6); break;
    case 'hurt': this.tone(110, 0.2, 'sawtooth', 0.08, 0.5); break;
```

```
    case 'pickup': this.tone(660, 0.08, 'sine', 0.07, 1.5); setTimeout(() => this.tone(990, 0.1, 'sine',
0.06), 70); break;
    case 'quest': [523, 659, 784, 1047].forEach((f, i) => setTimeout(() => this.tone(f, 0.18, 'triangle',
0.07), i * 110)); break;
    case 'level': [392, 523, 659, 784, 1047].forEach((f, i) => setTimeout(() => this.tone(f, 0.2, 'square',
0.05), i * 90)); break;
    case 'die': this.tone(90, 0.15, 'square', 0.05, 0.5); break;
    case 'click': this.tone(800, 0.03, 'sine', 0.04); break;
    case 'boss': this.tone(80, 0.8, 'sawtooth', 0.1, 0.7); break;
    case 'dash': this.tone(400, 0.1, 'sine', 0.04, 2); break;
    case 'reload': this.tone(250, 0.05, 'square', 0.04); setTimeout(() => this.tone(350, 0.05, 'square',
0.04), 120); break;
    case 'coin': this.tone(1300, 0.06, 'square', 0.03); break;
  }
}
```

O som de "missão concluída", por exemplo, são as notas 523, 659, 784 e 1047 Hz (dó, mi, sol, dó: um acorde maior) tocadas em sequência, 110 ms uma após a outra. É um arpejo alegre feito com 1 linha.

> **🛠️ Mão na massa**
>
> No Console de qualquer página do jogo (depois de um clique), teste: `Sfx.init(); Sfx.tone(440, 1, 'sawtooth', 0.1)`. Troque `'sawtooth'` por `'sine'` e ouça a diferença de timbre.

14.4 Teoria musical mínima

Para entender o motor de música, você precisa de quatro ideias.

1. Notas MIDI. Cada nota do piano recebe um número: o dó central é 60, o dó sustenido é 61, o ré é 62... Somar 12 sobe uma **oitava** (a mesma nota, mais aguda). A conversão para frequência é:

`js/music.js` — linhas 225–225

```
freq(m) { return 440 * Math.pow(2, (m - 69) / 12); },
```

2. Escalas e modos. Uma escala é uma seleção de notas dentro da oitava, descrita por distâncias em semitons a partir da nota base (a **tônica**). A música medieval usa os **modos eclesiásticos** e, na Península Ibérica, também escalas de origem árabe:

`js/music.js` — linhas 7–11

```
const MODES = {
  ionian: [0, 2, 4, 5, 7, 9, 11], dorian: [0, 2, 3, 5, 7, 9, 10], phrygian: [0, 1, 3, 5, 7, 8, 10],
  lydian: [0, 2, 4, 6, 7, 9, 11], mixolydian: [0, 2, 4, 5, 7, 9, 10], aeolian: [0, 2, 3, 5, 7, 8, 10],
  hijaz: [0, 1, 4, 5, 7, 8, 10], penta: [0, 3, 5, 7, 10]
};
```

Modo	Sensação	Onde aparece
Dórico	Melancólico, mas com brilho; muito usado nas Cantigas	Título, visigodos, Cid
Frígio	Sombrio, "espanhol", com o segundo grau grudado na tônica	Iberos, Reis Católicos
Hijaz (frígio dominante)	Exótico, típico da música andalusí e do flamenco	Al-Andalus

Modo	Sensação	Onde aparece
Mixolídio	Festivo	Roma, século XIX
Eólio	Triste (o menor natural)	Século de Ouro, Guerra Civil
Jônio / Lídio	Alegre, luminoso	Transição, século XXI
Pentatônica	Simples, arcaica	Pré-história

3. Ritmo. O motor conta o tempo em **colcheias**. Cada compasso tem 6 colcheias, que podem ser agrupadas em 3 + 3 (balanço ternário, como nas Cantigas) ou 2 + 2 + 2 (como numa valsa).

4. Cadências. É o "fim de frase" musical. A **cadência andaluza** (lá–sol–fá–mi, descendo) é a mais famosa da música espanhola. A **Folía**, uma sequência de acordes portuguesa e espanhola do século XVI, foi usada no capítulo do Século de Ouro.

14.5 Os temas

js/music.js linhas 25–41

```
const THEMES = {
  title: { mode: 'dorian', root: 62, bpm: 84, lead: 'recorder', second: 'lute', drone: true, drum:
'cantiga', seed: 7 },
  c1:  { mode: 'penta', root: 57, bpm: 66, lead: 'flute', second: 'lute', drone: true, drum: 'none', seed:
11, soft: true },
  c2:  { mode: 'phrygian', root: 64, bpm: 100, lead: 'shawm', second: 'lute', drone: true, drum: 'march',
seed: 22 },
  c3:  { mode: 'mixolydian', root: 67, bpm: 92, lead: 'recorder', second: 'lute', drone: false, drum:
'cantiga', seed: 33 },
  c4:  { mode: 'dorian', root: 62, bpm: 70, lead: 'flute', second: 'recorder', drone: true, drum: 'none',
seed: 44 },
  c5:  { mode: 'hijaz', root: 64, bpm: 104, lead: 'lute', second: 'shawm', drone: true, drum: 'darbuka',
seed: 55 },
  c6:  { mode: 'dorian', root: 69, bpm: 110, lead: 'shawm', second: 'recorder', drone: true, drum: 'march',
seed: 66 },
  c7:  { mode: 'phrygian', root: 64, bpm: 96, lead: 'lute', second: 'recorder', drone: false, drum:
'cantiga', seed: 77, andalusian: true },
  c8:  { mode: 'aeolian', root: 62, bpm: 100, lead: 'recorder', second: 'lute', drone: false, drum: 'pulse',
seed: 88, folia: true },
  c9:  { mode: 'dorian', root: 64, bpm: 120, lead: 'lute', second: 'shawm', drone: false, drum: 'jota',
seed: 99 },
  c10: { mode: 'mixolydian', root: 67, bpm: 108, lead: 'lute', second: 'recorder', drone: false, drum:
'jota', seed: 110 },
  c11: { mode: 'aeolian', root: 57, bpm: 64, lead: 'flute', second: 'lute', drone: true, drum: 'none', seed:
121, low: true },
  c12: { mode: 'dorian', root: 62, bpm: 80, lead: 'lute', second: 'flute', drone: false, drum: 'slow', seed:
132 },
  c13: { mode: 'ionian', root: 60, bpm: 100, lead: 'recorder', second: 'lute', drone: false, drum:
'cantiga', seed: 143 },
  c14: { mode: 'lydian', root: 65, bpm: 108, lead: 'bells', second: 'lute', drone: false, drum: 'pulse',
seed: 154 }
};
```

Cada capítulo tem um tema com modo, nota base, andamento (`bpm`, batidas por minuto), instrumento principal, instrumento secundário, se tem **bordão** (uma nota grave contínua, como uma sanfona ou gaita de foles), qual ritmo de percussão e uma semente para compor a melodia.

14.6 A composição automática

js/music.js — linhas 122–180

```
compose(th, variant) {
  const rng = mulberry32(th.seed * 1000 + variant);
  const scale = MODES[th.mode];
  const n = scale.length;
  // temas suaves usam notas longas e espaçadas; os demais, ritmos de dança
  const rhythms = th.soft ? [[3, 3], [6], [4, 2], [2, 4], [3, 3]] : [[3, 3], [2, 1, 2, 1], [2, 1, 3], [1, 1,
1, 3], [3, 2, 1], [2, 2, 2], [1, 1, 1, 1, 2], [4, 2]];
  const phrase = (bars, startDeg, endDeg, hi) => {
    const notes = [];
    let deg = startDeg, t = 0;
    for (let b = 0; b < bars; b++) {
      const last = b === bars - 1;
      const r = last ? (th.soft ? [6] : rng() < 0.5 ? [2, 1, 3] : [1, 1, 1, 3]) : rhythms[Math.floor(rng() *
rhythms.length)];
      r.forEach((dur, i) => {
        const final = last && i === r.length - 1;
        if (final) deg = endDeg;
        else if (last && i === r.length - 2) deg = endDeg + (rng() < 0.5 ? 1 : -1);
        else {
          const x = rng();
          const stepv = x < 0.55 ? 1 : x < 0.8 ? 2 : x < 0.92 ? 3 : 4;
          const dir = deg > (hi ? n + 3 : n) ? -1 : deg < (hi ? 2 : -1) ? 1 : (rng() < 0.5 ? -1 : 1);
          if (rng() > 0.12) deg += dir * (stepv === 1 ? 1 : stepv === 2 ? 1 : stepv - 1);
        }
        const rest = !final && rng() < 0.06;
        if (!rest) notes.push({ t, dur, deg });
        t += dur;
      });
    }
    return { notes, len: bars * 6 };
  };
  const tonic = 0, half = th.mode === 'phrygian' || th.mode === 'hijaz' ? 1 : 4;
  const A = phrase(4, 2, half, false);
  const B = phrase(4, 4, half, true);
  const A2 = { notes: A.notes.slice(), len: A.len };
  // A' termina na tônica
  const lastBar = phrase(1, A.notes.length ? A.notes[A.notes.length - 1].deg : 2, tonic, false);
  A2.notes = A.notes.filter(x => x.t < 18).concat(lastBar.notes.map(x => ({ t: x.t + 18, dur: x.dur, deg:
x.deg })));
  const form = [A, A, B, A2];
  const ev = [];
  let off = 0;
  form.forEach((ph, pi) => {
    for (const x of ph.notes) ev.push({ t: off + x.t, type: 'mel', dur: x.dur, midi: this.degMidi(th, x.deg)
});
    // baixo: alaúde grave nos tempos fortes (folía/andaluza quando definido)
    for (let bar = 0; bar < 4; bar++) {
      const bi = pi * 4 + bar;
      let bdeg = 0;
      if (th.folia) bdeg = FOLIA[bi % 8];
```

```
      else if (th.andalusian) bdeg = [3, 2, 1, 0][bar]; // cadência andaluza: lá-sol-fá-mi em mi frígio
      else bdeg = bar === 3 && pi % 2 === 0 ? 4 : 0;
      const root = this.degMidi(th, bdeg) - 24;
      ev.push({ t: off + bar * 6, type: 'bass', dur: 3, midi: root });
      ev.push({ t: off + bar * 6 + 3, type: 'bass', dur: 3, midi: root + (th.mode === 'penta' ? 0 : 7) });
      for (const [dt, k] of DRUMS[th.drum] || []) ev.push({ t: off + bar * 6 + dt, type: 'drum', k });
    }
    off += ph.len;
  });
  ev.sort((a, b) => a.t - b.t);
  this.events = ev;
  this.pieceLen = off;
},
```

O compositor cria uma música na forma **A A B A'**, muito comum em canções tradicionais:

- **Frase A** (4 compassos): começa no 3º grau do modo e termina numa nota de "suspense" (o 5º grau, ou o 2º nos modos frígios).
- **Frase B**: começa mais aguda, para contrastar.
- **Frase A'**: igual à A, mas o último compasso é refeito para terminar na **tônica**, a nota de "repouso". Isso dá a sensação de "fim".

Cada frase é gerada por um **passeio aleatório controlado** sobre os graus da escala: 55% das vezes a melodia anda um grau (passos são o que soa mais "cantável"), às vezes salta 2 ou 3, e é empurrada de volta quando sobe ou desce demais. Os ritmos de cada compasso são sorteados de uma lista de padrões típicos. Como tudo usa o gerador com semente, a mesma música toca sempre igual para o mesmo tema. A cada duas repetições, `compose` é chamada com uma `variant` nova, e a melodia muda, então a trilha não cansa.

O acompanhamento também é montado ali: o **baixo** toca a tônica e a quinta em cada compasso (ou a sequência da Folía, ou a cadência andaluza), e a **percussão** segue o padrão escolhido.

14.7 O agendador: tocar no tempo certo

Se o jogo tocasse cada nota "na hora" usando `setTimeout`, o ritmo ficaria irregular, porque `setTimeout` pode atrasar alguns milissegundos (o ouvido percebe). A técnica profissional é o **agendamento antecipado** (*lookahead scheduling*):

js/music.js — linhas 188–208

```
tick() {
  const ctx = Sfx.ctx;
  if (!ctx || !this.events) return;
  const ahead = ctx.currentTime + 0.35;
  // se a aba ficou parada, recomeça do ponto atual em vez de "atropelar" notas atrasadas
  if (this.loopStart + this.pieceLen * this.eighth < ctx.currentTime - 1) { this.loopStart = ctx.currentTime
+ 0.1; this.idx = 0; }
  for (let guard = 0; guard < 200; guard++) {
    if (this.idx >= this.events.length) {
      this.loopStart += this.pieceLen * this.eighth;
      this.idx = 0;
      this.loop++;
      if (this.loop % 2 === 0) this.compose(this.th, this.loop / 2);
    }
    const e = this.events[this.idx];
```

```
      const when = this.loopStart + e.t * this.eighth;
      if (when > ahead) break;
      this.idx++;
      if (when < ctx.currentTime - 0.05) continue;
      this.play(e, when);
    }
  },
```

A cada 60 ms, `tick` olha **0,35 segundo para o futuro** e agenda, com hora exata, todas as notas que cairão nesse intervalo. Quem toca é o relógio de áudio do navegador, que é preciso. O JavaScript só precisa "adiantar o trabalho" a tempo. Quando a peça acaba, recomeça (e a cada duas voltas, recompõe).

14.8 Os instrumentos

O alaúde: o algoritmo de Karplus-Strong

js/music.js — linhas 246–268

```
  pluck(midi, when, dur, vol) {
    const ctx = Sfx.ctx;
    let buf = this.ks[midi];
    if (!buf) {
      const sr = ctx.sampleRate, f = this.freq(midi);
      const len = Math.floor(sr * 1.6), period = Math.max(2, Math.round(sr / f));
      buf = ctx.createBuffer(1, len, sr);
      const d = buf.getChannelData(0);
      const rng = mulberry32(midi * 31);
      for (let i = 0; i < period; i++) d[i] = rng() * 2 - 1;
      const decay = midi < 50 ? 0.996 : 0.993;
      for (let i = period; i < len; i++) d[i] = decay * 0.5 * (d[i - period] + d[i - period + 1]);
      this.ks[midi] = buf;
    }
    const s = ctx.createBufferSource(), g = ctx.createGain(), lp = ctx.createBiquadFilter();
    s.buffer = buf;
    lp.type = 'lowpass'; lp.frequency.value = 2600;
    const len = Math.min(1.6, dur + 0.9);
    g.gain.setValueAtTime(0.32 * vol, when);
    g.gain.exponentialRampToValueAtTime(0.0001, when + len);
    s.connect(lp); lp.connect(g); g.connect(this.bus);
    s.start(when); s.stop(when + len + 0.05);
  },
```

Este é um dos algoritmos mais elegantes da síntese sonora (Karplus e Strong, 1983). Para simular uma corda dedilhada:

1. Encha um pequeno trecho de memória com **ruído aleatório**. O tamanho desse trecho é o **período** da nota: `taxa de amostragem / frequência` amostras.
2. Gere o resto do som repetindo esse trecho, mas cada amostra nova é a **média de duas amostras de um período atrás**, multiplicada por um pouco menos de 1 (`decay`).

A média funciona como um filtro que vai tirando os agudos a cada volta, exatamente como uma corda real que perde o brilho depois de tocada, e o `decay` faz o som morrer aos poucos. O resultado, a partir de puro ruído, é um som surpreendentemente parecido com um alaúde, um oud ou uma vihuela. Cada nota é calculada uma vez e guardada em cache (`this.ks[midi]`).

Os sopros: flauta, flauta doce e charamela

js/music.js linhas 271–299

```
wind(inst, midi, when, dur, vol) {
  const ctx = Sfx.ctx;
  const f = this.freq(midi);
  const o = ctx.createOscillator(), g = ctx.createGain(), flt = ctx.createBiquadFilter();
  const lfo = ctx.createOscillator(), lfoG = ctx.createGain();
  let peak;
  if (inst === 'shawm') { o.type = 'sawtooth'; flt.type = 'bandpass'; flt.frequency.value = 1300;
flt.Q.value = 1.2; peak = 0.11; }
  else if (inst === 'flute') { o.type = 'sine'; flt.type = 'lowpass'; flt.frequency.value = 3000; peak =
0.09; }
  else { o.type = 'triangle'; flt.type = 'lowpass'; flt.frequency.value = 2400; peak = 0.075; }
  o.frequency.value = f;
  const soft = this.th && this.th.soft;
  lfo.frequency.value = soft ? 4.2 : 5.2; lfoG.gain.value = f * (soft ? 0.003 : 0.006);
  if (soft) { peak *= 0.75; flt.frequency.value = 1800; }
  lfo.connect(lfoG); lfoG.connect(o.frequency);
  const att = soft ? 0.18 : inst === 'shawm' ? 0.03 : 0.05, end = when + Math.max(0.12, dur * (soft ? 1 :
0.92));
  g.gain.setValueAtTime(0.0001, when);
  g.gain.exponentialRampToValueAtTime(peak * vol, when + att);
  g.gain.setValueAtTime(peak * vol * 0.85, end - 0.04);
  g.gain.exponentialRampToValueAtTime(0.0001, end + 0.08);
  o.connect(flt); flt.connect(g); g.connect(this.bus);
  // sopro (ruído de ar) leve na flauta
  if (inst === 'flute') {
    const n = ctx.createBufferSource(), ng = ctx.createGain(), nf = ctx.createBiquadFilter();
    n.buffer = this.noiseBuf; nf.type = 'bandpass'; nf.frequency.value = f * 2; nf.Q.value = 3;
    ng.gain.setValueAtTime((soft ? 0.004 : 0.012) * vol, when); ng.gain.exponentialRampToValueAtTime(0.0001,
end + 0.05);
    n.connect(nf); nf.connect(ng); ng.connect(this.bus); n.start(when); n.stop(end + 0.1);
  }
  o.start(when); lfo.start(when); o.stop(end + 0.12); lfo.stop(end + 0.12);
},
```

Um oscilador com a forma certa para cada instrumento, um filtro para moldar o timbre (a **charamela**, antepassada do oboé, usa dente de serra com um filtro passa-banda que realça os médios e lhe dá o som anasalado) e um **vibrato**: um segundo oscilador, muito lento (5,2 Hz), que faz a frequência da nota oscilar levemente, como um músico de verdade faz. A flauta ganha ainda um pouco de ruído filtrado, imitando o "ar" do sopro.

O bordão (drone)

Três osciladores dente de serra graves (a tônica duas vezes, levemente desafinadas entre si, e a quinta) passam por um filtro que abafa os agudos. A leve desafinação cria um "batimento" que dá vida ao som, como numa sanfona (*zanfona*) ou gaita de foles galega.

Percussão

O tambor grave (pandeiro, adufe, darbuka) é uma senoide que **despenca** de 130 para 55 Hz em 0,18 s: o ouvido percebe isso como uma batida. As batidas agudas e as soalhas são rajadas de ruído passadas por filtros em frequências diferentes.

O "eco" da sala

Na criação do mixer (`ensure`), um nó de atraso (`DelayNode`) de 0,23 s com realimentação de 22% simula a reverberação de uma igreja ou de um salão de pedra, dando ambiente à trilha.

14.9 Como testar áudio sem ouvir

Durante o desenvolvimento, não era possível ouvir o som. A solução foi usar o `OfflineAudioContext`, que **renderiza o áudio na memória** em vez de tocar. Cada um dos 15 temas foi renderizado por 12 segundos e analisado: o **pico** (maior valor, para detectar distorção acima de 1,0), o **RMS** (a média do volume, para detectar silêncio) e a presença de valores inválidos (`NaN`). Todos ficaram com picos entre 0,12 e 0,30 e RMS entre 0,017 e 0,037: audíveis, sem distorção e num volume de fundo adequado. Medir em vez de "achar" é uma habilidade valiosa.

> ✍️ **Exercício**
>
> Em `js/music.js`, no tema `c1`, troque `bpm: 66` por `bpm: 120` e `lead: 'flute'` por `lead: 'shawm'`. Recarregue e entre no capítulo 1. A música da pré-história vira uma marcha agitada.

CAPÍTULO 15

Testar, depurar e publicar

Escrever código é metade do trabalho. A outra metade é descobrir **por que ele não faz o que você esperava**, e garantir que continue funcionando quando você mudar alguma coisa. Este capítulo mostra as técnicas usadas no Hispânia e como colocar o jogo na internet.

15.1 Depurando com o DevTools

Quando algo dá errado:

1. **Abra o Console (F12).** Erros aparecem em vermelho, com o arquivo e a linha. Clique no link para ir direto ao código. Leia a mensagem com calma; ela quase sempre diz exatamente o problema.
2. **Use** `console.log` para ver valores em pontos-chave: `console.log('vida', G.S.hp, 'inimigos', G.enemies.length)`.
3. **Inspecione o estado ao vivo.** Digite `G.player`, `G.enemies[0]` ou `qState('main')` no Console para ver os objetos.
4. **Pause o programa (breakpoints).** Na aba *Sources*, abra um arquivo e clique no número de uma linha. Quando o jogo passar por ali, ele congela, e você pode ver o valor de cada variável e avançar linha por linha.

As mensagens de erro mais comuns e o que significam:

Mensagem	Significado provável
`X is not defined`	Nome digitado errado, ou o script que cria `X` não carregou (ordem dos `<script>`)
`Cannot read properties of undefined (reading 'y')`	Você fez `algo.y`, mas `algo` não existe (ex.: um NPC com id errado)
`Identifier 'X' has already been declared`	Dois arquivos criaram o mesmo nome global
`Unexpected token`	Erro de digitação: vírgula, parêntese ou chave faltando ou sobrando
`X is not a function`	Você chamou `X()`, mas `X` não é uma função (ou ainda não foi criada)

> 💡 **Dica**
>
> Erros de sintaxe ("Unexpected token") num arquivo de dados são os mais frequentes ao editar missões. Quase sempre é uma **vírgula esquecida** entre dois objetos de uma lista, ou uma aspa que não fechou. O VS Code sublinha esses erros em vermelho antes mesmo de você rodar.

15.2 Os testes automáticos usados no projeto

Testar 5 horas de jogo à mão a cada mudança é impossível. Por isso foram escritos **programas que testam o programa**. Eles não fazem parte do jogo publicado, mas vale conhecer as ideias.

1. Verificação de sintaxe

```
node --check js/game.js
```

O Node lê o arquivo e acusa erros de sintaxe sem executá-lo. Foi rodado em todos os arquivos depois de cada mudança.

2. Validação dos dados

Um script percorreu os 14 capítulos e conferiu, para cada etapa de cada missão, se:

- o NPC citado existe naquele capítulo;
- o lugar citado (`at`) existe;
- o tipo de inimigo existe em `ENEMIES`;
- a resposta certa de cada quiz aponta para uma opção que existe;
- a primeira etapa das missões secundárias é `talk`;
- **todos os lugares e NPCs são alcançáveis a pé** no mapa gerado (usando o `reach` da busca em largura).

Resultado: 14 capítulos, 92 crônicas, 66 missões, 296 etapas, **zero erros**. Um script parecido conferiu que a tradução espanhola tem exatamente a mesma estrutura da portuguesa (mesmas etapas, mesmos falantes, mesmas respostas de quiz e todas as chaves de interface).

3. O "robô jogador"

O teste mais poderoso foi um robô que **joga o jogo inteiro**. A ideia, simplificada:

```
while (o jogo não acabou) {
  const missao = primeira missão ativa (secundárias antes da principal);
  const etapa = curStep(missao);
  if (etapa.t === 'talk' || etapa.t === 'quiz') {
    teleporta para perto do NPC; talkTo(npc); responde o quiz corretamente;
  } else if (etapa.t === 'kill') {
    teleporta ao acampamento; mata um inimigo do tipo certo (ou espera o respawn);
  } else if (etapa.t === 'collect') {
    pega um dos itens da missão;
  } else if (etapa.t === 'goto') {
    teleporta para o lugar e roda alguns quadros de update;
  } else if (etapa.t === 'boss') {
    vai à arena, espera o chefe surgir e o derrota;
  }
  fecha todos os diálogos, narrações e telas de evolução;
}
```

Ele completou as **66 missões dos 14 capítulos** até a tela final, em português e depois em espanhol, sem nenhum erro. No espanhol, ainda guardou o texto de todos os 327 diálogos exibidos e procurou palavras típicas do português que tivessem escapado da tradução: nenhuma.

> ❓ **Por quê?**
>
> Um teste assim não diz se o jogo é **divertido** (isso só jogando), mas garante que ele é **completável**: nenhuma missão trava, nenhum NPC sumiu, nenhum chefe deixa de aparecer. É exatamente o tipo de erro mais frustrante para o jogador e mais difícil de achar à mão.

4. Testes de áudio offline

Como vimos no capítulo 14, a música foi renderizada na memória e medida (pico, volume médio, valores inválidos).

15.3 Um servidor local mínimo

Abrir o `index.html` com dois cliques funciona, mas para testar em condições parecidas com a internet foi usado um servidor de 15 linhas, só com módulos que já vêm no Node:

.claude/serve.js — linhas 1–15

```
// Servidor estático mínimo para testar localmente (não é necessário no GitHub Pages).
const http = require('http'), fs = require('fs'), path = require('path');
const root = path.resolve(__dirname, '..');
const types = { '.html': 'text/html; charset=utf-8', '.js': 'text/javascript; charset=utf-8', '.css':
'text/css; charset=utf-8', '.json': 'application/json', '.png': 'image/png' };
http.createServer((req, res) => {
  let p = decodeURIComponent(req.url.split('?')[0]);
  if (p.endsWith('/')) p += 'index.html';
  const f = path.join(root, p);
  if (!f.startsWith(root)) { res.writeHead(403); return res.end(); }
  fs.readFile(f, (err, data) => {
    if (err) { res.writeHead(404); return res.end('404'); }
    res.writeHead(200, { 'Content-Type': types[path.extname(f)] || 'application/octet-stream', 'Cache-
Control': 'no-store' });
    res.end(data);
  });
}).listen(process.env.PORT || 5173, () => console.log('http://localhost:' + (process.env.PORT || 5173)));
```

Ele recebe cada pedido do navegador, encontra o arquivo correspondente na pasta e o devolve com o tipo certo (`text/html`, `text/javascript`...). O `Cache-Control: no-store` impede o navegador de guardar versões antigas, então cada F5 carrega o código mais novo. A verificação `f.startsWith(root)` impede que alguém peça arquivos de fora da pasta do projeto (`../../senhas.txt`), uma proteção básica de segurança.

Para usar: `node .claude/serve.js` e abra `http://localhost:5173`.

15.4 Git: guardando o histórico

O **Git** tira "fotos" (commits) do seu projeto ao longo do tempo. Se algo quebrar, você volta para a última foto boa. Os comandos essenciais:

```
git init                        # transforma a pasta num repositório (uma vez só)
git add .                       # prepara todas as mudanças para a próxima foto
git commit -m "Adiciona missão dos lobos"   # tira a foto, com uma descrição
```

```
git log --oneline              # lista as fotos tiradas
git diff                       # mostra o que mudou desde a última foto
```

> 💡 **Dica**
>
> Faça commits pequenos e frequentes, com mensagens que expliquem **o que** mudou. "Corrige respawn dos acampamentos" é muito mais útil daqui a seis meses do que "ajustes".

15.5 Publicando no GitHub Pages

O **GitHub Pages** hospeda sites estáticos de graça. Como o Hispânia é só HTML, CSS e JavaScript, ele roda lá sem nenhuma adaptação.

1. Crie uma conta em `github.com`.
2. Clique em **New repository**, dê um nome (por exemplo, `hispania`), deixe como **Public** e crie.
3. Na pasta do jogo, no terminal:

```
git init
git add .
git commit -m "Hispânia"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/hispania.git
git push -u origin main
```

1. No site do GitHub, no repositório: **Settings** → **Pages** → **Build and deployment** → **Source: Deploy from a branch**, escolha a branch `main` e a pasta `/ (root)`, e salve.
2. Em um ou dois minutos o jogo estará em `https://SEU_USUARIO.github.io/hispania/`.

Para publicar uma atualização depois, basta `git add .`, `git commit -m "..."` e `git push`. O site se atualiza sozinho.

> ⚠️ **Atenção**
>
> Os saves ficam no navegador de cada jogador, **separados por endereço**. Um save feito jogando em `localhost` não aparece na versão do GitHub Pages, e vice-versa. Para levar seu progresso, use **Exportar save** em um e **Importar save** no outro.

> 📌 **Nota**
>
> As pastas `guia/` e `.claude/` não atrapalham o site, mas, se preferir não publicá-las, crie um arquivo `.gitignore` na raiz com as linhas `guia/` e `.claude/`.

CAPÍTULO 16

Mãos à obra: estendendo o jogo

Agora você conhece todas as peças. Este capítulo traz **tutoriais completos**, do mais fácil ao mais difícil, para você modificar o Hispânia sozinho. Faça-os na ordem, testando cada um. É aqui que o aprendizado vira habilidade.

> 💡 **Dica**
>
> Antes de começar, faça uma cópia da pasta `game` (ou um commit no Git). Assim, se algo der errado, você volta ao ponto em que funcionava.

16.1 Tutorial 1: um novo inimigo

Vamos criar uma **Raposa**, rápida e fraca, para o capítulo 1.

Passo 1. Em `js/data/base.js`, dentro de `ENEMIES`, logo abaixo do `lobo`, adicione:

```
raposa: { name: 'Raposa', hp: 14, spd: 170, dmg: 4, ai: 'swarm', r: 10, xp: 6, gold: 2, shape: 'beast',
color: '#c8641e' },
```

Ela usa a IA de enxame (persegue e morde por contato), é desenhada como animal (`beast`) num tom laranja e é menor (`r: 10`).

Passo 2. Coloque um acampamento de raposas no mapa. Em `js/data/chapters1.js`, no capítulo 1, troque a linha `camps` por:

```
camps: [{ at: 'bosque', type: 'javali', n: 2 }, { at: 'dolmen', type: 'raposa', n: 4 }],
```

Passo 3. (Opcional) Tradução: em `js/data/es_base.js`, dentro de `ES_BASE.enemies`, adicione `raposa: 'Zorro',`.

Recarregue e vá até o Dólmen dos Antigos. As raposas estarão lá e reaparecerão quando derrotadas.

> ✍️ **Exercício**
>
> Crie também um **Corvo** que ataca à distância (`ai: 'ranged'`, `shape: 'blob'`, `color: '#222'`, `proj: '#555'`) e coloque-o na Gruta de Altamira. Observe como ele mantém distância.

16.2 Tutorial 2: uma nova missão com quiz

Vamos dar ao Pequeno Txo um quiz sobre o próprio jogo. Em `js/data/chapters1.js`, na lista `quests` do capítulo 1, adicione:

```
{ id: 's6', title: 'As Perguntas de Txo', req: 2, reward: { xp: 40, gold: 15 }, steps: [
  { t: 'talk', npc: 'txo', desc: 'Fale com o Pequeno Txo', say: [['txo', 'Aposto que você não sabe responder
```

```
isto!']] },
  { t: 'quiz', npc: 'txo', desc: 'Responda ao Pequeno Txo', qs: [
    { q: 'Qual animal tem chifres enormes?', o: ['Lobo', 'Auroque', 'Javali'], a: 1 },
    { q: 'Onde ficam as pinturas de bisontes?', o: ['Atapuerca', 'Altamira'], a: 1 }
  ] }
] }
```

Lembre: `a` é o **índice** da resposta certa, contando a partir de 0. `req: 2` libera a missão a partir da etapa 2 da história principal.

16.3 Tutorial 3: uma nova arma

Vamos dar ao caçador um **arco** no lugar da lança.

Passo 1. Em `WEAPONS`, adicione:

```
arco: { name: 'Arco de caça', kind: 'gun', dmg: 14, cd: 0.45, mag: 6, reload: 1.0, speed: 600, spread: 0.05,
draw: 'bow', color: '#8a6a3a' },
```

Usamos o tipo `gun` porque um arco funciona como uma arma de projéteis com munição (as flechas da aljava). O desenho `bow` já existe em `drawWeapon`.

Passo 2. Em `CLASSES`, no `cacador`, troque `weapon: 'lanca'` por `weapon: 'arco'`.

Recarregue. O caçador agora atira flechas, com 6 disparos antes de recarregar. Experimente mudar `dmg`, `cd` e `speed` para sentir como cada número muda o jogo.

> ⚠️ **Atenção**
>
> Repare que a cor da "bala" está fixa como amarela (`'#ffe080'`) em `playerAttack`. Desafio: faça a cor do projétil vir da arma, adicionando um campo `projColor` em `WEAPONS` e usando `w.projColor || '#ffe080'` no código.

16.4 Tutorial 4: um novo tipo de chapéu

Em `js/entities.js`, na função `drawHat`, adicione um novo `case` antes do fechamento do `switch`:

```
case 'crown':   // coroa
  ctx.fillRect(-8, -30, 16, 4);
  for (let i = -8; i <= 4; i += 6) {
    ctx.beginPath(); ctx.moveTo(i, -30); ctx.lineTo(i + 2, -36); ctx.lineTo(i + 4, -30); ctx.fill();
  }
  break;
```

Depois use `hat: 'crown', hatColor: '#d4af37'` na aparência de algum rei, como o `leovi` do capítulo 4. Desenhe no papel antes: lembre que y negativo é **para cima** e que a cabeça fica por volta de y = −21.

16.5 Tutorial 5: um novo tema musical

Em `js/music.js`, os temas ficam em `THEMES`. Para dar ao capítulo 1 uma música diferente, mude o tema `c1`:

```
c1: { mode: 'dorian', root: 62, bpm: 90, lead: 'recorder', second: 'lute', drone: true, drum: 'cantiga',
seed: 2024 },
```

Mudar só a `seed` compõe uma melodia nova com o mesmo estilo. Mudar `mode` altera o "clima". Experimente `hijaz` com `drum: 'darbuka'` para um som andalusí.

16.6 Tutorial 6: um capítulo inteiro

Este é o grande projeto. Um capítulo novo precisa de:

1. Um objeto no formato do capítulo 10 (`CHAPTERS.push({...})`), com `id` e `num` novos (`'c15'`, `15`).
2. Uma `cls` existente (ou uma classe nova em `CLASSES`).
3. Uma `seed` qualquer e um `biome` (copie de um capítulo parecido).
4. De 5 a 8 `places`. **O primeiro é o ponto de partida.** Espalhe-os pelo mapa (x de 10 a 100, y de 10 a 70) e inclua pelo menos um `camp` e uma `arena`.
5. NPCs com `at` apontando para lugares que existem, incluindo um comerciante (`shop: true`).
6. Pelo menos uma crônica em `codex`.
7. Uma missão `main` e algumas secundárias.
8. Um tema musical com a chave igual ao `id` (`c15`) em `THEMES`.

Coloque o capítulo num arquivo novo, `js/data/chapters4.js`, e adicione `<script src="js/data/chapters4.js"></script>` no `index.html`, **depois** de `chapters3.js`. Os capítulos são ordenados pelo campo `num`, então o seu capítulo será o 15º. Ele aparecerá depois do final atual. Que tal um capítulo sobre o futuro da Espanha em 2050?

> 💡 **Dica**
>
> Se o jogo não carregar depois de adicionar o capítulo, abra o Console (F12). O erro mais provável é uma vírgula faltando. O segundo mais provável é um `npc` ou `at` com um id escrito diferente do que foi definido.

16.7 Tutorial 7: uma mecânica nova (desafio)

Implemente uma **mochila de pedras**: no capítulo 1, cada pedra de sílex coletada aumenta 2% o dano da lança. Pistas:

1. Adicione um campo `silex: 0` em `newState`.
2. Em `pickItem`, se o nome do item for `'Sílex'`, faça `G.S.silex++`.
3. Em `playerDmg`, multiplique o resultado por `(1 + 0.02 * (G.S.silex || 0))`.
4. Mostre o bônus no HUD.

Note o `|| 0` no passo 3: saves antigos não têm o campo `silex`, e `undefined * 0.02` daria `NaN` (*Not a Number*), quebrando o dano. Pensar em dados antigos é uma marca de bom programador.

16.8 Para onde ir depois deste guia

Você aprendeu os fundamentos que servem para **qualquer** linguagem e qualquer tipo de jogo: variáveis, condições, laços, funções, objetos, eventos, o laço do jogo, colisão, IA por estados, dados separados da lógica, salvamento e testes. Sugestões de próximos passos:

1. **Faça um jogo pequeno do zero**, sozinho, sem copiar: um Pong, uma cobrinha (Snake) ou um Flappy Bird. Cada um cabe em 200 linhas e usa só o que você aprendeu aqui.
2. **Aprenda módulos ES** (`import` / `export`) e a usar um servidor local como o `serve.js`.
3. **Experimente um framework**, como o Phaser. Agora você vai entender o que ele faz por você.
4. **Estude estruturas de dados e algoritmos**: filas, pilhas, grafos e o algoritmo A* (busca de caminho), que faria os inimigos contornarem obstáculos em vez de andar em linha reta.
5. **Leia código dos outros**: projetos de jogos de código aberto no GitHub.
6. **Documentação de referência**: o site **MDN Web Docs** (`developer.mozilla.org`, com versão em português) é a melhor referência de HTML, CSS, JavaScript, Canvas e Web Audio que existe.

CAPÍTULO 17

Referência completa: cada função do jogo

Este capítulo é para consulta. Ele lista **todas as funções e métodos** do jogo, arquivo por arquivo, com os parâmetros e o que cada um faz. Quando estiver lendo o código e encontrar um nome desconhecido, procure aqui.

17.1 js/util.js: utilidades

Função	O que faz
`TILE`	Constante: tamanho de um tile em pixels (32)
`mulberry32(seed)`	Cria um gerador de números pseudoaleatórios com semente; devolve uma função que, a cada chamada, dá o próximo número entre 0 e 1
`hash2(x, y)`	Número "aleatório" fixo (0 a 1) para uma coordenada; usado para variar detalhes visuais
`makeNoise(rng, w, h, cell)`	Cria uma função de ruído suave (value noise) com células de tamanho `cell`
`clamp(v, a, b)`	Limita `v` ao intervalo de `a` a `b`
`dist(ax, ay, bx, by)`	Distância entre dois pontos
`lerp(a, b, t)`	Interpolação linear: valor entre `a` e `b` na proporção `t`
`randRange(a, b)` / `randInt(a, b)`	Número aleatório real / inteiro entre `a` e `b`
`angleDiff(a, b)`	Menor diferença entre dois ângulos, de −π a π
`shade(hex, amt)`	Clareia (amt positivo) ou escurece (negativo) uma cor hexadecimal
`escapeHtml(s)`	Torna um texto seguro para `innerHTML` (protege contra XSS)
`Sfx.init()`	Cria o contexto de áudio (só depois de um clique do usuário)
`Sfx.tone(freq, dur, type, vol, slide)`	Toca uma nota com envelope e deslize opcional de frequência
`Sfx.noise(dur, vol)`	Toca uma rajada de ruído
`Sfx.play(name)`	Toca um efeito pelo nome (`'swing'`, `'shot'`, `'hit'`, `'quest'`, `'level'`…)

17.2 js/i18n.js: idiomas

Função	O que faz
`LANG`	Idioma atual (`'pt'` ou `'es'`), lido do localStorage ou do navegador
`setLang(l)`	Grava o idioma escolhido e recarrega a página

Função	O que faz
`STR`	Dicionário de textos da interface em PT e ES
`T(key, ...args)`	Devolve o texto da chave no idioma atual, trocando `{0}`, `{1}` … pelos argumentos
`ES_CHAPTERS`, `ES_BASE`	Recipientes das traduções do conteúdo (preenchidos pelos arquivos `es*.js`)
`applyLanguage()`	Se o idioma for espanhol, copia as traduções por cima dos dados portugueses

17.3 js/world.js: o mapa

Função / método	O que faz
`TL`, `WALKABLE`, `TALL`	Tipos de tile e as tabelas "dá para andar?" e "bloqueia tiros?"
`new World(ch)`	Cria e gera o mapa de um capítulo
`idx(x, y)` / `get(x, y)` / `set(x, y, v)`	Índice no array, ler e escrever o tipo de um tile (fora do mapa conta como obstáculo)
`generate()`	Gera o mapa inteiro: terreno, água, obstáculos, bordas, lugares, casas, estradas e alcance
`carvePath(x0, y0, x1, y1, rng)`	Cava uma estrada sinuosa entre dois pontos, criando pontes sobre a água
`computeReach()`	Busca em largura que marca todos os tiles alcançáveis a partir do início
`randomSpot(px, py, rTiles, minR)`	Ponto aleatório alcançável dentro de um raio (em tiles) de um ponto
`randomReachableFar(rng, places, minDist)`	Ponto alcançável longe de todos os lugares (usado para os baús)
`solidAt(px, py)` / `tallAt(px, py)`	O pixel está em tile bloqueado? / em tile alto?
`collides(x, y, r)`	Um círculo de raio `r` nessa posição bate em algo? (testa 4 cantos)
`chunk(cx, cy)`	Devolve (criando na primeira vez) o canvas em cache de um bloco de 16×16 tiles
`drawTile(g, tx, ty, X, Y)`	Desenha um tile num contexto
`drawObstacle(g, kind, X, Y, h)`	Desenha um obstáculo (árvore, pinheiro, rocha, coluna, carro…)
`draw(ctx, camX, camY, vw, vh)`	Desenha os blocos visíveis do mapa na tela
`minimap()`	Cria (uma vez) a imagem do minimapa, 1 pixel por tile
`drawLandmark(ctx, icon, x, y, t)`	Desenha o marco de um lugar (caverna, castelo, aqueduto, navio…)

17.4 js/entities.js: desenho e entidades

Função / classe	O que faz
`drawHat(ctx, hat, color, dir)`	Desenha um dos 17 tipos de chapéu ou elmo

Função / classe	O que faz
`drawWeapon(ctx, kind, color, swing)`	Desenha uma das 15 armas, já girada na direção da mira
`drawHuman(ctx, x, y, look, aim, t, o)`	Desenha um humano completo a partir de um objeto `look`; `o` traz opções (andando, escala, transparência, montado, golpe)
`drawBeast(ctx, x, y, d, face, t, o)`	Desenha um animal de quatro patas (com chifres ou presas, se houver)
`drawBlob(ctx, x, y, d, aim, t, o)`	Desenha criaturas simbólicas: sombras, bolhas, vírus, bots
`drawTower(ctx, x, y, d, aim, t, o)`	Desenha a torre do Bastião de Málaga com o canhão
`class Player`	O herói: posição, mira, cronômetros de ataque, esquiva, invulnerabilidade e recarga
`class Enemy`	Um inimigo criado a partir da ficha de `ENEMIES`, com vida e dano multiplicados pela dificuldade
`class Proj`	Um projétil: posição, velocidade, dano, dono (`'p'` jogador ou `'e'` inimigo), cor, raio e texto

17.5 js/game.js: o núcleo

Estado e fórmulas

Função	O que faz
`G`	Objeto com todo o estado do jogo em execução
`resize()`	Ajusta o canvas ao tamanho da janela e à densidade de pixels
`newState(name)`	Cria um save novo (nível 1, 3 poções...)
`chState(id)`	Registro de progresso de um capítulo (criado se não existir)
`qState(qid)`	Registro de progresso de uma missão no capítulo atual: `step`, `prog`, `done`
`mainStep()`	Etapa atual da missão principal (9999 se concluída)
`questActive(quest)`	A missão está disponível agora? (não concluída e com o requisito atendido)
`curStep(quest)`	A etapa atual da missão
`xpNeed(l)`	XP necessário para passar do nível `l`
`maxHp()`	Vida máxima (depende do nível e do capítulo)
`weapon()`	A arma do herói no capítulo atual
`playerDmg()` / `playerDef()`	Dano e defesa do herói, com todos os bônus
`fmt(s)`	Troca `{nome}` pelo nome da linhagem

Salvamento

Função	O que faz
`SLOT_COUNT`, `slotKey(n)`	Número de espaços (5) e a chave do localStorage de cada um
`migrateOldSave()`	(Executada uma vez) move o save da versão antiga para o espaço 1
`save()`	Grava o estado atual no espaço da partida
`loadSave(n)`	Lê o save do espaço `n` (ou `null`)
`deleteSave(n)`	Apaga o save do espaço `n`
`exportSave()`	Baixa o save atual como arquivo `.json`
`importSave(file, n, cb)`	Lê um arquivo `.json`, valida e grava no espaço `n`

Capítulos e missões

Função	O que faz
`loadChapter(idx, keepPos)`	Monta um capítulo: mapa, NPCs, crônicas, baús, acampamentos, inimigos e itens
`spawnCampEnemy(camp, awayFromPlayer)`	Cria um inimigo num acampamento (longe do herói, se pedido)
`spawnItems(quest, step, n)`	Espalha `n` itens de coleta perto do lugar da etapa
`advance(quest)`	Conclui a etapa atual e passa para a próxima (ou conclui a missão)
`startStep(quest)`	Prepara a nova etapa (itens de coleta) e avisa o objetivo
`completeQuest(quest)`	Dá as recompensas; se for a principal, encerra o capítulo
`chapterComplete()`	Narração final, evolução da linhagem e próximo capítulo (ou o final)
`enterChapter(idx, keepPos)`	Entra num capítulo: carrega, troca a música, mostra a abertura na primeira vez
`questTarget(quest)`	Ponto do mundo para onde a seta do objetivo aponta
`onKill(type)`	Conta a morte de um inimigo nas missões de caça ativas
`checkGotoAndBoss()`	Verifica a chegada a lugares de missão e faz surgir os chefes
`talkTo(npc)`	Conversa: etapa de missão, quiz, loja ou fala solta
`runQuiz(quest, s, npc)`	Conduz as perguntas de um quiz
`pickItem(o)`	Coleta um item de missão
`interact(o)`	Interage com um objeto (NPC, item, crônica, baú) se estiver perto
`nearestInteractable()`	O objeto interativo mais próximo do herói (tecla E)

Progressão, combate e efeitos

Função	O que faz
`gainXp(n)`	Soma experiência e sobe de nível quantas vezes for preciso
`usePotion()`	Bebe uma poção (recupera 50% da vida)
`particle(x, y, color, spd, life, size)`	Cria uma partícula
`floatText(x, y, text, color, life)`	Cria um texto flutuante (dano, "+1 poção")
`hurtPlayer(dmg, fromX, fromY)`	Aplica dano ao herói, com defesa, invulnerabilidade e empurrão
`playerDie()`	Queda do herói: perde 10% do ouro e volta ao início do capítulo
`damageEnemy(e, dmg, kx, ky)`	Aplica dano a um inimigo, com número, partículas e empurrão
`killEnemy(e)`	Morte do inimigo: XP, moedas, poção, missões e chefe
`moveEnt(ent, mx, my)`	Move uma entidade com colisão, um eixo de cada vez
`playerAttack()`	Ataque do herói (corpo a corpo, tiro ou palavra)
`startReload()`	Começa a recarregar a arma de fogo
`update(dt)`	Um quadro de lógica (veja o capítulo 8)
`updateEnemy(e, dt)`	IA dos inimigos comuns: ocioso/alerta e os 4 comportamentos
`updateBoss(e, dt, d, dx, dy)`	IA dos chefes: padrões de ataque e fase 2

Desenho e laço

Função	O que faz
`render()`	Desenha o quadro inteiro, na ordem certa
`drawPlayer()`	Desenha o herói, o arco do golpe e a barra de recarga
`npcMarker(npc)`	Cor do "!" sobre o NPC (dourado: principal; azul: secundária)
`drawNpc(n)`	Desenha um NPC com nome e marcador
`drawEnemy(e)`	Desenha um inimigo com o aviso de ataque e a barra de vida
`drawObjective(t, color)`	Seta e marcador do objetivo atual
`drawBossBar()`	Barra de vida do chefe
`drawMinimap()`	Minimapa no canto, com lugares, objetivo e herói
`drawCrosshair()`	A mira no lugar do cursor
`frame(now)`	O laço principal, chamado ~60 vezes por segundo
`startGame(state, slot)`	Inicia uma partida com um save e um espaço

17.6 js/ui.js: a interface

Método	O que faz
`$(id)`	Atalho para `document.getElementById`
`UI.hud(force)`	Atualiza barras, textos e o rastreador de missões
`UI.toast(msg, kind)`	Mostra uma notificação temporária
`UI.speaker(who)`	Nome e estilo de quem fala (`'N'` narrador, `'H'` herói ou um NPC)
`UI.dialog(lines, cb)`	Mostra uma sequência de falas e chama `cb` no fim
`UI.showLine()` / `UI.nextLine()`	Mostra a fala atual / avança para a próxima
`UI.quiz(name, question, options, cb)`	Mostra uma pergunta com opções
`UI.answer(i)`	Registra a resposta escolhida
`UI.modal(html, cls, onClose)` / `UI.closeModal()`	Abre / fecha uma janela central
`UI.narration(title, pages, cb, subtitle)`	Narração em páginas (abertura e encerramento dos capítulos)
`UI.evolution(from, to, ch, cb)`	Tela de evolução da linhagem entre capítulos
`UI.codexFound(def)`	Mostra uma crônica recém-encontrada
`UI.shop(npc)`	Loja: poções e melhorias
`UI.journal()`	Diário de missões (J)
`UI.codex(sel)`	Livro de crônicas por capítulo (C)
`UI.map()`	Mapa grande do capítulo (M)
`UI.langButtons()` / `UI.bindLang(root, before)`	Botões de idioma e sua ligação
`UI.pause()`	Menu de pausa (Esc)
`UI.help(back)`	Tela "Como jogar"
`UI.death(lost, cb)`	Tela de queda do herói
`UI.ending()`	Tela final do jogo, com estatísticas
`UI.title()`	Tela de título com os 5 espaços de save
`UI.titleToast(msg)`	Mensagem curta na tela de título
`UI.onKey(e)`	Encaminha teclas para diálogos e janelas
`UI.staticTexts()`	Traduz os textos fixos do HTML

17.7 js/music.js: a música

Item	O que faz
`MODES`, `DRUMS`, `THEMES`, `FOLIA`	Escalas, ritmos de percussão, temas de cada capítulo e o encadeamento da Folía
`Music.setEnabled(on)`	Liga ou desliga a música (e lembra a escolha)
`Music.ensure()`	Cria o mixer (volume geral, eco) e o buffer de ruído
`Music.start(key, force)`	Começa o tema de um capítulo (ou do título)
`Music.stop(quick)`	Para a música e o bordão com um fade
`Music.compose(th, variant)`	Compõe a peça A A B A' com baixo e percussão
`Music.degMidi(th, deg)`	Converte um grau da escala numa nota MIDI
`Music.tick()`	Agendador: programa as notas dos próximos 0,35 s
`Music.play(e, when)`	Toca um evento (melodia, baixo ou percussão)
`Music.freq(m)`	Nota MIDI para frequência em Hz
`Music.note(inst, midi, when, dur, vol)`	Toca uma nota no instrumento pedido
`Music.pluck(...)`	Alaúde (Karplus-Strong)
`Music.wind(...)`	Flauta, flauta doce e charamela
`Music.startDrone(th)`	Bordão contínuo
`Music.drum(k, when)`	Percussão

Glossário

Termo	Significado
API	Conjunto de funções que um sistema oferece para outros programas usarem (ex.: a API do Canvas)
Array	Lista ordenada de valores
BFS (busca em largura)	Algoritmo que explora um mapa "em ondas" a partir de um ponto; usado para saber o que é alcançável
Biblioteca	Código pronto, escrito por terceiros, que você chama
Bioma	Conjunto de cores e obstáculos que dá o "clima" de um mapa
Cache	Guardar um resultado para não precisar calculá-lo de novo
Callback	Função passada para ser chamada depois, quando algo acontecer
Canvas	Elemento HTML onde se desenha pixel a pixel
Chunk	Bloco de um mapa grande, processado ou desenhado separadamente
Classe	Molde para criar objetos parecidos (`new`)
Closure	Função que "lembra" variáveis do lugar onde foi criada
Commit	"Foto" do projeto guardada pelo Git
Console	Painel do navegador para mensagens, erros e testes
Design orientado a dados (data-driven)	Conteúdo descrito em dados e executado por um motor genérico
DOM	A árvore de elementos de uma página, que o JavaScript manipula
dt (delta time)	Tempo entre dois quadros, usado para que tudo ande na mesma velocidade em qualquer computador
Envelope	Curva de volume de um som ao longo do tempo
Evento	Algo que acontece (tecla, clique) e que pode disparar uma função
FPS	Quadros por segundo
Framework	Estrutura que organiza seu programa e chama o seu código
Função	Bloco de instruções com nome, que pode receber parâmetros e devolver um resultado
Game loop	O laço ler entrada → atualizar → desenhar, repetido continuamente
Global	Variável visível em todos os scripts da página
HUD	Informações sobrepostas ao jogo (vida, XP, missões)

Termo	Significado
i18n	Internacionalização: preparar o programa para vários idiomas
IIFE	Função criada e executada imediatamente: `(function () { ... })();`
JSON	Formato de texto para representar objetos e listas
Karplus-Strong	Algoritmo que simula uma corda dedilhada a partir de ruído
Knockback	Empurrão aplicado a quem leva um golpe
Lerp	Interpolação linear entre dois valores
localStorage	Armazenamento de textos no navegador, permanente
Máquina de estados	Modelo em que algo está sempre num estado e muda por regras
Método	Função que pertence a um objeto
MIDI (número)	Número de uma nota musical (60 = dó central)
Modo (musical)	Tipo de escala (dórico, frígio...) que dá o "clima" da música
Normalizar (vetor)	Deixar um vetor com tamanho 1, mantendo a direção
Objeto	Conjunto de valores com nomes (chaves)
Procedural	Gerado por algoritmo em vez de feito à mão
Radiano	Unidade de ângulo dos computadores (uma volta = 2π)
Recursão	Função que chama a si mesma
Ruído (noise)	Valores pseudoaleatórios suaves, usados para formas naturais
Semente (seed)	Número que define uma sequência pseudoaleatória sempre igual
Sprite	Imagem 2D de um personagem ou objeto
Telegraph	Aviso visual antes de um ataque, para o jogador reagir
Throttling	Limitar a frequência com que algo é executado
Tile	Quadradinho de uma grade de mapa
Vetor	Par (dx, dy) que representa direção e tamanho
Web Audio API	Sistema de som do navegador, baseado em nós conectados
XSS	Ataque que injeta código numa página por meio de texto não escapado
Y-sorting	Desenhar os personagens na ordem da posição vertical para simular profundidade

Palavras finais

Você chegou ao fim. Se leu com atenção e fez os exercícios, já sabe:

- ler e escrever JavaScript, HTML e CSS;
- estruturar um jogo com um laço, estados e entidades;
- desenhar, animar e mover coisas na tela com matemática simples;
- gerar mundos, detectar colisões e programar inimigos que parecem pensar;
- separar dados de lógica, salvar progresso, traduzir, tocar som e música;
- testar, depurar e publicar na internet.

Isso é programar. O resto é prática, curiosidade e paciência. Todo programador, sem exceção, começou sem saber nada e passou muitas horas olhando para um erro no Console até entender. A diferença entre quem aprende e quem desiste é só continuar.

Boa sorte nos seus próximos jogos. E que a sua linhagem, como a de Íber, vá longe.